



PRACTICA 06: MANEJO DE MAPAS DE CALOR GEOREFERENCIABLES EN EL ANÁLISIS EDA



Institución: Universidad Tecnológica de Xicotepec de Juárez

Materia: Extracción del Conocimiento de Bases de Datos

Docente: M.T.I Marco Antonio Ramírez Hernández

Alumno: Jonathan Baldemar Ramírez Reyes

Un viaje visual y revelador con DonorsChoose

DonorsChoose.org
Support a classroom. Build a future.

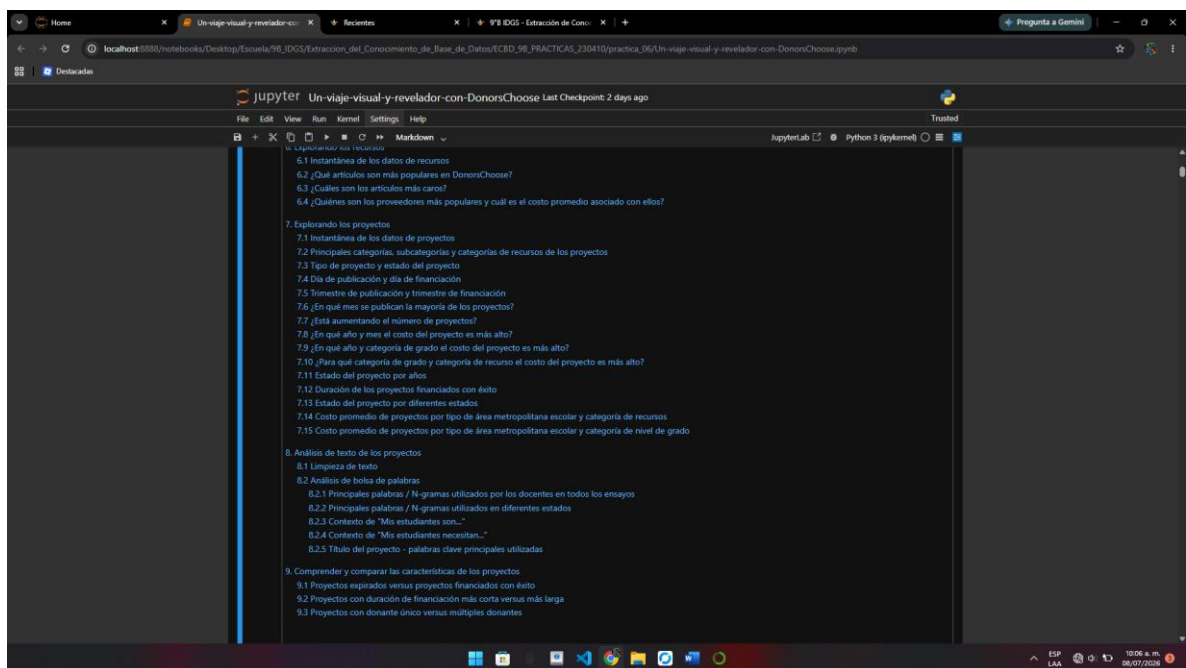
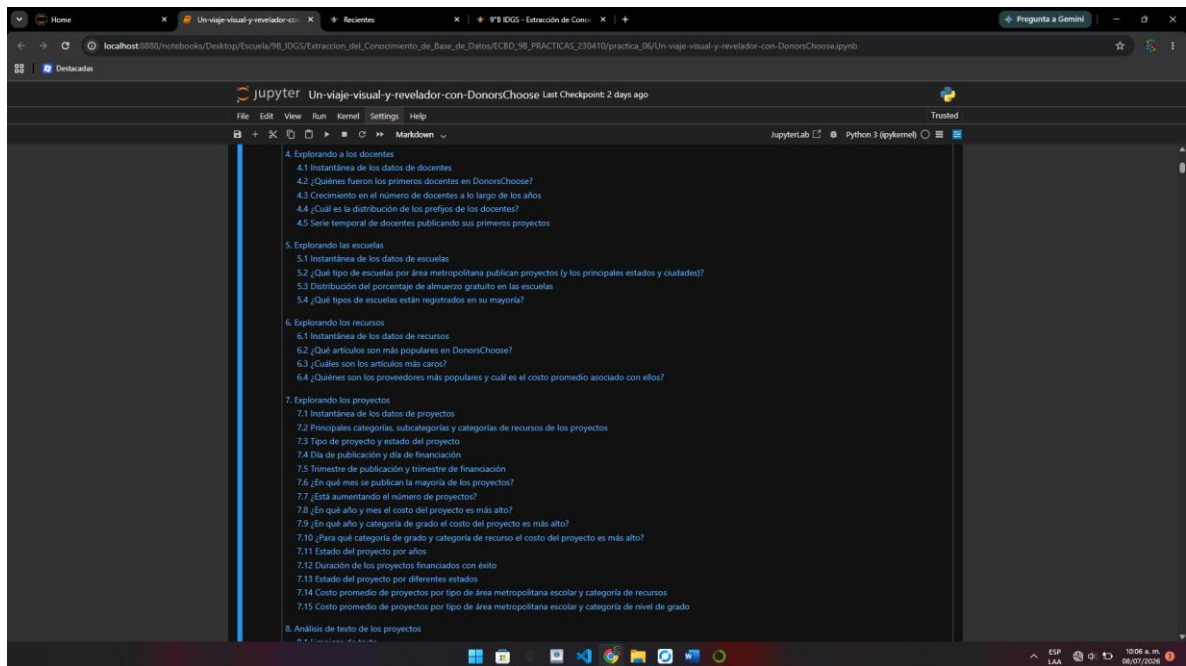
Un viaje visual y revelador - DonorsChoose

Ciencia de datos para el bien: DonorsChoose

DonorsChoose es una organización sin fines de lucro con sede en Estados Unidos que permite a particulares donar directamente a proyectos para aulas de escuelas públicas. Para enero de 2018, DonorsChoose.org había financiado un millón de proyectos; de hecho, el 77 % de las escuelas públicas de Estados Unidos ha solicitado al menos un proyecto a través de esta plataforma. DonorsChoose ha compartido un conjunto de datos completo que incluye todos los proyectos anteriores y los recursos asociados, así como metadatos sobre docentes, escuelas, donantes y donaciones. Estos datos permiten obtener información valiosa y desarrollar una solución que posibilite a DonorsChoose.org crear campañas de correo electrónico dirigidas, recomendando solicitudes específicas de aulas a donantes anteriores. En este notebook, he realizado un análisis y una exploración exhaustivos del conjunto de datos compartido. Este documento no solo aporta información clave derivada de los datos de DonorsChoose, sino que también sirve como tutorial para llevar a cabo análisis de datos (incluido el análisis exploratorio) y visualizaciones de datos.

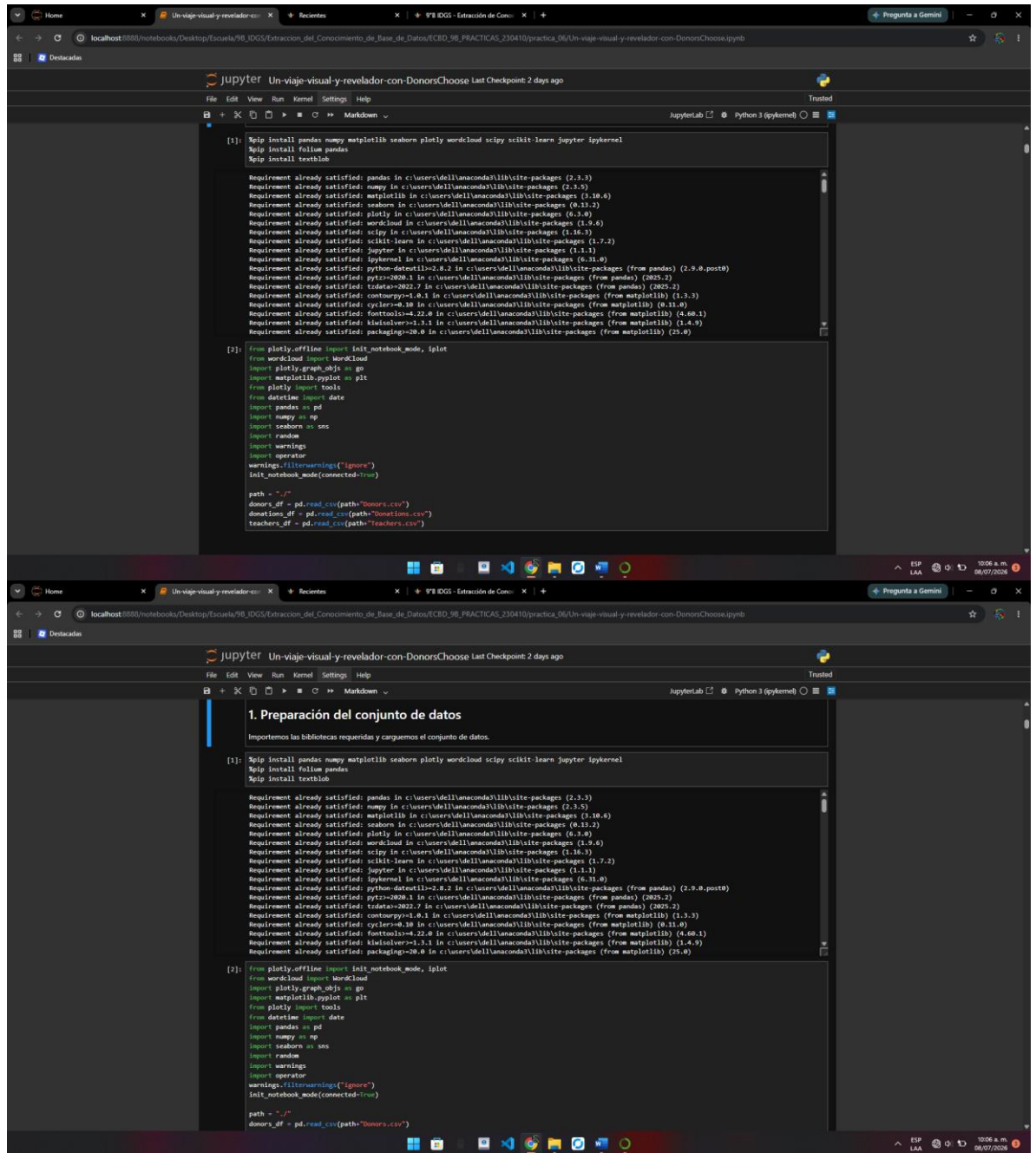
Contenido

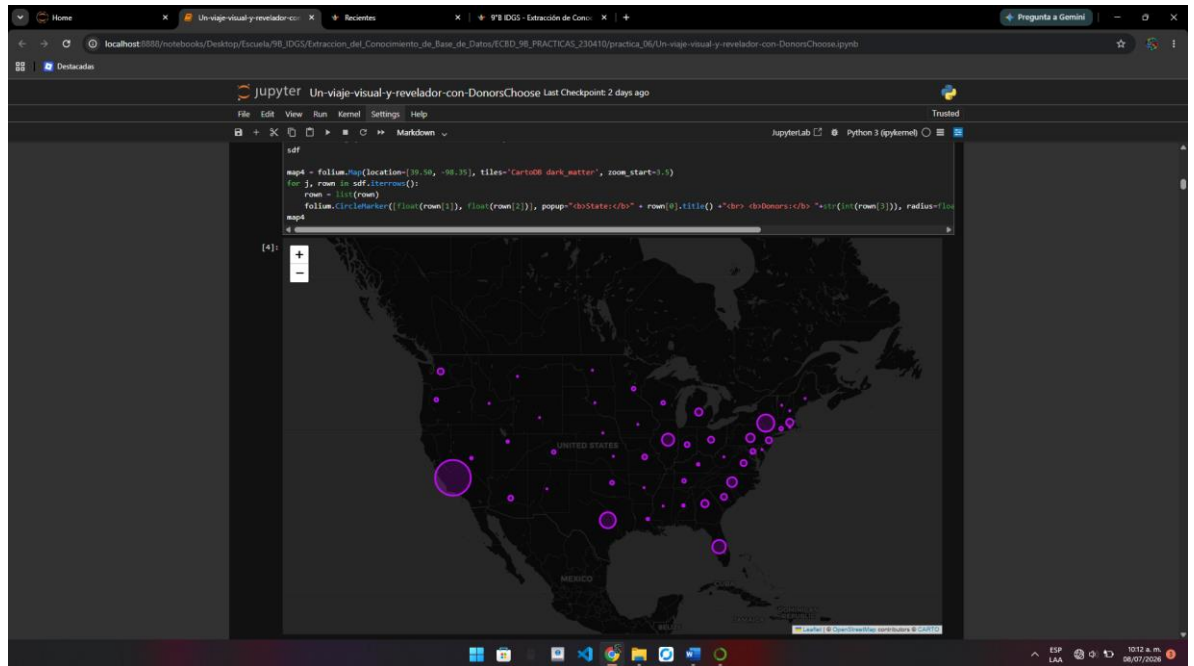
- Preparación del conjunto de datos
- Explorando a los donantes
 - Resumen de donantes
 - Donantes de diferentes estados
 - Estados con la mayor proporción de donantes por habitante
 - Visualizando dónde se encuentran los donantes
 - Dónde se encuentran los donantes en California
 - Dónde se encuentran los donantes en Florida
 - Dónde se encuentran los donantes en Nueva York
 - Dónde se encuentran los donantes en Texas
 - Dónde se encuentran los donantes
 - Donantes docentes y no docentes en diferentes estados
- Explorando las donaciones
 - Instantánea de las donaciones
 - Proyectos y sus donaciones
 - Proyectos con mayor número de donantes
 - Proyectos con los montos totales financiados más altos
 - Proyectos con los montos promedio más altos por donante
 - Proyectos con el monto donado único más alto
 - Donantes y sus donaciones
 - Donantes que han realizado el mayor número de donaciones
 - Donantes que han financiado los montos totales más altos
 - Donantes que financian los montos promedio más altos en donaciones individuales
 - Donantes que han financiado el monto donado único más alto
 - Montos de donación por año, mes, día, hora
 - Donaciones opcionales y donaciones de docentes donantes
 - Número y monto promedio de donación, donaciones opcionales
 - Número y monto promedio de donación, docentes donantes
 - ¿Qué estado ha recibido el monto de donación más alto?
 - ¿Qué estado recibe los montos de donación promedio más altos?
 - Estados y el total de donaciones recibidas
 - Donaciones desde California a lo largo del tiempo



1. Preparación del conjunto de datos

Configuración del entorno: instalación de librerías y carga de los conjuntos de datos con `pandas.read_csv()`.





2. Resumen de donantes

Vista general del conjunto de datos mediante `head()`, `info()` y `value_counts()` para conocer la estructura de la información.

2. Explorando los Donantes

El conjunto de datos de donantes contiene información sobre cada donante que ha donado al menos una vez en DonorsChoose, como su ciudad, estado, código postal y condición de docente.

2.1 Vista rápida de los Donantes

Echemos un vistazo a la vista previa de las primeras filas del conjunto de datos de donantes.

```
[3]: ##### total rows
# donors_df.shape

##### total cities
# len(donors_df['Donor City'].value_counts())

# more than one donation
# t = donors_df['Donor ID'].value_counts().to_frame()
# len(t[t['Donor ID'] > 1])

donors_df.head(3)
```

	Donor ID	Donor City	Donor State	Donor Is Teacher	Donor Zip
0	00000ce45cd0b3f068c992f369d4	Evanston	Illinois	No	602
1	00002783bc5d10d510f9f966c8b1edd	Appomattox	other	No	245
2	00003d44003e4d066607c455a999a	Winton	California	Yes	953

Hasta abril de 2018, DonorsChoose contaba con un total de **2.122.640** donantes registrados. Estos donantes provienen de **15.204 ciudades diferentes** de Estados Unidos y de algunos otros países. De esta gran cantidad de donantes, solo **552.941 (es decir, alrededor del 26 %)** han realizado más de una donación. Uno de los objetivos principales del desafío Data Science for Good es hallar información relevante o recomendaciones que inspiren a los donantes a volver a contribuir. Dado que hay aproximadamente 1.471.613 donantes que han realizado una única donación, DonorsChoose tiene la oportunidad de motivar a estas personas to make another payment.

2.2 Donantes de cada estado

2.2.1 Número de donantes por estado

3. Donantes de diferentes estados

Conteo de donantes por estado y ciudad utilizando `pandas` y visualización mediante gráficos de barras en `Plotly`.

```
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago
File Edit View Run Kernel Settings Help
Python 3 (ipykernel)

2.2 Donantes de cada estado
2.2.1 Número de donantes por estado

[4]:
import folium
from folium import plugins
from IPython import StringIO
import folium

stateall=StringIO("""State,Latitude,Longitude
Alabama,32.806671,-86.701130
Alaska,63.370776,-152.404419
Arizona,33.229759,-111.431221
Arkansas,34.969704,-92.273123
California,36.184649,-118.481504
Colorado,39.409811,-105.311104
Connecticut,41.597782,-72.755371
Delaware,39.138523,-75.507341
District of Columbia,38.907193,-77.458117
Florida,27.766279,-81.488783
Georgia,33.040819,-83.643074
Hawaii,19.941118,-155.480117
Idaho,44.244859,-114.478828
Illinois,40.344857,-88.588137
Indiana,39.494426,-86.258228
Iowa,42.081539,-93.238526
Kansas,38.526600,-96.726488
Kentucky,37.653586,-84.470687
Louisiana,31.180546,-91.067805
Maine,44.693947,-69.381927
Maryland,39.063946,-76.782181
Massachusetts,42.280171,-71.528046
Michigan,43.126618,-84.536895
Minnesota,45.494454,-93.480832
Mississippi,32.741646,-89.479696
Missouri,38.456885,-92.288368
Montana,46.922923,-110.454353
Nebraska,41.253786,-96.264802
Nevada,38.315151,-117.495374
New Hampshire,43.452492,-71.563896
New Jersey,40.290064,-74.521811
```

```
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago
File Edit View Run Kernel Settings Help
Python 3 (ipykernel)

New Hampshire,43.452492,-71.563896
New Jersey,40.290064,-74.521811
New Mexico,34.849515,-106.246482
New York,42.165726,-74.944851
North Carolina,35.610665,-79.486119
North Dakota,47.228932,-99.744812
Ohio,40.388783,-82.764915
Oklahoma,35.145342,-96.828917
Oregon,44.573821,-122.470938
Pennsylvania,40.508752,-77.209795
Rhode Island,41.680893,-71.511788
South Carolina,33.948892,-80.940007
South Dakota,44.299782,-99.438828
Tennessee,35.747845,-86.492345
Texas,31.804687,-97.365861
Utah,40.150031,-111.862834
Vermont,44.045876,-72.718888
Virginia,37.769337,-78.169668
Washington,47.406061,-121.480448
West Virginia,38.493226,-80.954453
Wisconsin,44.268543,-89.635588
Wyoming,42.725868,-107.304907")

tempdf = states_df[['Donor State']].value_counts()
ti = pd.Series()
ti['Donor State'] = tempdf.index
ti['Donor Count'] = tempdf.values

# tempdf = combined_df.groupby(['Donor State']).agg(['Donation Amount'].sum).reset_index()
# ti = tempdf.reset_index('Donation Amount', ascending=False)

sdf = pd.read_csv(stateall, columns=['State','Donor State'])
sdf = sdf.merge(ti, on='Donor State', how='inner')
sdf

map4 = folium.Map(location=[39.58,-98.35], tiles='CartoDB dark_matter', zoom_start=1.5)
for j, row in sdf.iterrows():
    row = list(row)
    folium.CircleMarker((float(row[1]), float(row[2])), popup=f'Donations: {row[0]} + row[4], title=f'Donations: {row[4]}', radius=float(row[4]))
map4
```

```

def bar_bar(df, col, title, color, w=None, h=None, lw=0, limit=100, return_trace=False, rev=False):
    cnt_srs = df[col].value_counts()
    yy = cnt_srs.head(limit).index[1:-1]
    xx = cnt_srs.head(limit).values[1:-1]
    if rev:
        yy = cnt_srs.tail(limit).index[1:-1]
        xx = cnt_srs.tail(limit).values[1:-1]

    trace = go.Bar(y=yy, x=xx, orientation='h', marker=dict(color=xx))
    if return_trace:
        return trace
    layout = dict(title=title, margin=dict(l=10), width=w, height=h)
    data = [trace]
    fig = go.Figure(data=data, layout=layout)
    iplot(fig)

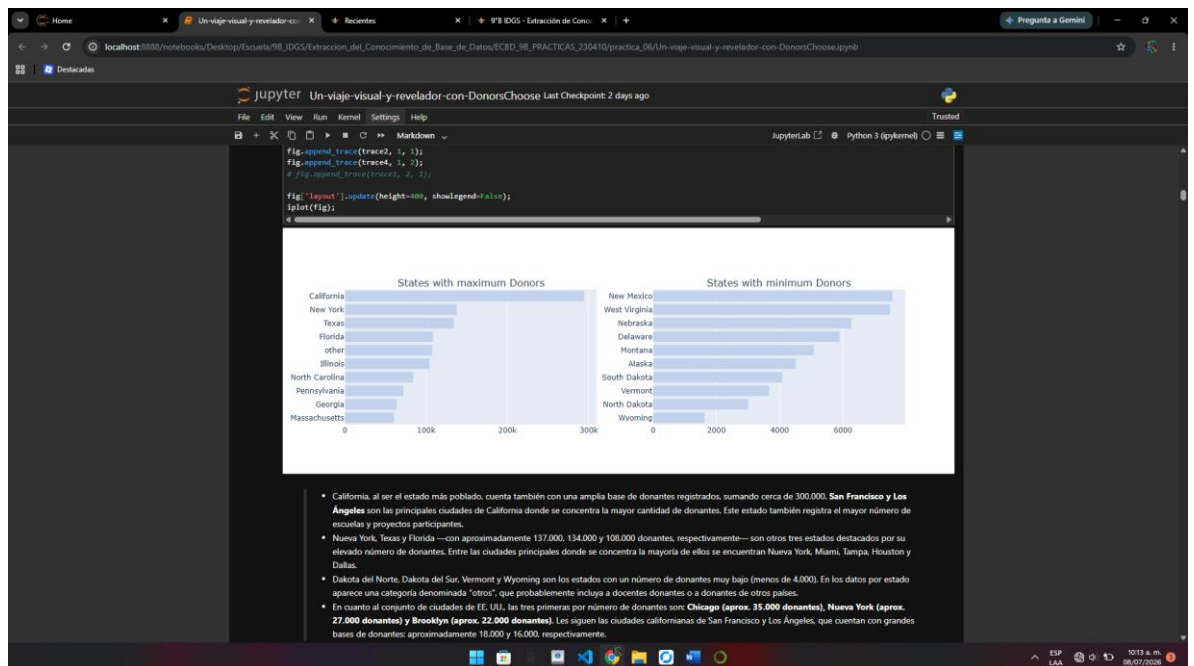
def bar_bar_range(x, y, title, color, w=None, h=None, lw=0, limit=100, rt=False):
    trace = go.Bar(x=x, y=y, orientation='h', marker=dict(color=xx))
    if rt:
        return trace
    layout = dict(title=title, margin=dict(l=10), width=w, height=h)
    data = [trace]
    fig = go.Figure(data=data, layout=layout)
    iplot(fig)

def bar_bar_range(x, y, title, color, w=None, h=None, lw=0, rt=False):
    trace = go.Bar(x=x, y=y, marker=dict(color=xx))
    if rt:
        return trace
    layout = dict(title=title, margin=dict(l=10), width=w, height=h)
    data = [trace]
    fig = go.Figure(data=data, layout=layout)
    iplot(fig)

trace1 = bar_bar(donors_df, "Donor City", "Top Cities with maximum Donors", "#c2d6d0", 500, 400, 200, limit=10, return_trace=True)
trace2 = bar_bar(donors_df, "Donor State", "Top States with maximum Donors", "#c2d6d0", 500, 400, 200, limit=10, return_trace=True)
trace3 = bar_bar(donors_df, "Donor State", "Top States with minimum Donors", "#c2d6d0", 600, 400, 200, limit=10, return_trace=True, rev=True)

fig = tools.make_subplots(rows=2, cols=2, print_grid=False, subplot_titles=["States with maximum Donors", "States with minimum Donors"])
# fig = tools.make_subplots(rows=2, cols=2, specs=[([1], [{"color": "#c2d6d0"}]), [{"color": "#c2d6d0"}]), print_grid=False, subplot_titles=["States with maximum Donors",

```



4. **Estados con la mayor proporción de donantes por habitante**
Cálculo del número de donantes por cada 100,000 habitantes usando `pandas` y representación gráfica en Plotly.

Conjunto de datos ajustado por población

Los estados que encabezaban los gráficos anteriores son, en realidad, aquellos con una población muy elevada. Sería interesante conocer el número de donantes por cada 100.000 habitantes en cada estado. He añadido un conjunto de datos ajustado según la población, que incluye la cifra de habitantes de cada estado correspondiente al año 2013. Population Dataset: <http://www.enchantedlearning.com/usa/states/population.shtml>

2.3 Estados con las cifras más altas Relación entre donantes y población

```
[4]: census_2013 = {'Mississippi': 2991267, 'Iowa': 3000416, 'Oklahoma': 3859668, 'Delaware': 925749, 'Minnesota': 5428086, 'Alaska': 739132, 'Illinois': 12688323}

donors_from_states = dict(zip(donors_df['Donor State'], value_counts()))

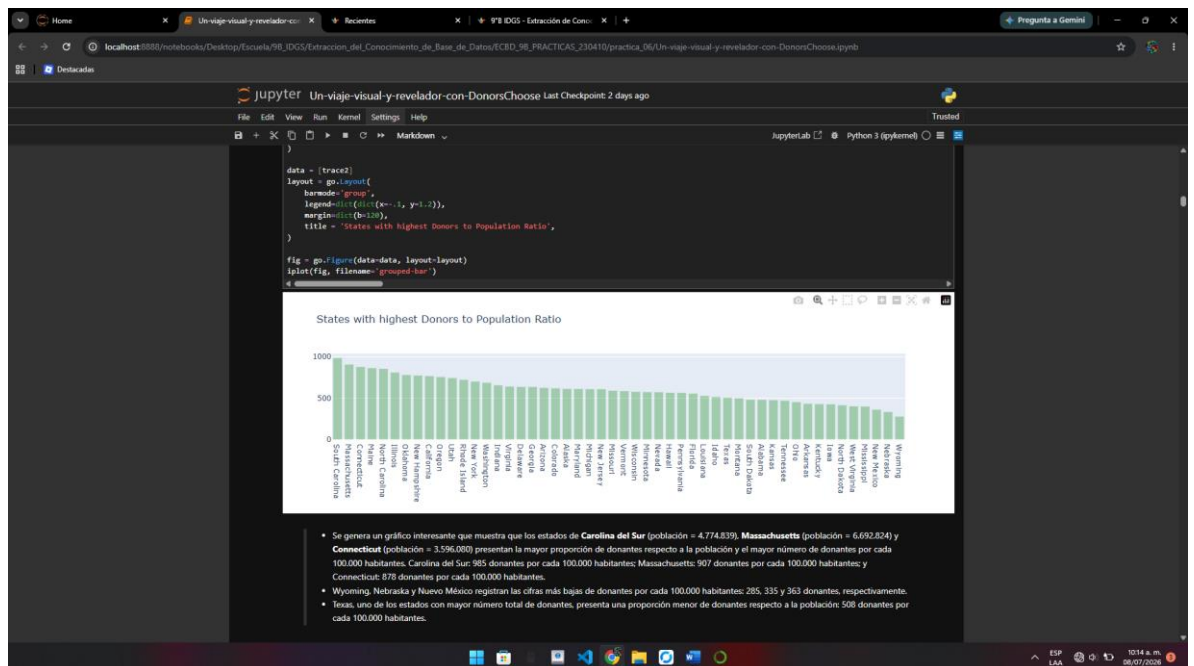
don_pop = {}
for state, don in donors_from_states.items():
    if state not in census_2013:
        continue
    don_pop[state] = float(don)*100000/census_2013[state]

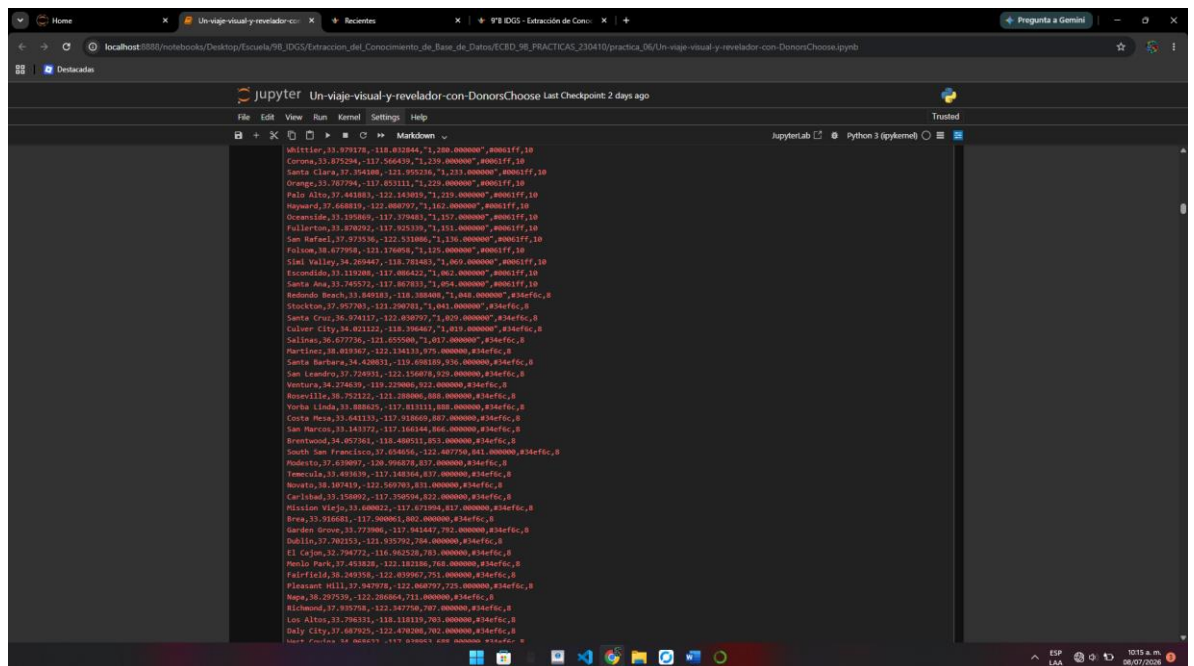
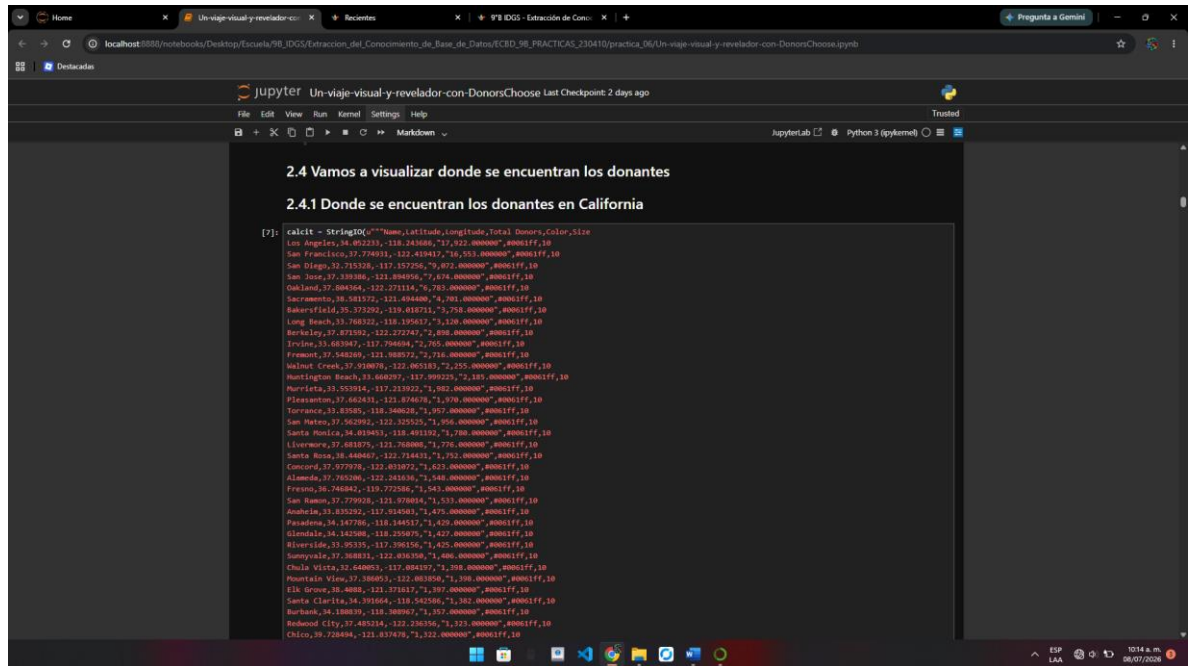
import operator
don_pop = sorted(don_pop.items(), key=operator.itemgetter(1), reverse=True)
xx = [x[0] for x in don_pop][0:11]
yy = [x[1] for x in don_pop][0:11]

trace2 = go.Bar(
    x=xx,
    y=yy,
    name='Donors to Population',
    marker=dict(color='green'),
    opacity=0.3
)

data = [trace2]
layout = go.Layout(
    bargroups='group',
    legend=dict(x=-1, y=1.2)),
    margin=dict(b=100),
    title = 'States with highest Donors to Population Ratio',
)

fig = go.Figure(data=data, layout=layout)
plot(fig, filename='grouped-bar')
```





```
Home | Un-viaje-visual-y-revelador-con-DonorsChoose | Recientes | V9 IDG5 - Extracción de Cono... | Preguntar a Gemini
localhost:8888/notebooks/Desktop/Escuela/98_IDG5/Extraccion_de_Base_de_Datos/ECID_98_PRACTICAS_230410/practica_06/Un-viaje-visual-y-revelador-con-DonorsChoose.ipynb
Detección

jupyter Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago
File Edit View Run Kernel Settings Help
Python 3 (ipykernel)

Daily City,37.687925,-122.478288,762.000000,#34efcc,8
West Covina,34.068822,-122.938953,688.000000,#34efcc,8
Lafayette,37.885758,-122.118051,668.000000,#34efcc,8
La Mesa,32.767823,-117.023883,649.000000,#34efcc,8
Milpitas,37.428272,-121.986825,649.000000,#34efcc,8
Carson,33.831486,-118.283817,647.000000,#34efcc,8
Camelio,34.216294,-118.027881,643.000000,#34efcc,8
Petaluma,38.234417,-122.636853,629.000000,#34efcc,8
Newport Beach,33.618911,-117.928847,626.000000,#34efcc,8
Pacific,37.433825,-122.488819,619.000000,#34efcc,8
Gillroy,37.080193,-121.548825,618.000000,#34efcc,8
Encinitas,33.876888,-117.291381,607.000000,#34efcc,8
El Segundo,33.919181,-118.416484,595.000000,#34efcc,8
Downey,33.944814,-118.117849,592.000000,#34efcc,8
Rancho Palos Verdes,33.744461,-118.387817,592.000000,#34efcc,8
San Clemente,33.428872,-117.611992,589.000000,#34efcc,8
Vacaville,38.366878,-122.967764,584.000000,#34efcc,8
Chino Hills,33.980859,-117.732586,583.000000,#34efcc,8
Campbell,37.287184,-121.949958,580.000000,#34efcc,8
Mill Valley,37.964678,-122.548975,578.000000,#34efcc,8
El Cerrillo,37.918133,-112.118764,576.000000,#f2ae93,6
Emeryville,37.811317,-122.285247,572.000000,#f2ae93,6
Thousand Oaks,34.187731,-118.877847,572.000000,#f2ae93,6
Burlington,51.541818,-112.948885,538.000000,#f2ae93,6
Antioch,38.084922,-121.889789,568.000000,#f2ae93,6
Beverly Hills,34.073615,-118.408356,568.000000,#f2ae93,6
Lakewood,33.857828,-118.137095,568.000000,#f2ae93,6
Manhattan Beach,33.884736,-118.418888,557.000000,#f2ae93,6
Lancaster,34.688788,-118.543164,551.000000,#f2ae93,6
Oxnard,34.197386,-119.177095,539.000000,#f2ae93,6
San Carlos,37.587138,-112.348825,527.000000,#f2ae93,6
Rocklin,38.788733,-121.235783,526.000000,#f2ae93,6
Placentia,33.872236,-117.878336,526.000000,#f2ae93,6
Cupertino,37.322887,-122.821881,516.000000,#f2ae93,6
Tustin,33.745885,-117.828367,508.000000,#f2ae93,6
Hermosa Beach,33.862236,-118.399519,507.000000,#f2ae93,6
Rancho Cordova,38.588872,-121.987728,501.000000,#f2ae93,6
Fountain Valley,33.789186,-117.953868,498.000000,#f2ae93,6
Cypress,37.128931,-121.962242,497.000000,#f2ae93,6
Union City,37.593878,-122.843817,494.000000,#f2ae93,6
Morgan Hill,37.11305,-121.854385,488.000000,#f2ae93,6
Vallejo,38.184888,-122.256336,485.000000,#f2ae93,6
Alhambra,34.095288,-118.127814,474.000000,#f2ae93,6
Lake Forest,33.646887,-117.689217,465.000000,#f2ae93,6
```

```
Home | Un-viaje-visual-y-revelador-con-DonorsChoose | Recientes | V9 IDG5 - Extracción de Cono... | Preguntar a Gemini
localhost:8888/notebooks/Desktop/Escuela/98_IDG5/Extraccion_de_Base_de_Datos/ECID_98_PRACTICAS_230410/practica_06/Un-viaje-visual-y-revelador-con-DonorsChoose.ipynb
Detección

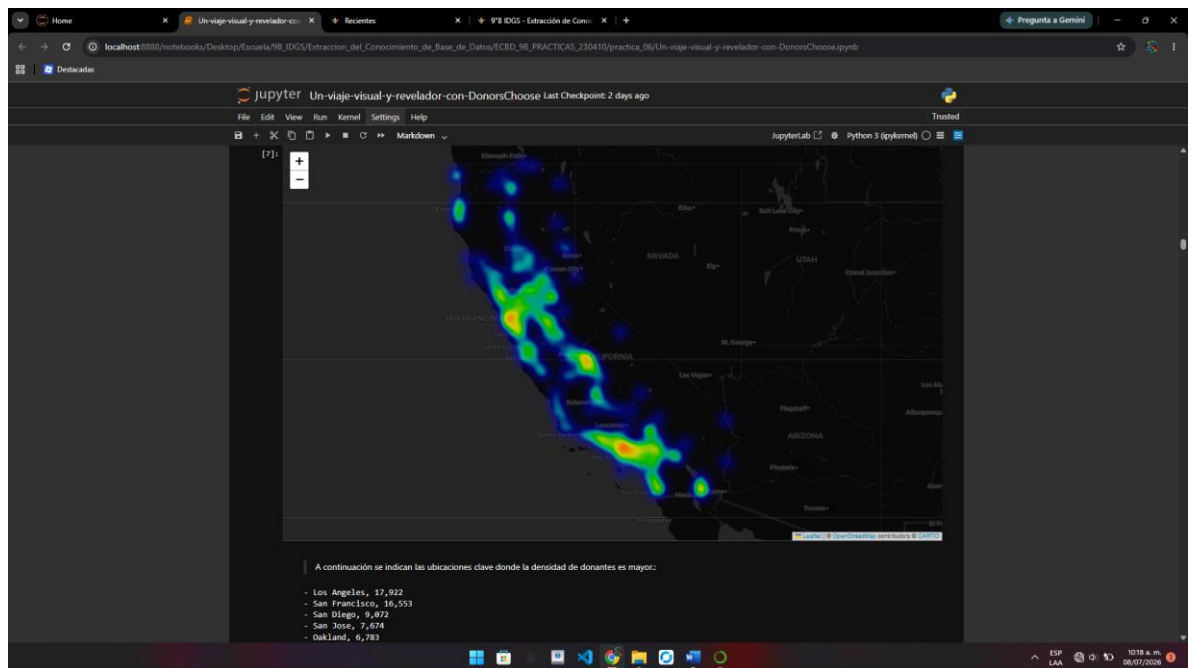
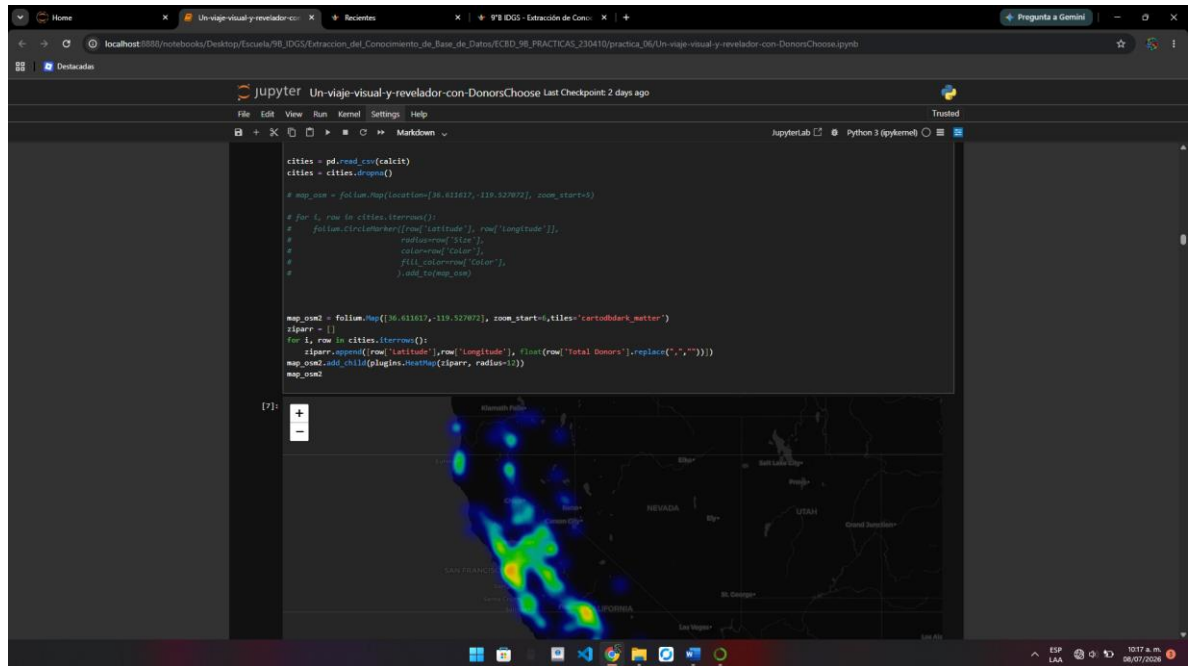
jupyter Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago
File Edit View Run Kernel Settings Help
Python 3 (ipykernel)

Lake Forest,33.646887,-117.689217,465.000000,#f2ae93,6
Rancho Cucamonga,34.18824,-117.591188,463.000000,#f2ae93,6
Vista,33.208895,-112.242536,461.000000,#f2ae93,6
Fontana,34.092233,-117.439847,459.000000,#f2ae93,6
Moreno Valley,33.942867,-117.229672,455.000000,#f2ae93,6
Hawthorne,33.516483,-118.192575,450.000000,#f2ae93,6
Covina,34.088888,-117.888139,441.000000,#f2ae93,6
San Luis Obispo,35.282783,-120.659817,434.000000,#f2ae93,6
Newport,37.528888,-122.688219,411.000000,#f2ae93,6
Ingleside,33.981881,-118.781112,411.000000,#f2ae93,6
Oakley,37.997422,-121.712453,410.000000,#f2ae93,6
Tracy,37.739885,-121.425222,408.000000,#f2ae93,6
Davis,38.644886,-121.748517,412.000000,#f2ae93,6
Redding,40.588539,-122.391879,404.000000,#f2ae93,6
Glendora,34.136119,-117.865339,423.000000,#f2ae93,6
San Bruno,37.638489,-122.411883,423.000000,#f2ae93,6
Davis,38.644886,-121.748517,412.000000,#f2ae93,6
Westminster,33.721362,-117.925292,413.000000,#f2ae93,6
Pittsburg,38.827975,-121.894882,412.000000,#f2ae93,6
Palmdale,34.129433,-118.136481,408.000000,#f2ae93,6
Vicksburg,36.330278,-118.287889,405.000000,#f2ae93,6
Paway,32.962822,-117.858884,401.000000,#f2ae93,6
Santa Maria,34.953833,-120.435719,401.000000,#f2ae93,6
Upland,34.097811,-117.688188,398.000000,#f2ae93,6
Laguna Niguel,33.525252,-117.787553,393.000000,#f2ae93,6
La Mirada,33.917238,-118.812888,392.000000,#f2ae93,6
South Pasadena,34.118117,-118.188118,391.000000,#f2ae93,6
Cerritos,33.893847,-118.864786,381.000000,#f2ae93,6
Lodi,38.134147,-121.772219,380.000000,#f2ae93,6
Ontario,34.063344,-117.158889,380.000000,#f2ae93,6
Lincoln,38.881564,-121.283888,378.000000,#f2ae93,6
Buena Park,33.867514,-117.908117,375.000000,#f2ae93,6
Rancho Santa Margarita,33.848856,-117.681881,373.000000,#f2ae93,6
Santer,32.838181,-118.979127,372.000000,#f2ae93,6
Sonoma,38.251853,-122.458836,372.000000,#f2ae93,6
Menlo Park,34.144428,-118.081347,371.000000,#f2ae93,6
Alhambra,34.088888,-122.787777,368.000000,#f2ae93,6
La Habra,33.919181,-121.948127,366.000000,#f2ae93,6
San Pablo,37.962147,-122.345525,366.000000,#f2ae93,6
Belmont,37.518814,-122.278889,364.000000,#f2ae93,6
Pomona,34.091228,-117.761786,364.000000,#f2ae93,6
Gardena,33.88815,-118.88815,361.000000,#f2ae93,6
Arcadia,34.139728,-118.851344,361.000000,#f2ae93,6
Clovis,36.821228,-119.787817,358.000000,#f2ae93,6
Orinda,37.877147,-122.179889,358.000000,#f2ae93,6
```



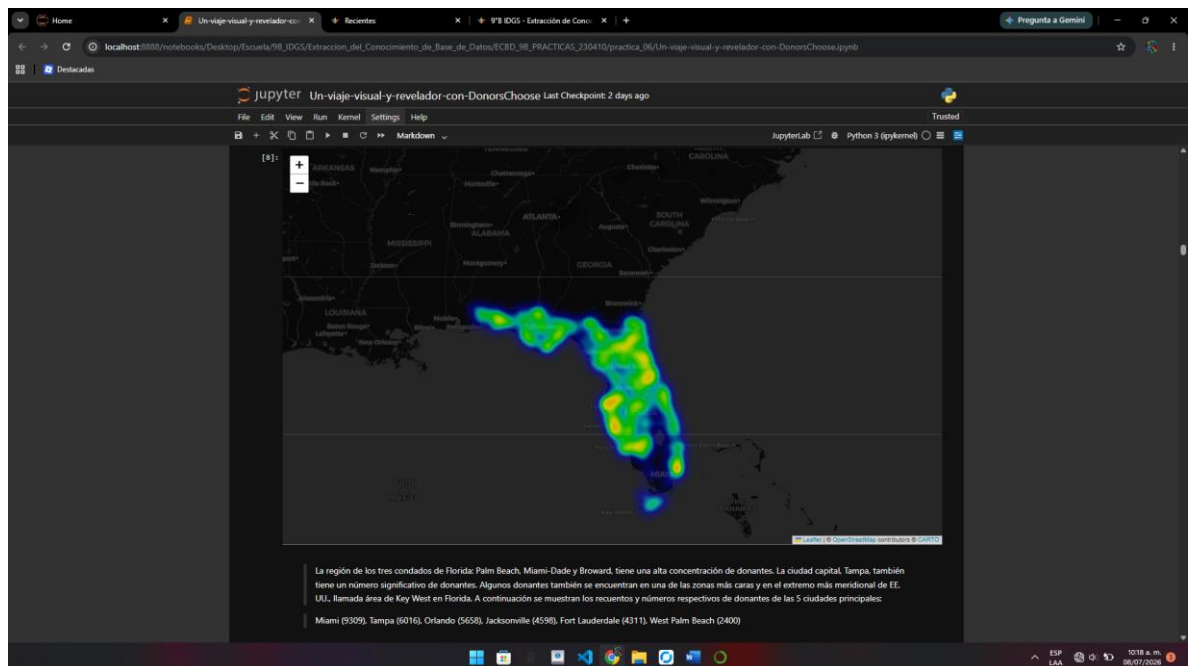
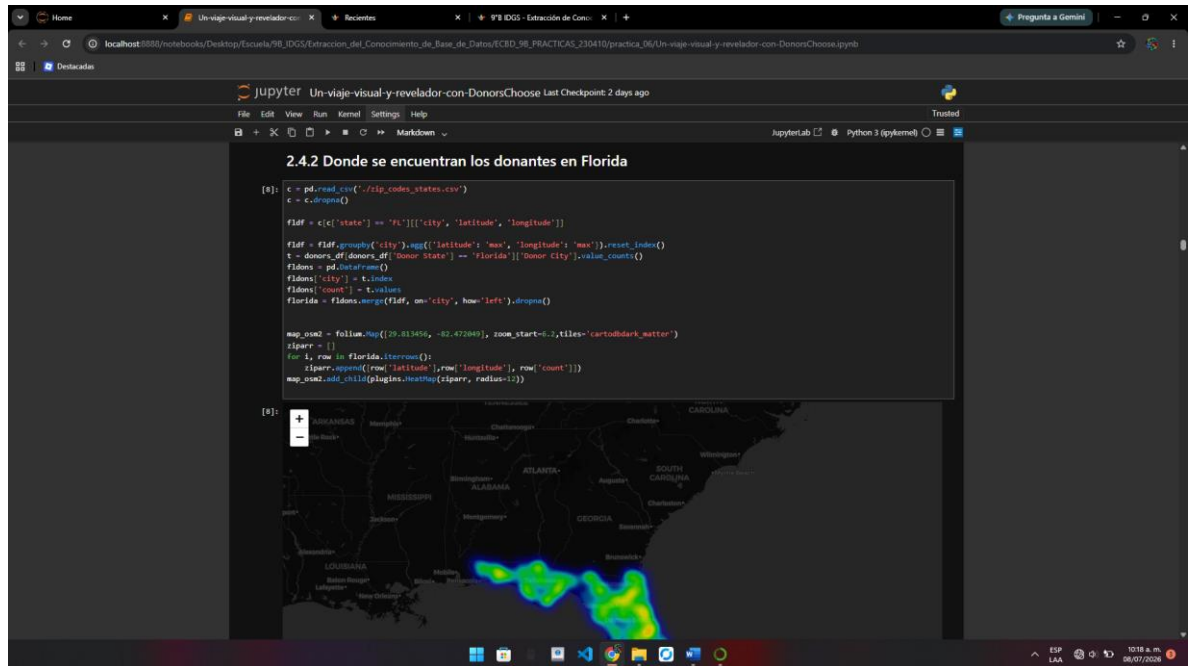
```
Home | Un-viaje-visual-y-revelador-con-DonorsChoose | Recientes | V9 IDGS - Extracción de Cono... | Preguntas a Gemini
localhost:8888/notebooks/Desktop/Escuela/98_IDGS/Extracción_de_Base_de_Datos/ECID_98_PRACTICAS_230410/practica_06/Un-viaje-visual-y-revelador-con-DonorsChoose.ipynb
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)
Panteca,37.797428,-123.216853,357.000000,rtab76,4
Citrus Heights,38.707125,-121.261861,355.000000,rtab76,4
Redlands,34.055549,-117.525379,353.000000,rtab76,4
Aliso Viejo,33.575,-117.72555,345.000000,rtab76,4
Eureka,40.802672,-124.163672,338.000000,rtab76,4
West Hollywood,34.090885,-118.361744,337.000000,rtab76,4
Auburn,38.895664,-121.676829,335.000000,rtab76,4
Saratoga,37.263833,-122.023814,331.000000,rtab76,4
Turlock,37.494658,-120.540594,327.000000,rtab76,4
Chico,34.812126,-117.448944,325.000000,rtab76,4
Diamond Bar,34.820622,-117.818136,324.000000,rtab76,4
South Gate,33.954736,-118.213827,319.000000,rtab76,4
San Bernardino,34.108349,-117.287954,318.000000,rtab76,4
Claremont,34.096679,-117.718778,316.000000,rtab76,4
Calabasas,34.138333,-118.668833,298.000000,rtab76,4
Los Alamitos,33.863972,-118.497564,297.000000,rtab76,4
Agoura Hills,34.152339,-118.763679,295.000000,rtab76,4
San Gabriel,34.096111,-118.105833,295.000000,rtab76,4
Palo Alto,35.432278,-120.644186,292.000000,rtab76,4
Normal,33.062236,-118.061233,286.000000,rtab76,4
Indio,33.729878,-116.315941,284.000000,rtab76,4
Pompano,34.144897,-118.208742,284.000000,rtab76,4
Pompano,36.080239,-121.894875,275.000000,rtab76,4
Arroyo Grande,35.118568,-120.589725,274.000000,rtab76,4
San Dimas,34.186679,-117.886725,274.000000,rtab76,4
La Puente,34.420811,-117.949756,273.000000,rtab76,4
West Sacramento,38.580441,-121.382133,272.000000,rtab76,4
Robert Park,38.339636,-122.783897,269.000000,rtab76,4
Pompano Park,34.062111,-118.222847,268.000000,rtab76,4
Laguna Hills,33.599722,-117.690441,268.000000,rtab76,4
Watsonville,36.918218,-121.754894,259.000000,rtab76,4
Clayton,37.948833,-121.935792,256.000000,rtab76,4
Hemet,33.247812,-116.972897,255.000000,rtab76,4
La Verne,34.180842,-117.767836,255.000000,rtab76,4
Ukiah,39.158372,-123.287783,255.000000,rtab76,4
Palm Desert,33.722244,-116.54666,251.000000,rtab76,2
Atascadero,33.428812,-118.470279,241.000000,rtab76,2
Petaluma,33.678333,-117.166944,241.000000,rtab76,2
Longme,34.63835,-120.457942,240.000000,rtab76,2
Mendocino,33.381164,-120.463982,240.000000,rtab76,2
Sebastopol,38.482136,-122.823883,237.000000,rtab76,2
Lake Elsinore,33.668878,-117.127783,236.000000,rtab76,2
Pellissippi,37.599847,-122.882844,234.000000,rtab76,2
Yuba City,39.148847,-121.618911,231.000000,rtab76,2
Louisville,38.258333,-121.618911,231.000000,rtab76,2
```

```
Home | Un-viaje-visual-y-revelador-con-DonorsChoose | Recientes | V9 IDGS - Extracción de Cono... | Preguntas a Gemini
localhost:8888/notebooks/Desktop/Escuela/98_IDGS/Extracción_de_Base_de_Datos/ECID_98_PRACTICAS_230410/practica_06/Un-viaje-visual-y-revelador-con-DonorsChoose.ipynb
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago
File Edit View Run Kernel Settings Help
JupyterLab Python 3 (ipykernel)
Woodland,36.598447,-114.773833,400.000000,rtab76,2
Bellflower,33.861661,-118.127623,227.000000,rtab76,2
Grass Valley,39.239861,-121.062861,227.000000,rtab76,2
Seal Beach,33.741486,-118.184786,227.000000,rtab76,2
Alhambra,34.082829,-118.081539,227.000000,rtab76,2
Martinez,34.016586,-118.113793,225.000000,rtab76,2
Yuba City,39.148847,-114.773833,225.000000,rtab76,2
Hawford,36.32748,-118.645883,224.000000,rtab76,2
Galeto,34.435628,-118.829329,223.000000,rtab76,2
Hesperia,34.426389,-117.388878,221.000000,rtab76,2
Laguna Beach,33.542447,-117.781111,221.000000,rtab76,2
Ridgeway,33.624846,-117.478897,218.000000,rtab76,2
Galt,38.254636,-121.299847,217.000000,rtab76,2
Victorville,34.536186,-117.291538,217.000000,rtab76,2
San Juan Capistrano,33.581892,-117.862859,209.000000,rtab76,2
Atwater,37.547717,-120.680853,208.000000,rtab76,2
Palm Springs,33.838297,-116.545292,208.000000,rtab76,2
Azusa,34.135819,-117.997564,207.000000,rtab76,2
Hercules,38.671744,-122.288581,202.000000,rtab76,2
Sonoma,37.968892,-120.582139,199.000000,rtab76,2
Beaumont,33.928483,-116.977247,198.000000,rtab76,2
Half Moon Bay,37.482853,-122.485564,187.000000,rtab76,2
Hollister,36.952453,-121.485683,156.000000,rtab76,2
Westlake Village,34.145839,-118.885647,154.000000,rtab76,2
Pico Rivera,33.983899,-118.486276,150.000000,rtab76,2
Pismo,38.08367,-122.288581,150.000000,rtab76,2
Compton,33.89585,-118.228672,185.000000,rtab76,2
Sanuality,37.858664,-122.485296,183.000000,rtab76,2
La Quinta,33.644892,-116.318888,181.000000,rtab76,2
Dana Point,33.466972,-117.488188,175.000000,rtab76,2
Rialto,34.1884,-117.378823,172.000000,rtab76,2
Perris,33.742519,-117.228647,171.000000,rtab76,2
Arcata,40.866517,-124.488339,168.000000,rtab76,2
Pacific Grove,36.617738,-123.956622,168.000000,rtab76,2
Lemoore,33.798239,-118.385972,167.000000,rtab76,2
San Fernando,34.281947,-115.458972,165.000000,rtab76,2
South Lake Tahoe,39.339225,-119.977386,163.000000,rtab76,2
El Monte,34.088422,-118.427649,158.000000,rtab76,2
Scotts Valley,37.893861,-121.624683,158.000000,rtab76,2
Tehachapi,35.132189,-118.448897,157.000000,rtab76,2
Baldwin Park,34.082886,-117.968897,156.000000,rtab76,2
Perris,33.742519,-117.228647,156.000000,rtab76,2
Rosemead,34.088422,-118.427649,155.000000,rtab76,2
Placerville,38.728425,-120.788547,151.000000,rtab76,2
Highland,34.128444,-117.288581,148.000000,rtab76,2
```



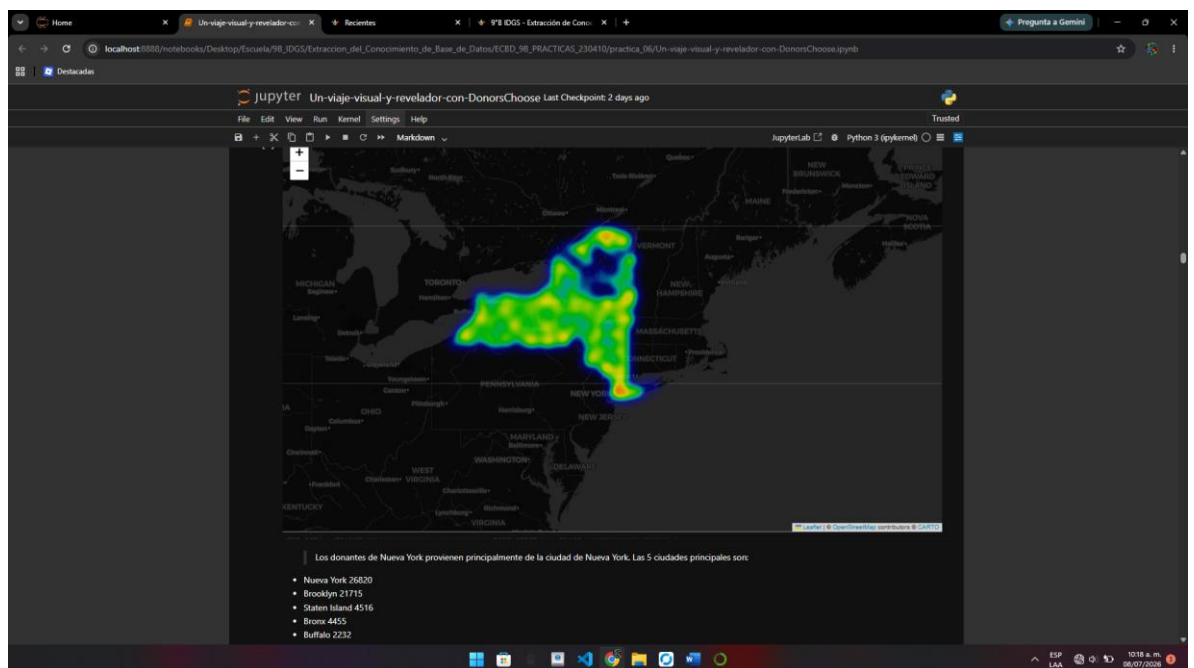
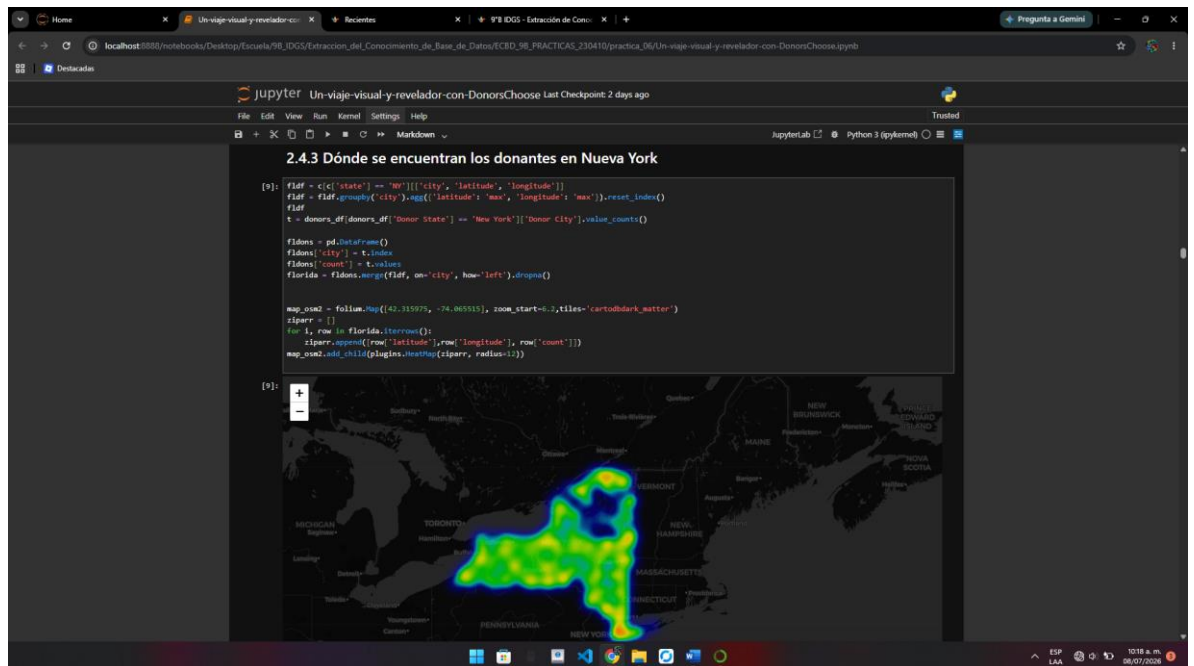
7. Dónde se encuentran los donantes en Florida

Elaboración de un mapa de calor con Folium (`HeatMap`) para visualizar la concentración de donantes en Florida.



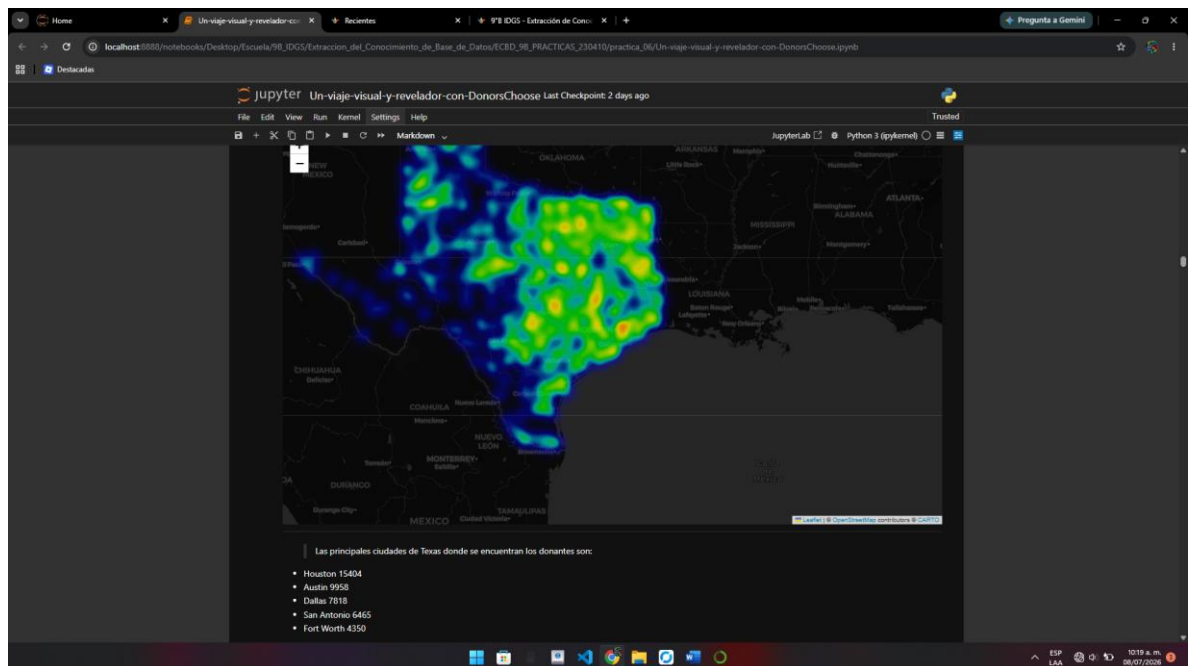
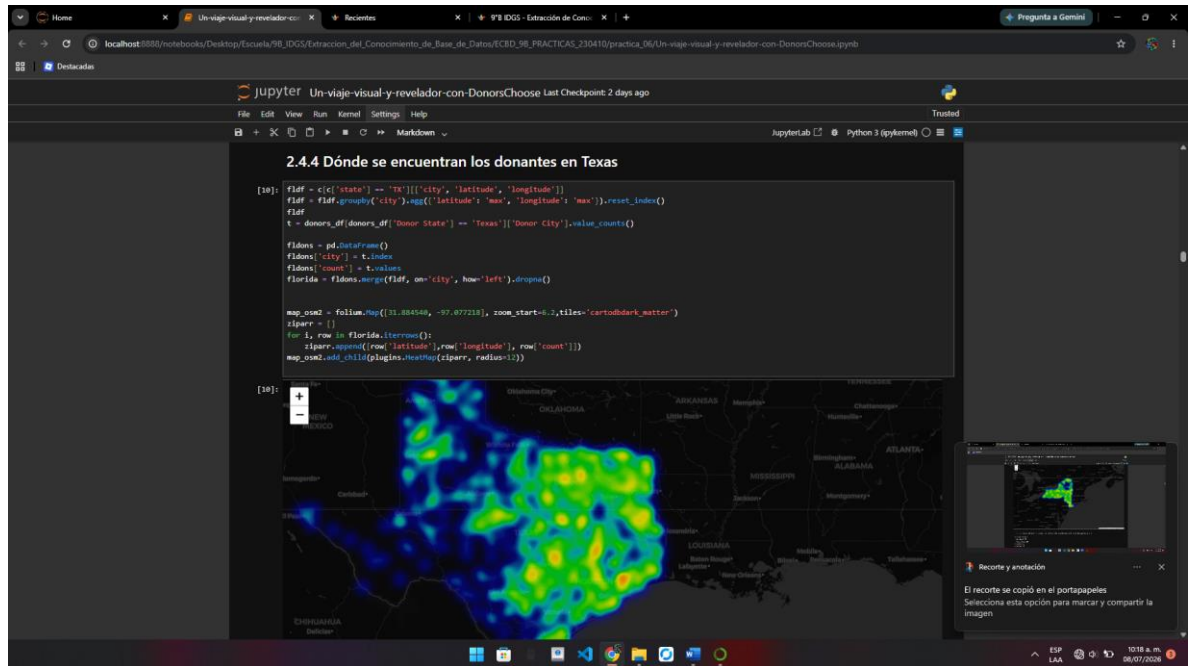
8. Dónde se encuentran los donantes en Nueva York

Elaboración de un mapa de calor con Folium (`HeatMap`) para visualizar la concentración de donantes en Nueva York.



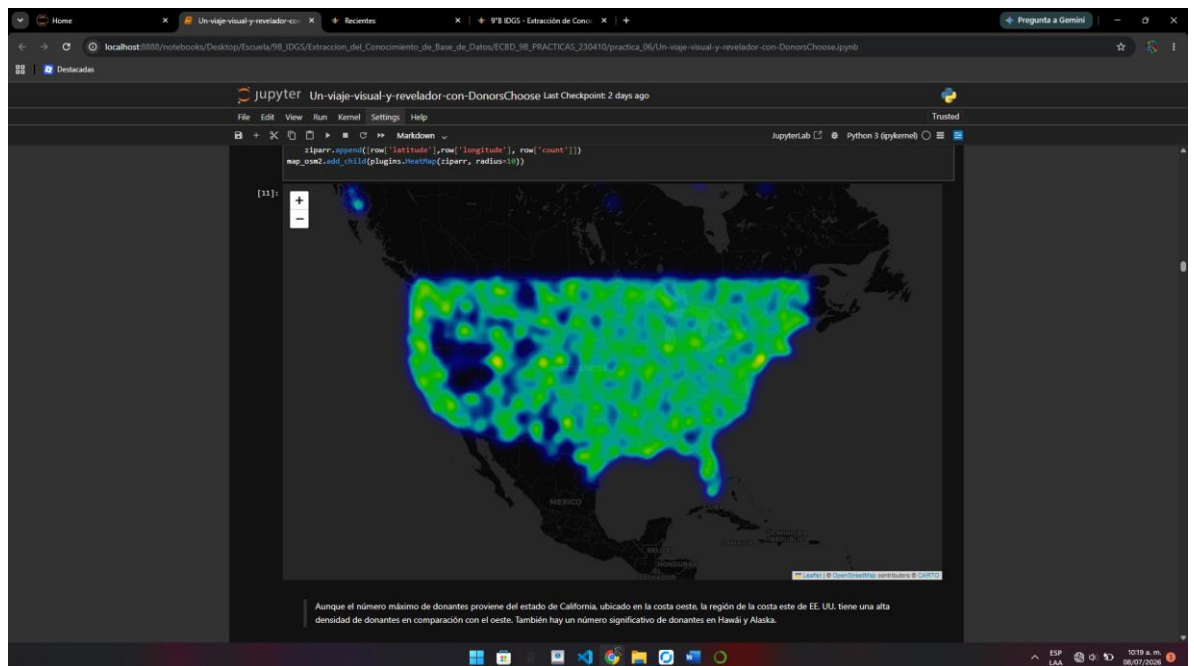
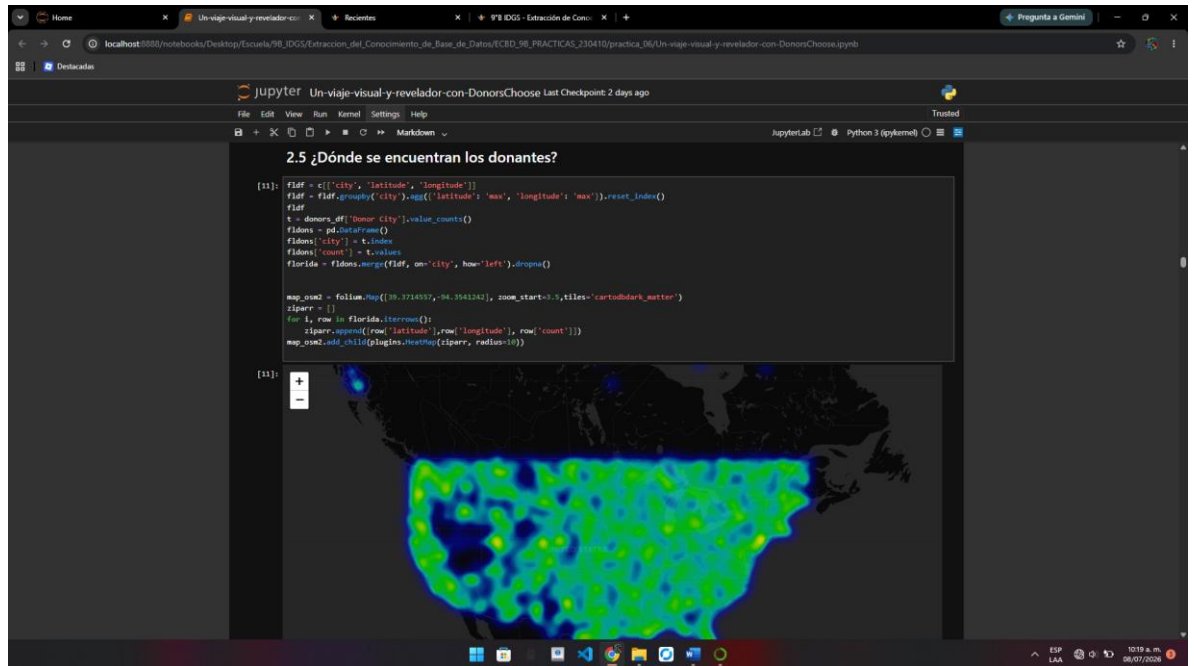
9. Dónde se encuentran los donantes en Texas

Elaboración de un mapa de calor con Folium (`HeatMap`) para visualizar la concentración de donantes en Texas.



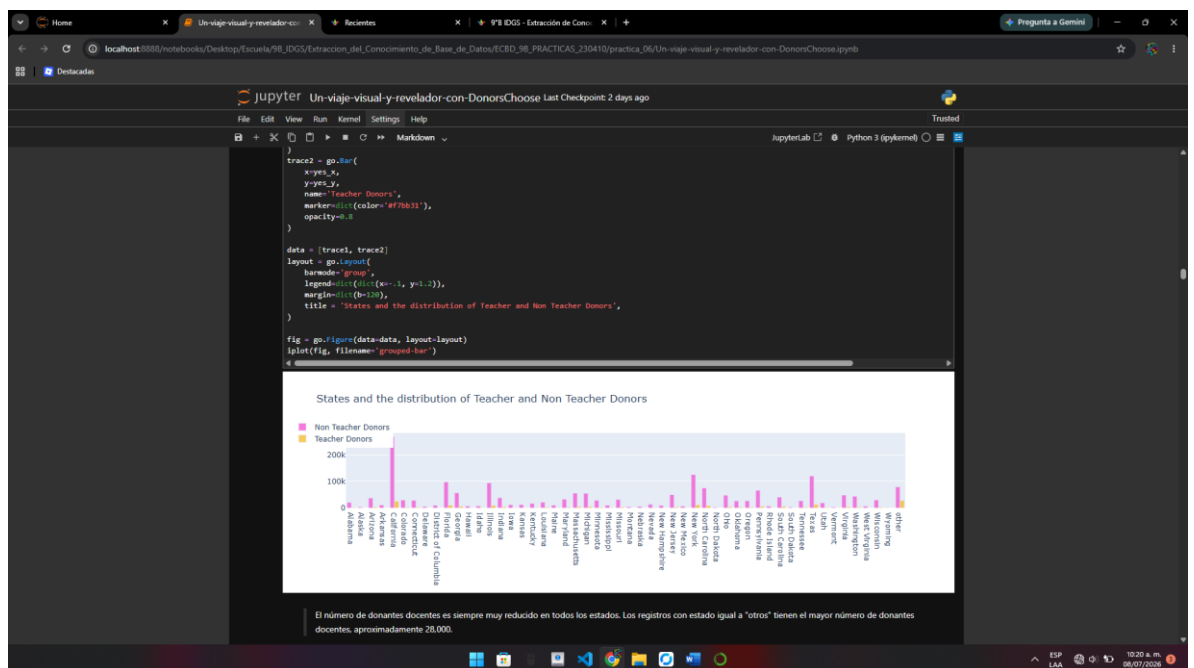
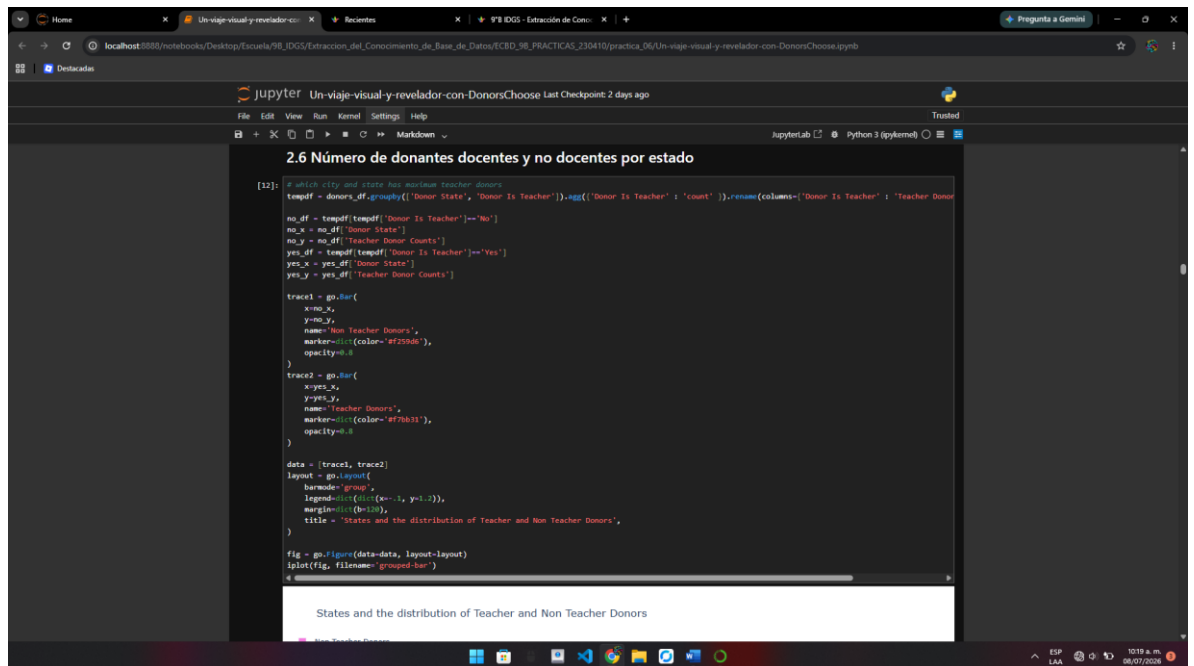
10. Dónde se encuentran los donantes

Construcción de un mapa de calor nacional de los donantes utilizando Folium.



11. Donantes docentes y no docentes en diferentes estados

Comparación entre docentes y no docentes mediante barras agrupadas con pandas y Plotly.



12. Instantánea de las donaciones

Obtención de estadísticas descriptivas utilizando funciones como `mean()`, `describe()` y filtros en pandas.

Un-viaje-visual-y-revelador-con-DonorsChoose Last checkpoint 2 days ago

3. Exploración de las donaciones

El archivo de donaciones incluye información sobre las donaciones realizadas por los donantes, el monto total de la donación, la secuencia del carrito del donante, la fecha, y el ID del proyecto y del donante.

3.1 Instantáneas de las donaciones

```
[13]: donations_df[['Donation ID', 'Donation Included Optional Donation', 'Donation Amount',
'Donor Cart Sequence', 'Donation Received Date', 'Project ID', 'Donor ID']].head(1)
# donations_df.columns

# len(donations_df)
# len(donations_df[donations_df['Donation Amount'] > 20000]) / len(donations_df)
# len(donations_df[donations_df['Donation Amount'] > 20000]) / len(donations_df)
# len(donations_df[donations_df['Donation Amount'] < 1])
# np.mean(donations_df['Donation Amount'])
```

	Donation ID	Donation Included Optional Donation	Donation Amount	Donor Cart Sequence	Donation Received Date	Project ID	Donor ID
0	608729120858666221208529e3615de	No	178.37	11	2016-09-23 13:15:57	000009891526c0ade71809423792063	1f4b3b6e68445dc0a050983ac93f3b
1	dcf1071d3aa356191ac09d1ff73dee	Yes	25.00	2	2016-06-06 20:05:23	000009891526c0ade71809423792063	4aaab6d246b359968239e05591a8a
2	1ba23489d1e538431761d521ea7799d	Yes	20.00	3	2016-06-06 14:08:46	000009891526c0ade71809423792063	6b075dc9c759ad348a7688ba25e4e

Hasta abril de 2018, se han realizado un total de **4.637.884** donaciones en DonorsChoose. El monto donado más alto asciende a **USD 60.000**, donados en 2010. Solo **6** donaciones superan los **USD 20.000**, mientras que únicamente **61** pagos son mayores a **USD 10.000**. La mayoría de los pagos (aproximadamente el 83% de todas las donaciones) son inferiores a **USD 100**. Hay 350 pagos de menos de USD 1 – una cantidad pequeña, pero que igual puede marcar la diferencia. Hasta abril de 2018, DonorsChoose ha recaudado una cantidad total enorme de **USD 284.408.243** a través de su plataforma. El monto promedio donado en DonorsChoose es de **USD 60**. Veamos los detalles de los donantes y los proyectos relacionados con estas donaciones en las siguientes secciones.

3.2 Proyectos y sus donaciones

Identifiquemos los mejores proyectos y los valores de sus atributos relacionados:

13. Proyectos y sus donaciones

Agrupación de la información por proyecto utilizando `groupby()` para analizar el comportamiento de las donaciones.

Un-viaje-visual-y-revelador-con-DonorsChoose Last checkpoint 2 days ago

3.2 Proyectos y sus donaciones

Identifiquemos los mejores proyectos y los valores de sus atributos relacionados:

- 3.2.1 Proyectos con el mayor número de donantes
- 3.2.2 Proyectos con los montos totales financiados más altos
- 3.2.3 Proyectos con los montos promedio más altos por donante
- 3.2.4 Proyectos con el monto donado único más alto

```
[14]: # Which Projects Had the Maximum Donations
tempdf = donations_df.groupby('Project ID').agg(['Project ID': 'count', 'Donation Amount': 'sum']).rename(columns={'Project ID': 'Total Donations',
'Average Donation Amount' : tempdf['Total Amount'] / tempdf['Total Donations']})

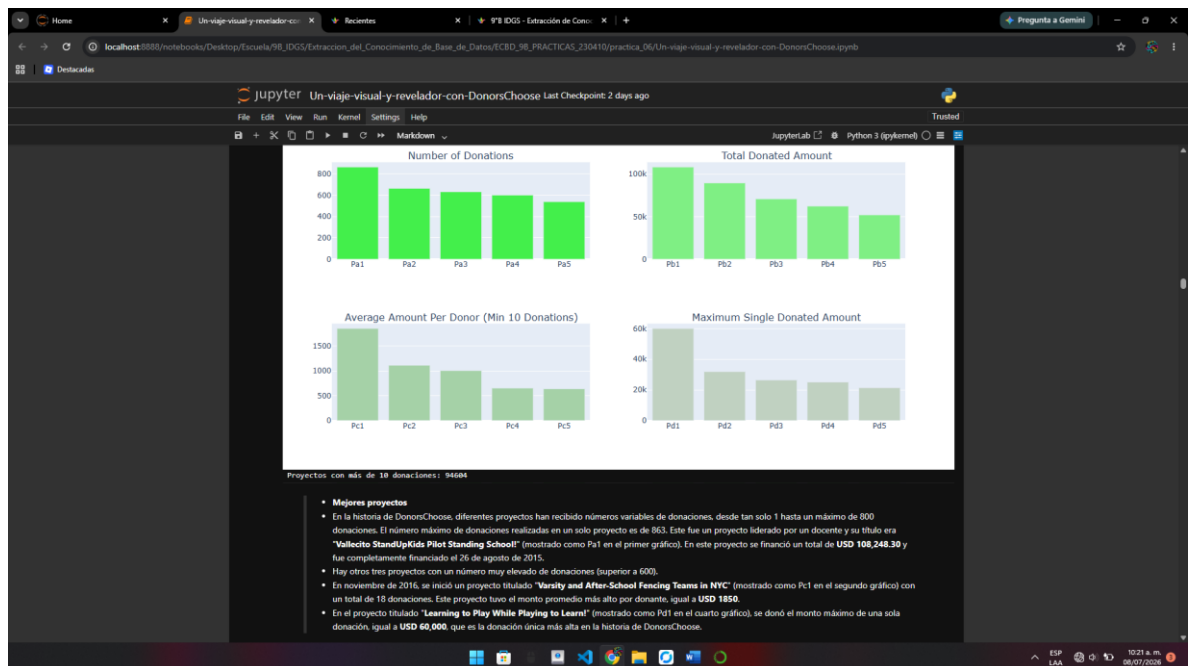
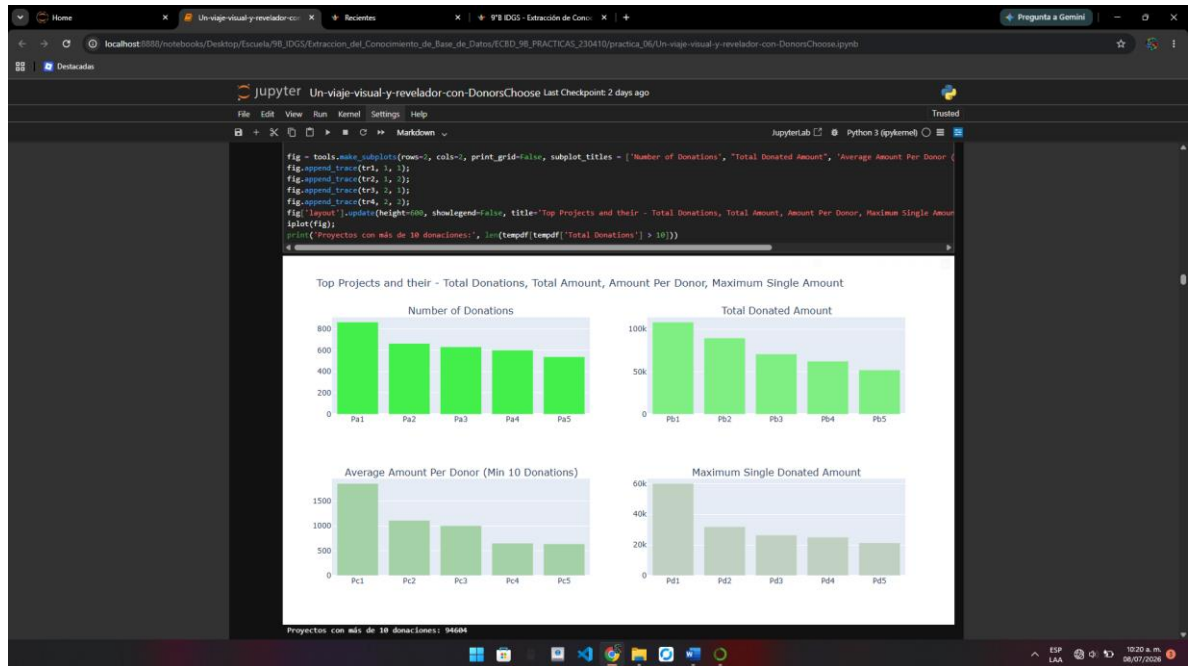
tempdf1 = tempdf.sort_values(by=['Total Donations'], ascending=False)
x = tempdf1.head(3)['Project ID']
x = [f'{x+1:1s}' for i in range(len(x))]
y = tempdf1.head(3)['Total Donations']
tr1 = bar_ver_mongo(x, y, 'Total Donations', '#3ef4b', lw=2, rt=True)

tempdf1 = tempdf.sort_values(by=['Total Amount'], ascending=False)
x = tempdf1.head(3)['Project ID']
x = [f'{x+1:1s}' for i in range(len(x))]
y = tempdf1.head(3)['Total Amount']
tr2 = bar_ver_mongo(x, y, 'Average Donations', '#7fe7b4', lw=2, rt=True)

tempdf_filtered = tempdf[tempdf['Total Donations'] > 10].sort_values(by=['Average Donation Amount'], ascending=False)
x = tempdf_filtered.head(3)['Project ID']
x = [f'{x+1:1s}' for i in range(len(x))]
y = tempdf_filtered.head(3)['Average Donation Amount']
tr3 = bar_ver_mongo(x, y, 'Average Donations (Min: 10 Donations)', '#d62728', lw=2, rt=True)

tempdf = donations_df.groupby('Project ID').agg(['Project ID': 'count', 'Donation Amount': 'max']).rename(columns={'Project ID': 'Total Donations',
'Maximum Donations' : tempdf['Maximum Donations'], ascending=False})
x = tempdf1.head(3)['Project ID']
x = [f'{x+1:1s}' for i in range(len(x))]
y = tempdf1.head(3)['Maximum Donations']
tr4 = bar_ver_mongo(x, y, 'Maximum Donations', '#d62728', lw=2, rt=True)

fig = tools.make_subplots(rows=2, cols=2, print_grid=False, subplot_titles = ['Number of Donations', 'Total Donated Amount', 'Average Amount Per Donor ('])
fig.append_trace(tr1, 1, 1);
fig.append_trace(tr2, 1, 2);
fig.append_trace(tr3, 2, 1);
fig.append_trace(tr4, 2, 2);
```



14. Proyectos con mayor número de donantes

Identificación de los cinco proyectos con más donantes mediante `sort_values()` y gráficos de barras en Plotly.

15. Proyectos con los montos totales financiados más altos

Cálculo y visualización de los proyectos con mayor monto total financiado utilizando `pandas` y Plotly.

16. Proyectos con los montos promedio más altos por donante

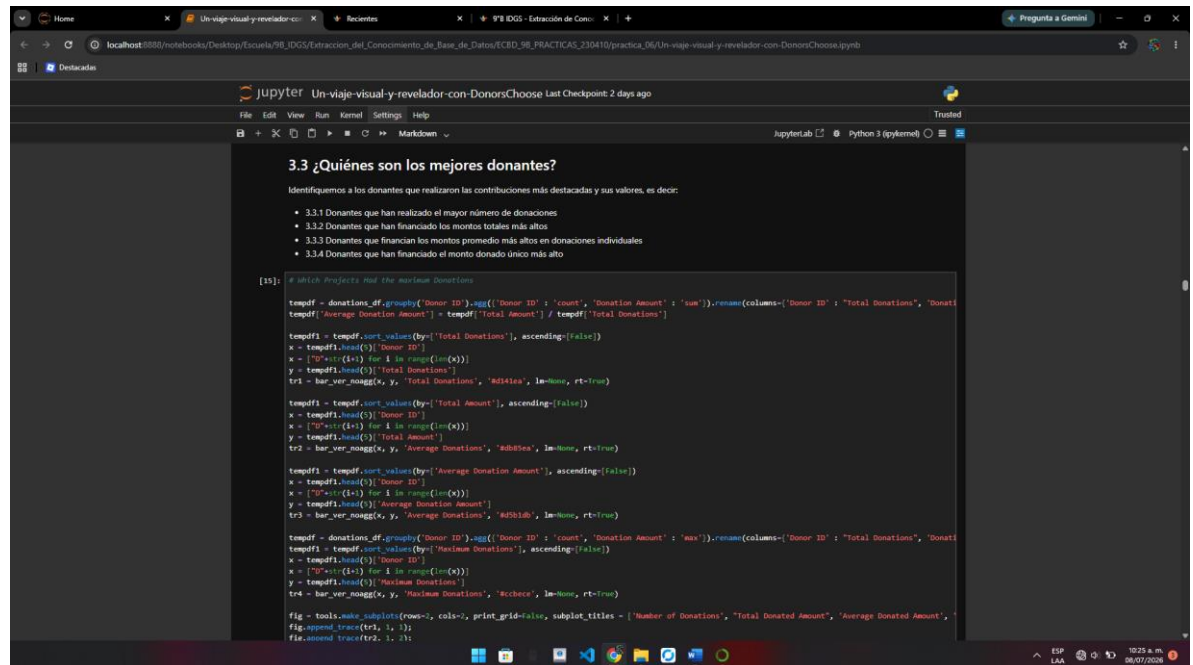
Cálculo del promedio donado por proyecto y representación mediante gráficos en Plotly.

17. Proyectos con el monto donado único más alto

Identificación de la donación individual de mayor valor utilizando `sort_values()` y Plotly.

18. Donantes y sus donaciones

Agrupación de la información por donante mediante `groupby()` para analizar sus aportaciones.



```
[15]: # Which Projects Had the Highest Donations

tempdf = donations_df.groupby('Donor ID').agg({'Donor ID': 'count', 'Donation Amount': 'sum'}).rename(columns={'Donor ID': 'Total Donations', 'Donation Amount': 'Average Donation Amount'}) + tempdf[['Total Amount']] / tempdf[['Total Donations']]

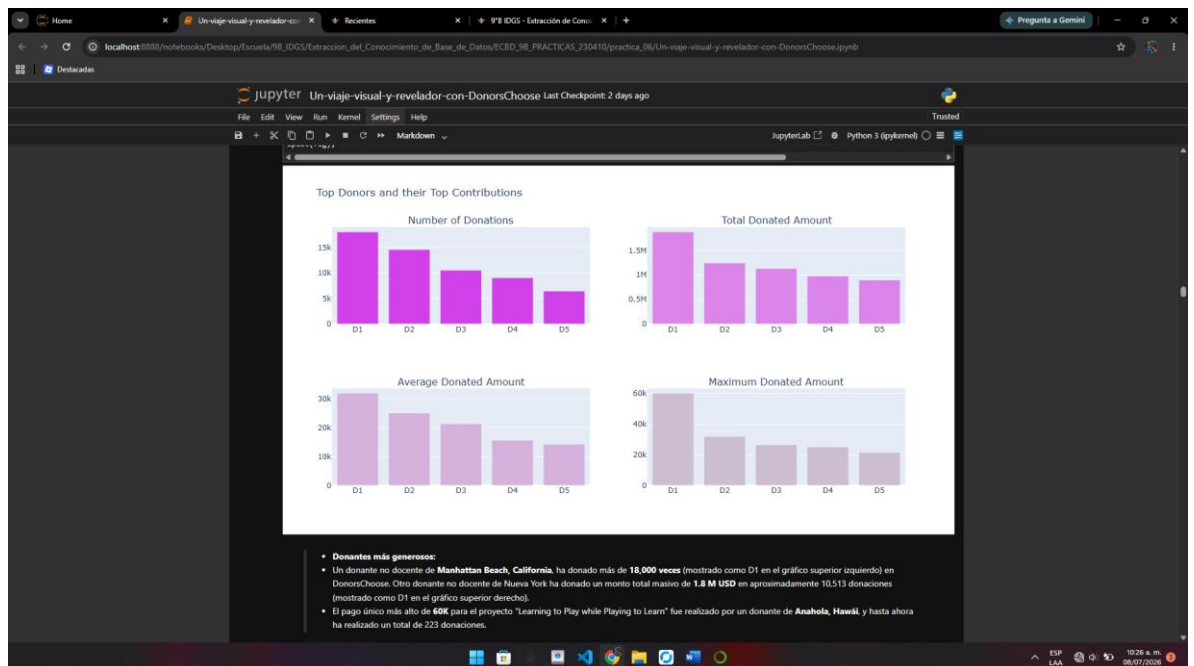
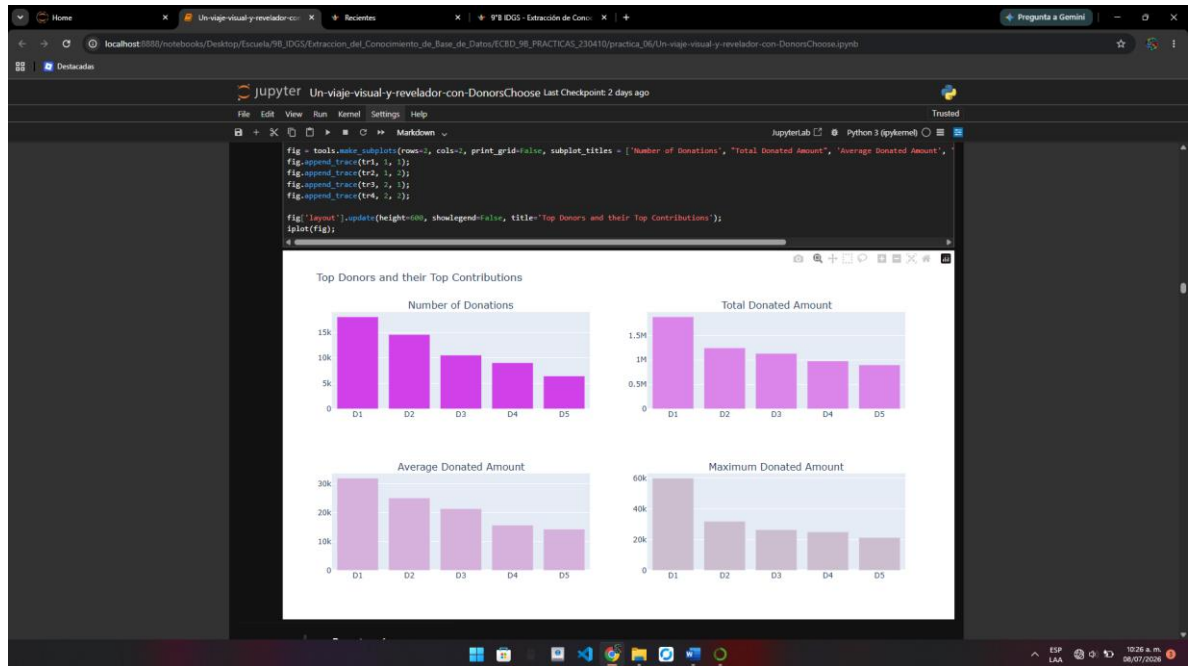
tempdf1 = tempdf.sort_values(by='Total Donations', ascending=False)
x = tempdf1.head(3)
x = [0] * (len(x) - 1)
y = tempdf1.head(3)
tr1 = bar_ver_magg(x, y, 'Total Donations', 'M1414a', lw=2, rt=True)

tempdf1 = tempdf.sort_values(by='Total Amount', ascending=False)
x = tempdf1.head(3)
x = [0] * (len(x) - 1)
y = tempdf1.head(3)
tr2 = bar_ver_magg(x, y, 'Average Donations', 'M1414a', lw=2, rt=True)

tempdf1 = tempdf.sort_values(by='Average Donation Amount', ascending=False)
x = tempdf1.head(3)
x = [0] * (len(x) - 1)
y = tempdf1.head(3)
tr3 = bar_ver_magg(x, y, 'Average Donations', 'M1414b', lw=2, rt=True)

tempdf = donations_df.groupby('Donor ID').agg({'Donor ID': 'count', 'Donation Amount': 'max'}).rename(columns={'Donor ID': 'Total Donations', 'Donation Amount': 'Maximum Donations'})
tempdf1 = tempdf.sort_values(by='Maximum Donations', ascending=False)
x = tempdf1.head(3)
x = [0] * (len(x) - 1)
y = tempdf1.head(3)
tr4 = bar_ver_magg(x, y, 'Maximum Donations', 'M1414c', lw=2, rt=True)

fig = tools.magg_subplot(rows=2, cols=2, print_grid=False, subplot_titles = ['Number of Donations', 'Total Donated Amount', 'Average Donated Amount', 'Maximum Donated Amount'])
fig.append_trace(tr1, 1, 1)
fig.append_trace(tr2, 1, 2)
fig.append_trace(tr3, 2, 1)
fig.append_trace(tr4, 2, 2)
```



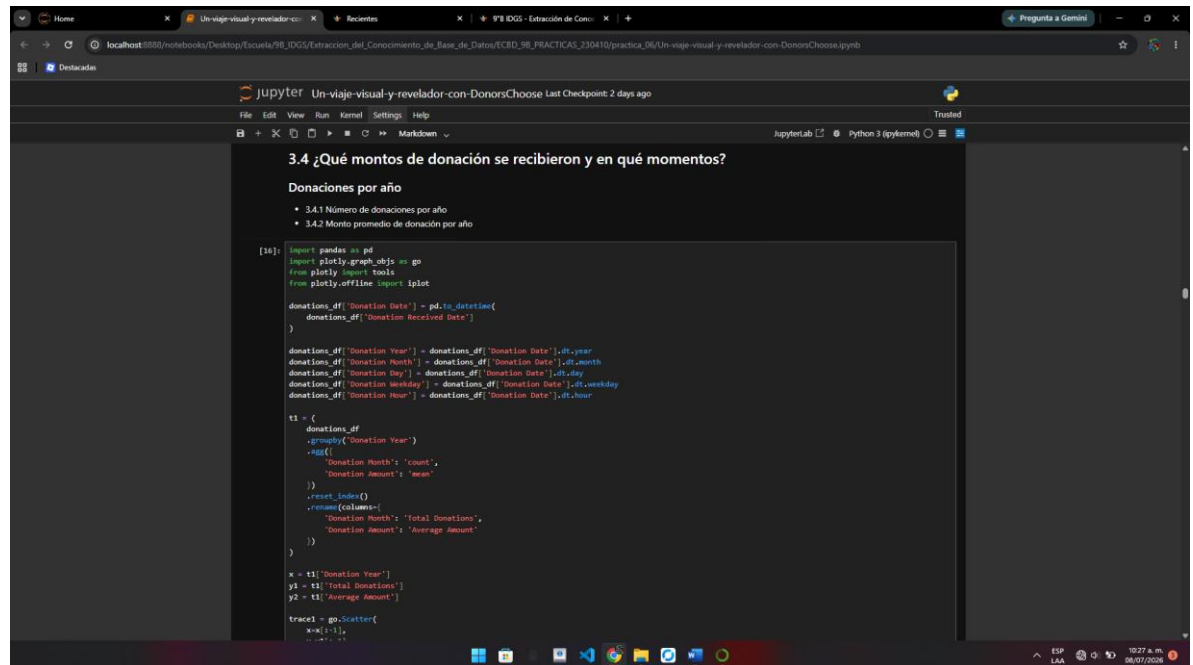
19. Donantes que han realizado el mayor número de donaciones

Identificación de los donantes con mayor número de aportaciones mediante `pandas` y `Plotly`.

20. Donantes que han financiado los montos totales más altos

Cálculo de los donantes con mayor monto acumulado de donaciones utilizando `pandas` y `Plotly`.

21. **Donantes que financian los montos promedio más altos en donaciones individuales**
Cálculo del promedio donado por cada donante y representación gráfica en Plotly.
22. **Donantes que han financiado el monto donado único más alto**
Identificación de la donación individual más alta realizada por un donante utilizando `pandas` y Plotly.
23. **Montos de donación por año, mes, día, hora**
Extracción de componentes de fecha con `.dt` y análisis temporal mediante series en Plotly.



```
3.4 ¿Qué montos de donación se recibieron y en qué momentos?

Donaciones por año
• 3.4.1 Número de donaciones por año
• 3.4.2 Monto promedio de donación por año

[16]: import pandas as pd
import plotly.graph_objs as go
from plotly import tools
from plotly.offline import plot

donations_df['Donation Date'] = pd.to_datetime(
    donations_df['Donation Received Date']
)

donations_df['Donation Year'] = donations_df['Donation Date'].dt.year
donations_df['Donation Month'] = donations_df['Donation Date'].dt.month
donations_df['Donation Day'] = donations_df['Donation Date'].dt.day
donations_df['Donation Weekday'] = donations_df['Donation Date'].dt.weekday
donations_df['Donation Hour'] = donations_df['Donation Date'].dt.hour

t1 = {
    donations_df
    .groupby('Donation Year')
    .agg({
        'Donation Month': 'count',
        'Donation Amount': 'mean'
    })
    .reset_index()
    .rename(columns={
        'Donation Month': 'Total Donations',
        'Donation Amount': 'Average Amount'
    })
}

x = t1['Donation Year']
y1 = t1['Total Donations']
y2 = t1['Average Amount']

trace1 = go.Scatter(
    x=x[1:],
    y=y1[1:],
    y2=y2[1:]
)
```



```
trac1 = go.scatter(
    x=x[i-1],
    y=y1[i-1],
    fill='tosera',
    fillcolor='efcd0d',
    mode='none'
)

trac2 = go.scatter(
    x=x[i-1],
    y=y2[i-1],
    fill='tosera',
    fillcolor='e099ff',
    mode='none'
)

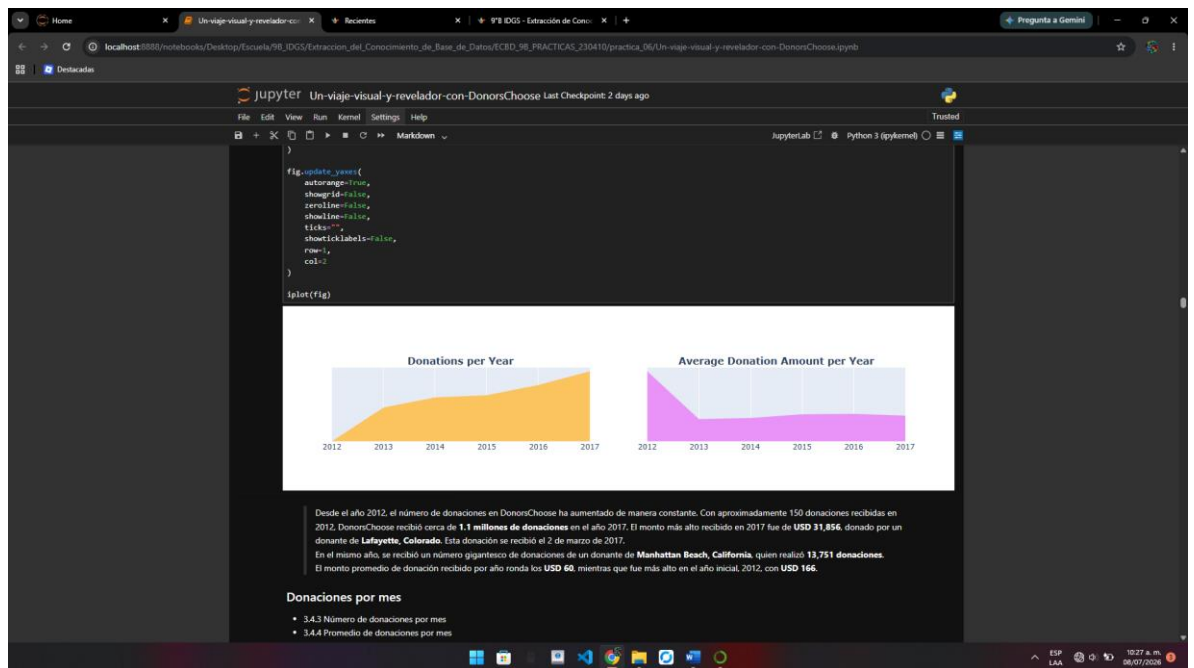
fig = tools.make_subplots(
    rows=1,
    cols=2,
    subplot_titles=(
        'bDonations per Year/b',
        'bAverage Donation Amount per Year/b'
    )
)

fig.append_trace(trac1, 1, 1)
fig.append_trace(trac2, 1, 2)

fig.update_layout(
    height=500,
    showlegend=False
)

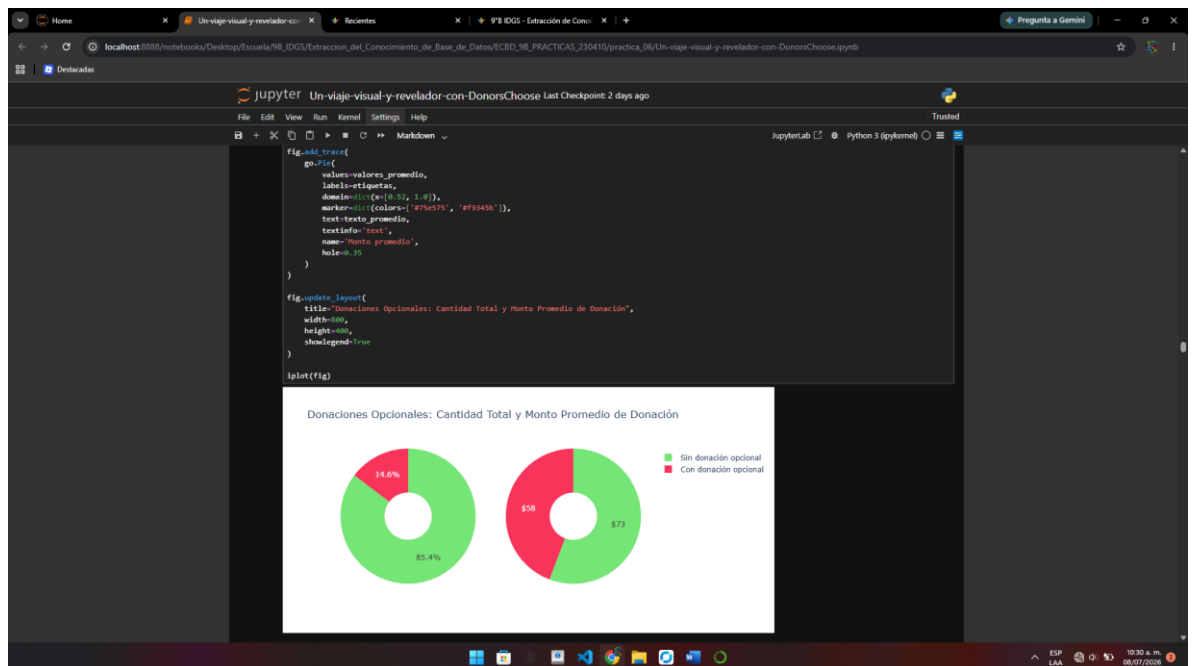
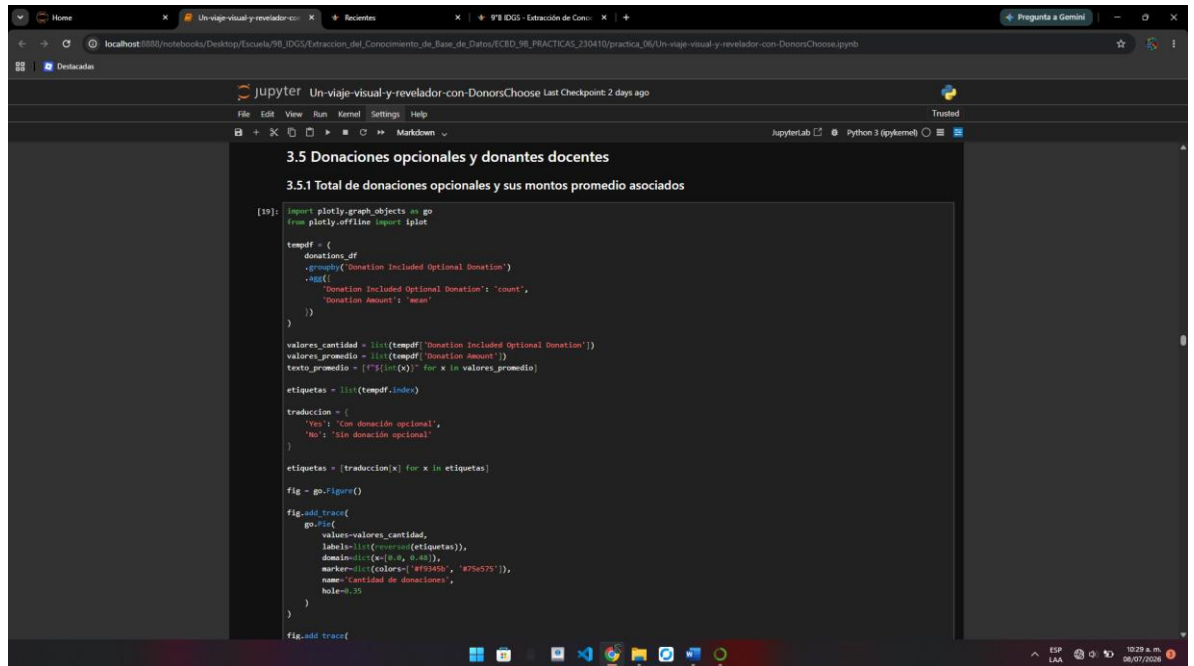
fig.update_yaxes(
    autorange=True,
    showgrid=False,
    zeroline=False,
    showline=False,
    ticks="1",
    showticklabels=False,
    row=1,
    col=1
)

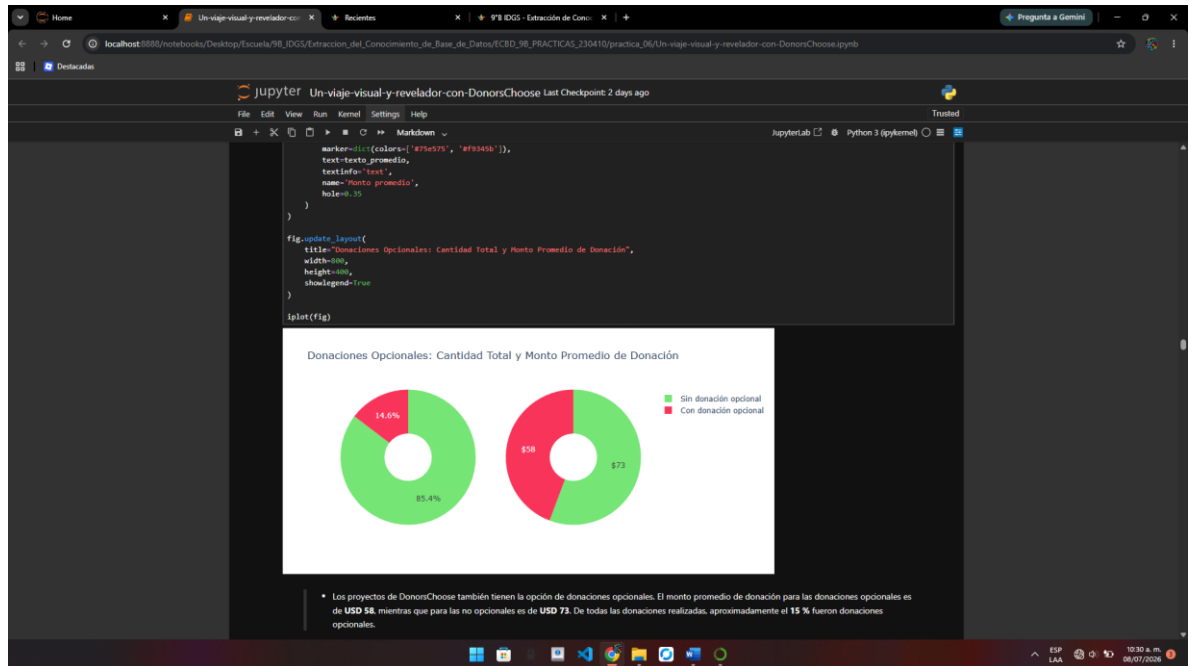
#fig.update_yaxes(
#    autorange=True,
#    showgrid=False,
#    zeroline=False,
#    showline=False,
#    ticks="1",
#    showticklabels=False,
#    row=1,
#    col=2
#)
```



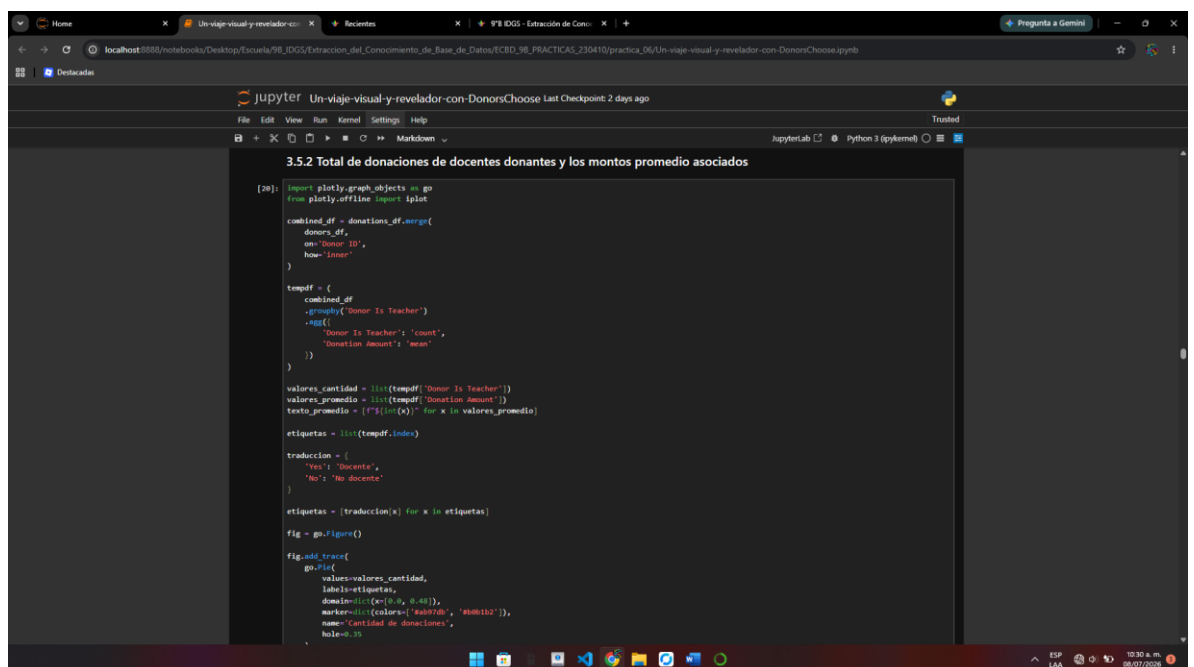
24. Donaciones opcionales y donaciones de docentes donantes

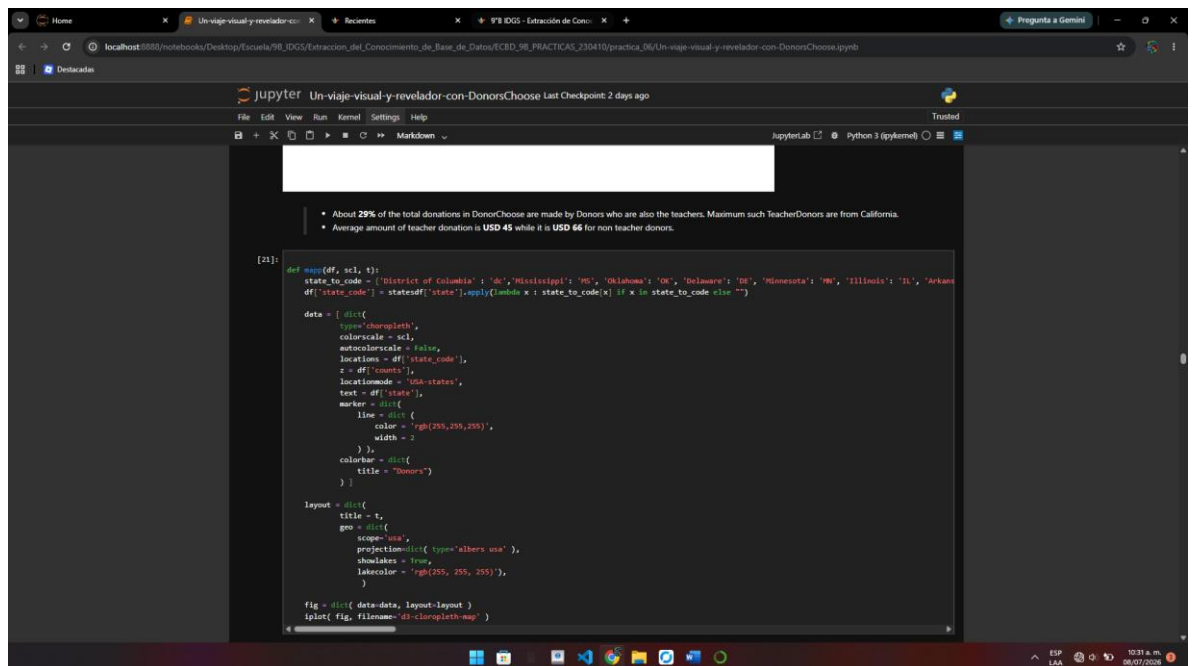
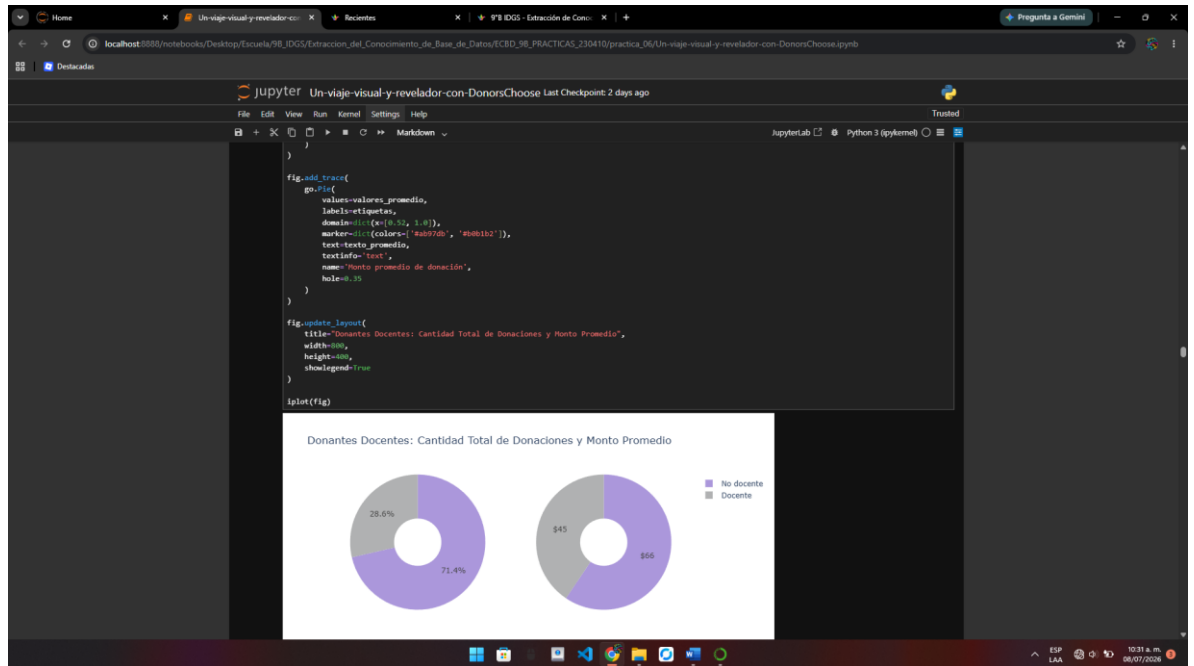
Preparación de agrupaciones y análisis comparativo mediante pandas.





25. **Número y monto promedio de donación: donaciones opcionales**
Visualización de la cantidad y el promedio de donaciones opcionales mediante gráficos de pastel en Plotly.



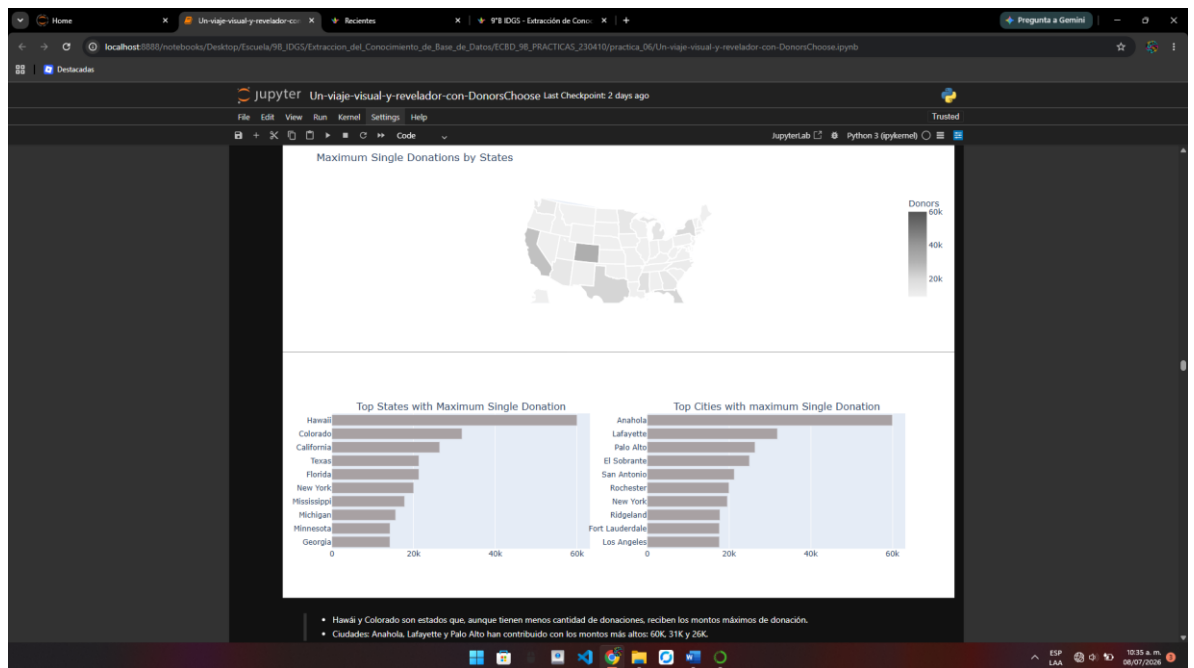
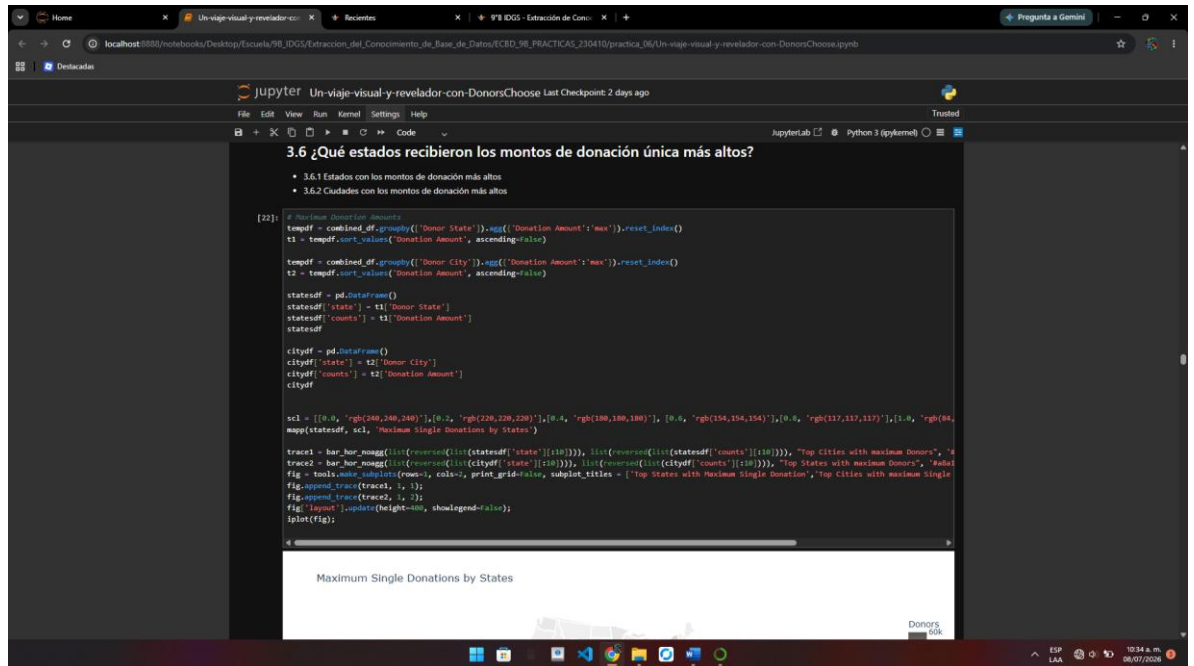


26. Número y monto promedio de donación: docentes donantes

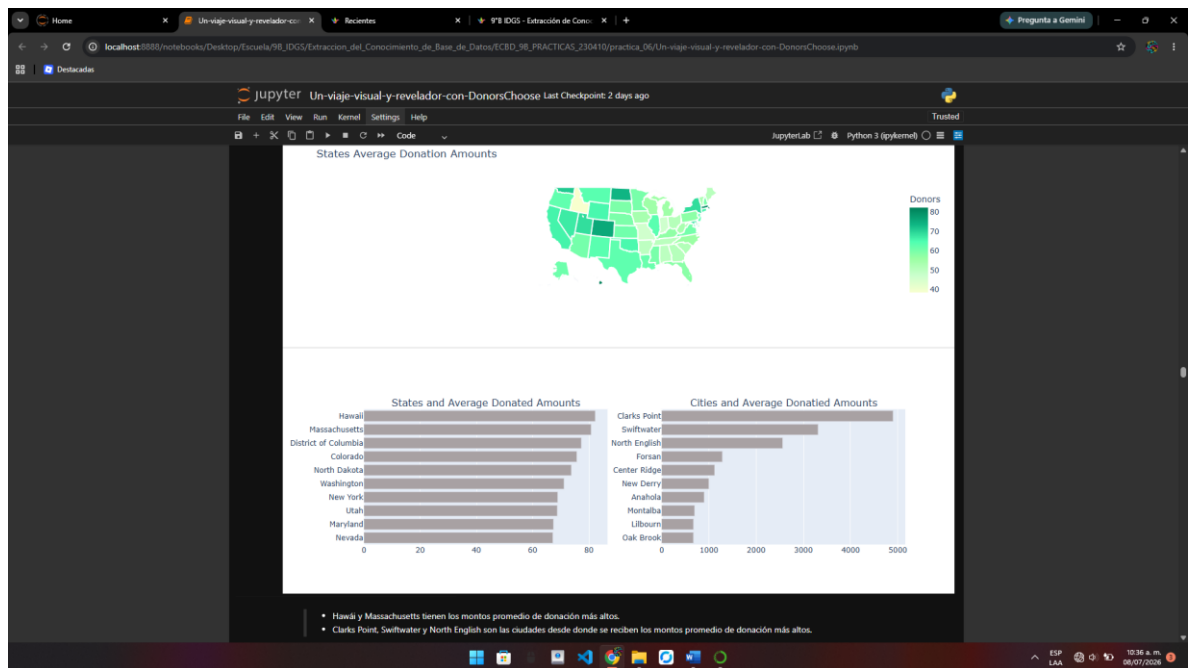
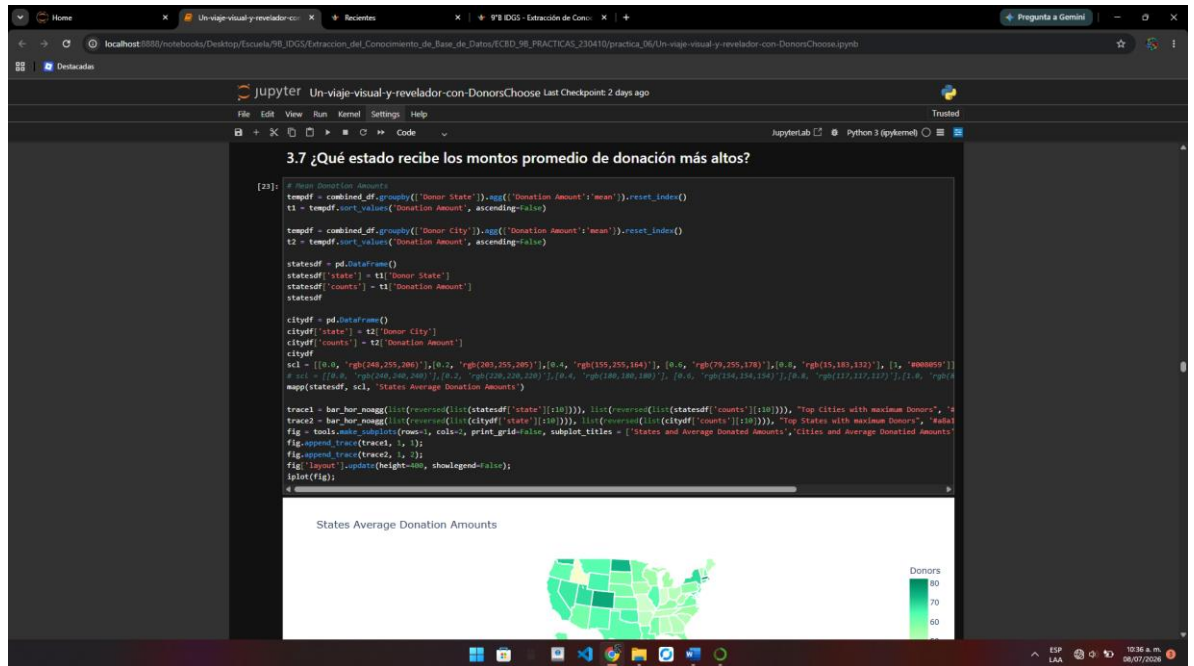
Integración de datos mediante `merge()` y representación con gráficos de pastel en Plotly.

27. ¿Qué estado ha recibido el monto de donación más alto?

Elaboración de un mapa coroplético para identificar el estado con mayor monto recibido.

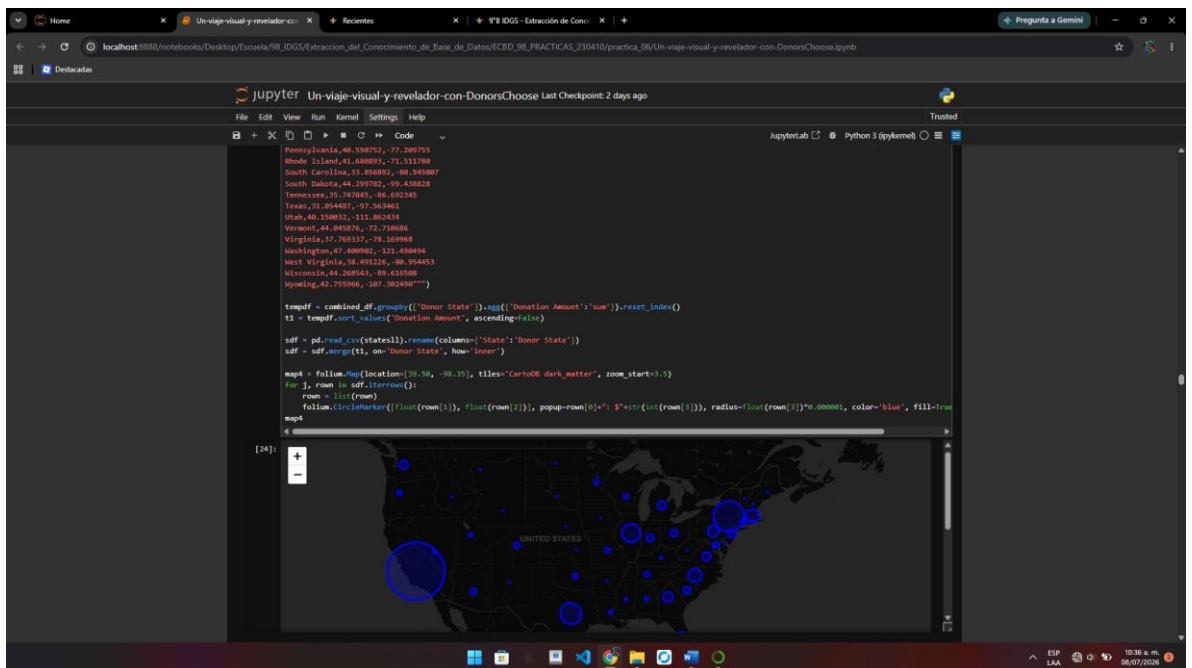
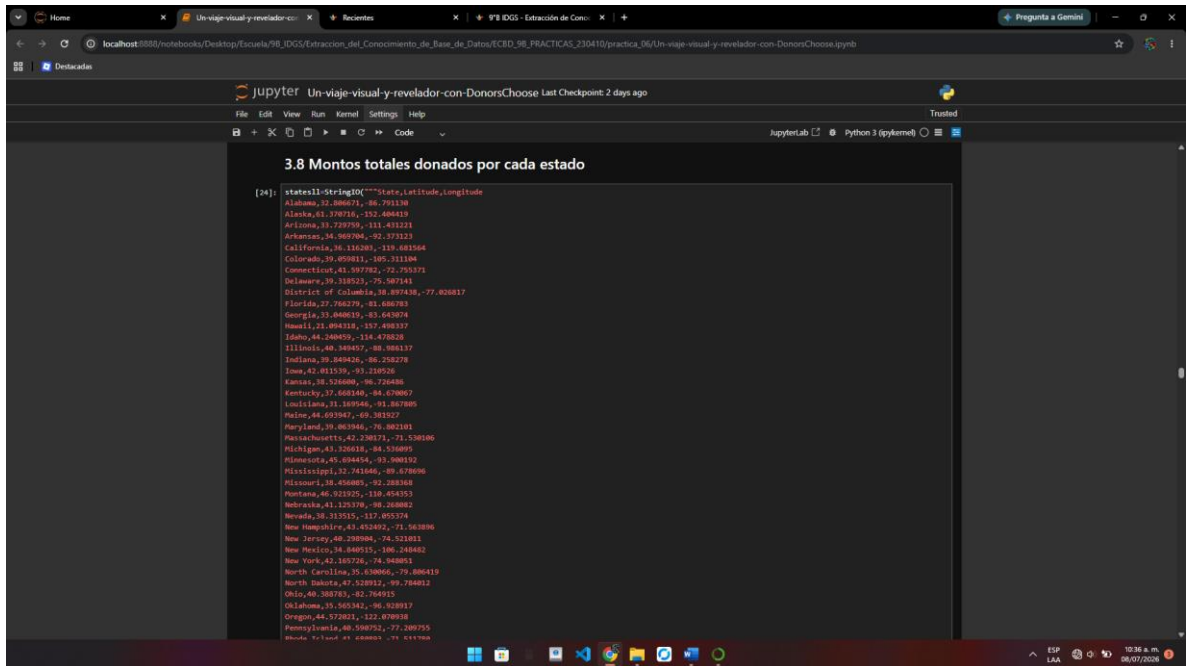


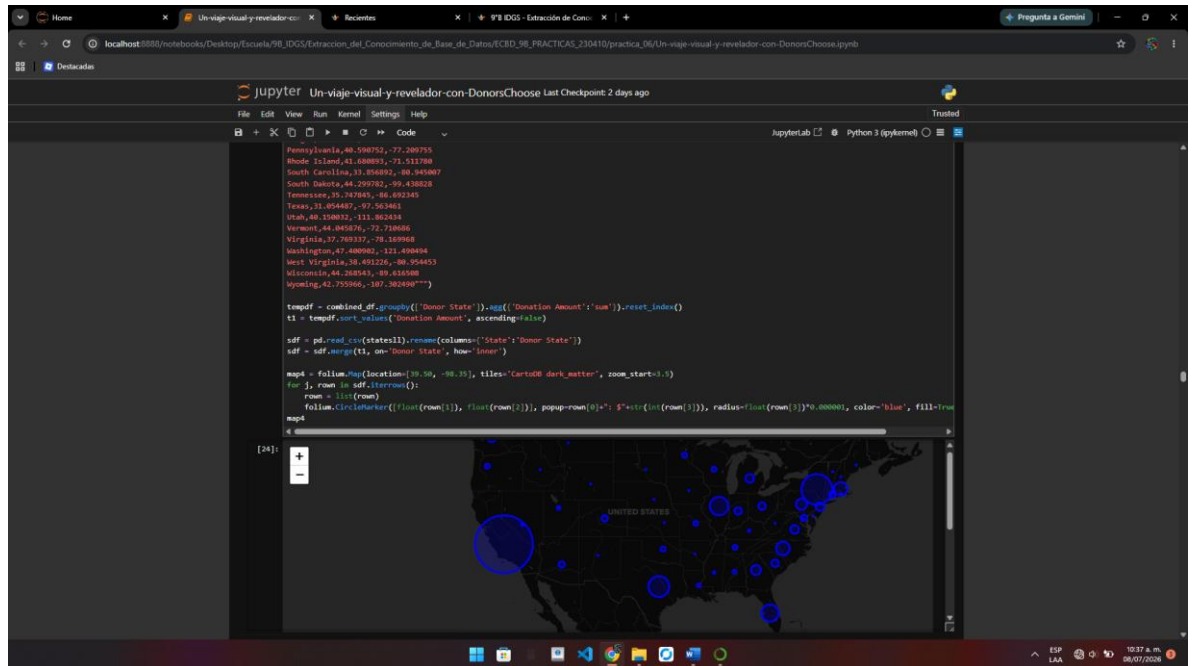
28. **¿Qué estado recibe los montos de donación promedio más altos?**
 Análisis del promedio de donaciones por estado mediante mapas coropléticos en Plotly.



29. Estados y el total de donaciones recibidas

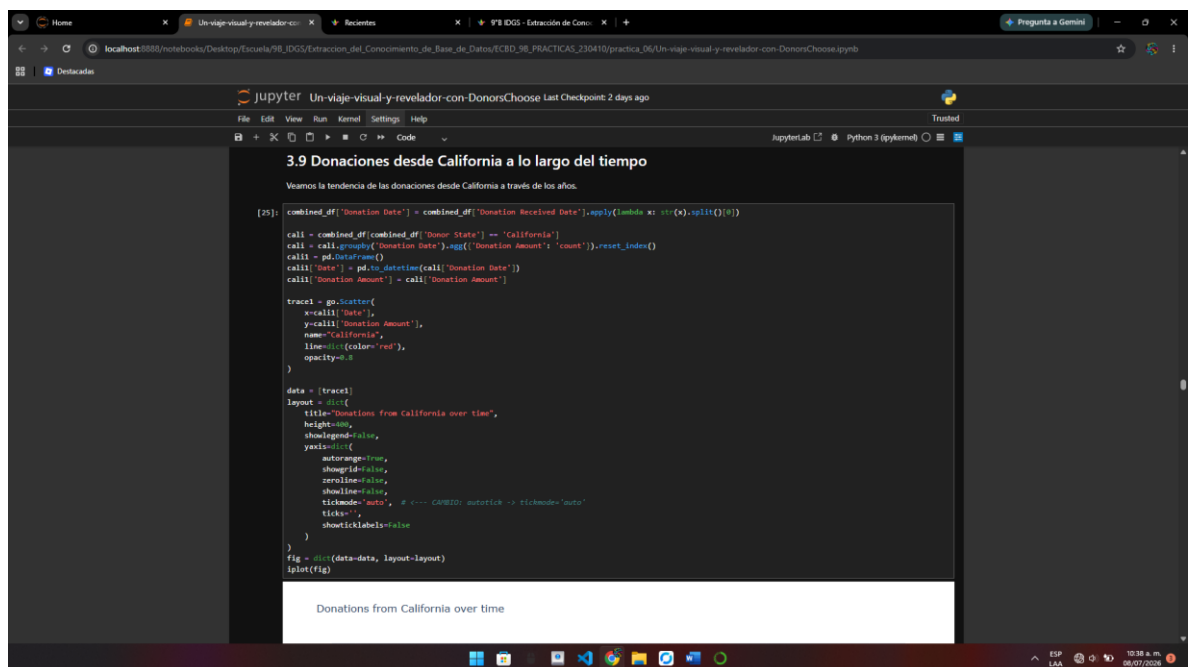
Representación del total de donaciones recibidas por estado mediante mapas coropléticos.

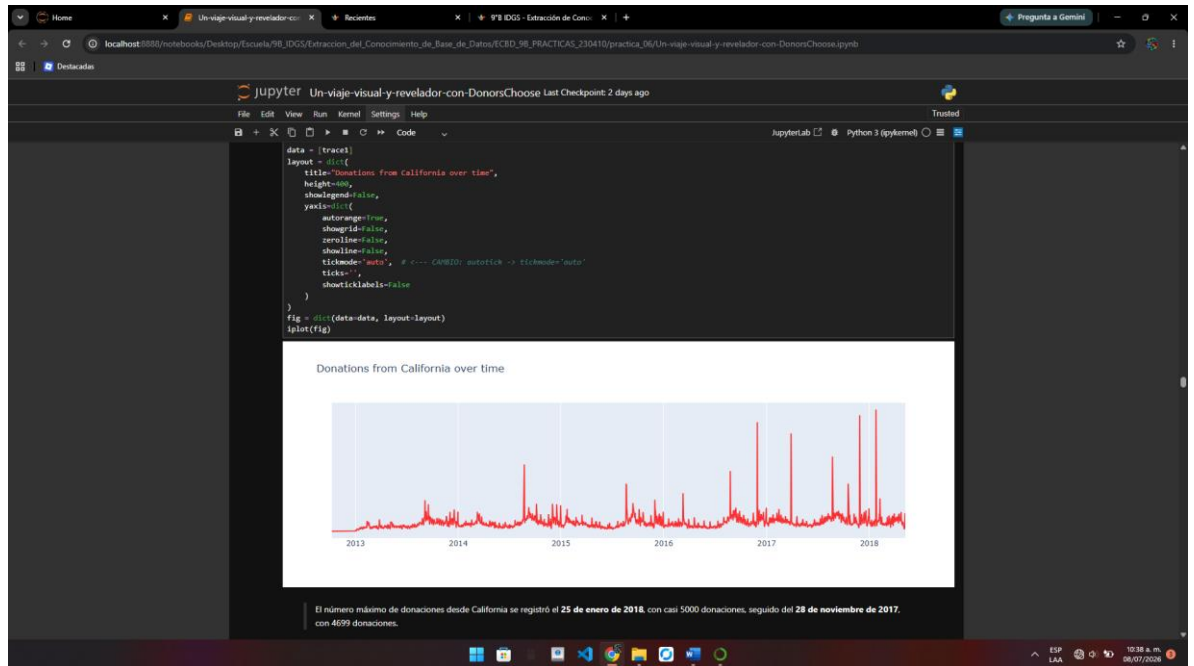




30. Donaciones desde California a lo largo del tiempo

Construcción de una serie temporal con `go.Scatter` para analizar la evolución de las donaciones provenientes de California.





31. Instantánea de los datos de docentes

Vista general del conjunto de datos de docentes mediante `head()`.

4. Exploración de los docentes

4.1 Instantánea de los datos de docentes

```
[28]: teachers_df.head()
```

	Teacher ID	Teacher Prefix	Teacher First Project Posted Date
0	000007264c27baf8a083feda0baa	Mrs.	2013-08-21
1	00002d44003ed4b066607c545a99ba	Mrs.	2016-10-23
2	00000064c3d92d904a22e6a705c119a	Mr.	2016-09-08

Hay un total de 402,900 docentes que han publicado al menos un proyecto en DonorsChoose desde 2002. La gran mayoría de los docentes son mujeres. Veamos quiénes fueron los primeros docentes en publicar proyectos en DonorsChoose.

4.2 ¿Quiénes fueron los primeros docentes en DonorsChoose.org?

```
[29]: teachers_df['Posted Date'] = pd.to_datetime(teachers_df['Teacher First Project Posted Date'])
teachers_df['Posted Year'] = teachers_df['Posted Date'].dt.year
```

Let's find out who were the first donors on DonorsChoose.com of the Earliest Teachers

```
teachers_df[['Teacher ID', 'Posted Date']].sort_values(['Posted Date'])[0:10]
```

	Teacher ID	Posted Date
280335	b218f64064e1746373363229c231962	2002-09-17
245493	9c0a56b63b743454d5d6d6f6f1226	2002-09-17
77122	31367152c3a08d8a97848b6b07	2002-09-19
357964	e3751e935d13deb637864194955d1548	2002-09-19
245631	9c214c58003f1d4d82261404b6b6c5	2002-09-19
41877	1aacfb01eb99211423690c3f844027b	2002-09-19
271224	ac3d575c2d7a58063677162af9907fd	2002-09-19
295166	bb9431f6e6f76852f4030238a73ac41	2002-09-20

32. ¿Quiénes fueron los primeros docentes en DonorsChoose?

Ordenamiento cronológico utilizando `sort_values()` para identificar los primeros docentes registrados.

```
[29]: teachers_df['Posted Date'] = pd.to_datetime(teachers_df['Teacher First Project Posted Date'])
teachers_df['Posted Year'] = teachers_df['Posted Date'].dt.year

# Lets find out who were the first donors on DonorsChoose.org of the earliest Teachers
teachers_df[['Teacher ID', 'Posted Date']].sort_values(['Posted Date'])[0:10]

[29]:
```

	Teacher ID	Posted Date
280335	b21894064e174657336d27962231962	2002-09-17
245493	9c0a56b63b7434548dc9efcd122fc	2002-09-17
77122	31367152c3a08a0f9a97840b6b807	2002-09-19
357964	d3751e935d13deeb37864194955d1548	2002-09-19
245631	9c714c500371d4d82761404b0b6c5	2002-09-19
41877	1aac9791eb3921142368dc38f44027b	2002-09-19
271224	ac3d575cd27a58063677162af4d907d	2002-09-19
285166	bb9451f6ef76852f403023ba73ac41	2002-09-20
272164	acd4f1b0b4b696c27ad18236d9d377d	2002-10-11
282250	b1b643f2558b0c27ce9a93a03c3f33	2002-11-05

Los primeros docentes en DonorsChoose se registraron el 17 de septiembre de 2002. Ese día se registraron 2 docentes, ambas mujeres. En el mes de septiembre de 2002, se registraron un total de 8 docentes, que fueron principalmente el primer grupo de docentes en DonorsChoose.

```
[30]: temp = teachers_df.groupby('Posted Year').agg({'Teacher ID': 'count'}).reset_index()
temp = temp.sort_values('Posted Year')

trace = go.Scatter(
    x=temp['Posted Year'],
    y=temp['Teacher ID'],
    mode='lines+markers',
    line=dict(color='red', width=2),
    marker=dict(size=10),
    connectgaps=True,
    fill='tozeroy', # CORRECCIÓN: fill debe estar aquí
    fillcolor='rgb(0,100,0,0.2)',
)

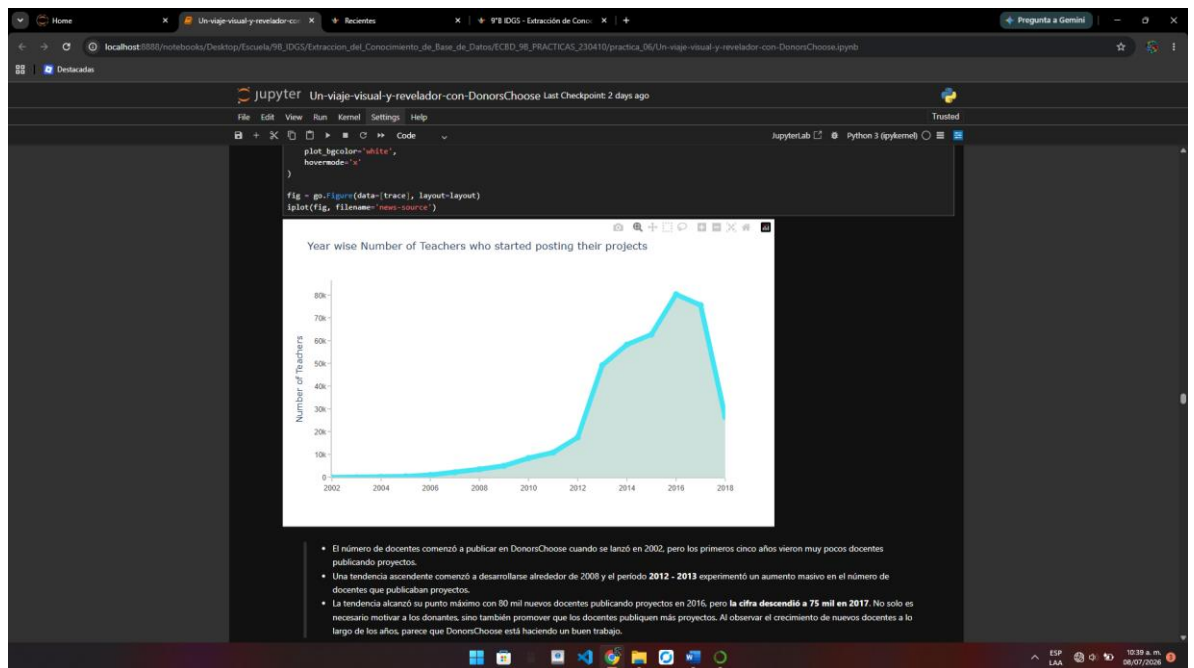
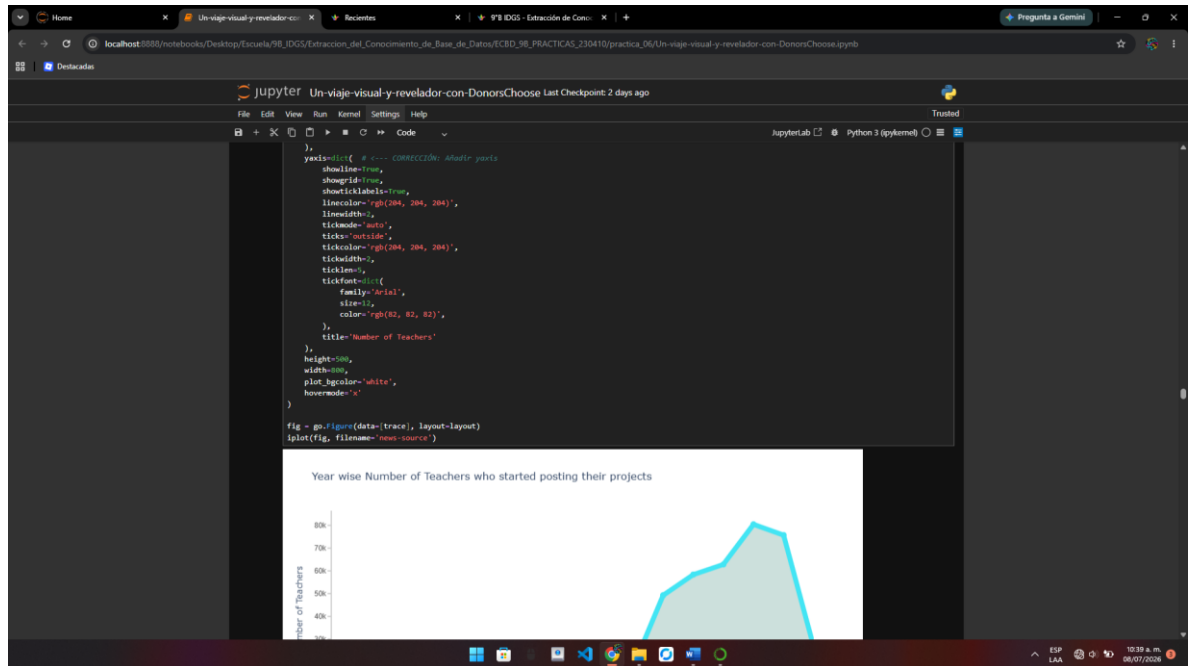
layout = go.Layout(
    title='Year size Number of Teachers who started posting their projects',
    xaxis=dict(
        showline=True,
        showgrid=True,
        showticklabels=True,
        linecolor='rgb(204, 204, 204)',
        linewidth=2,
        tickmode='auto', # CORRECCIÓN: autotick -> tickmode
        ticks='outside',
        tickcolor='rgb(204, 204, 204)',
        tickdash=8,
        ticklen=5,
        tickfont=dict(
            family='Arial',
            size=12,
            color='rgb(82, 82, 82)',
        ),
    ),
    yaxis=dict( # CORRECCIÓN: Aclarar yaxis
        showline=True,
        showgrid=True,
        showticklabels=True,
        linecolor='rgb(204, 204, 204)',
        linewidth=2,
        tickmode='auto',
        ticks='outside',
    ),
)
```

33. **Crecimiento en el número de docentes a lo largo de los años**
Análisis de la evolución anual del número de docentes mediante series temporales en Plotly.

```
[30]: temp = teachers_df.groupby('Posted Year').agg({'Teacher ID': 'count'}).reset_index()
temp = temp.sort_values('Posted Year')

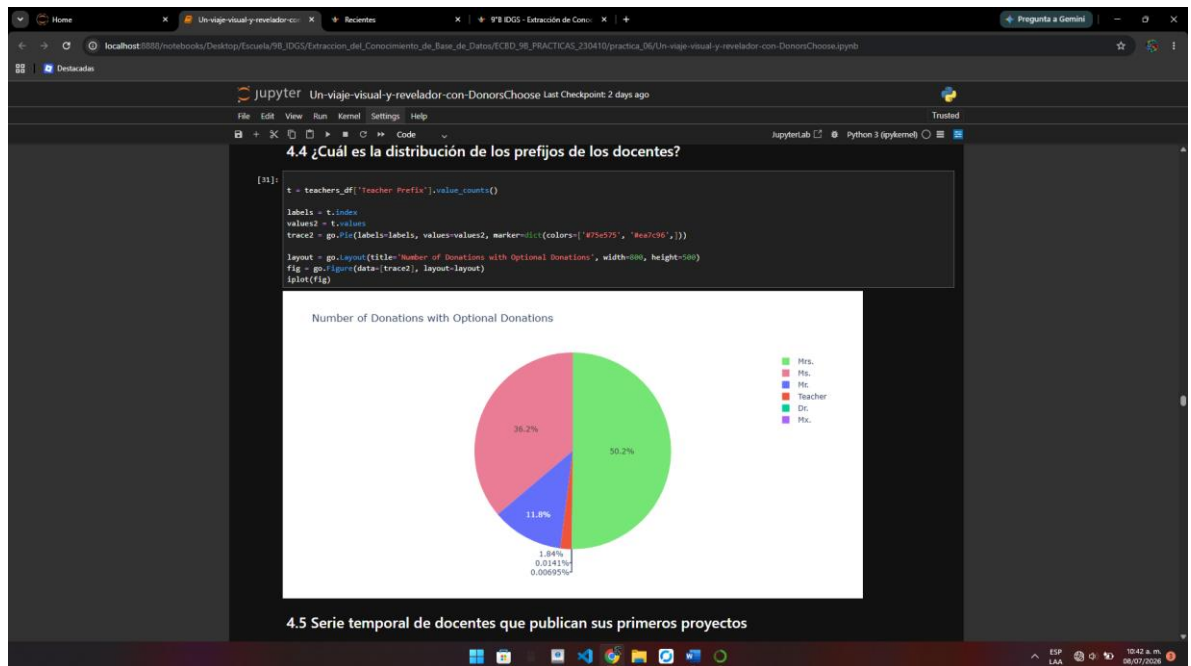
trace = go.Scatter(
    x=temp['Posted Year'],
    y=temp['Teacher ID'],
    mode='lines+markers',
    line=dict(color='red', width=2),
    marker=dict(size=10),
    connectgaps=True,
    fill='tozeroy', # CORRECCIÓN: fill debe estar aquí
    fillcolor='rgb(0,100,0,0.2)',
)

layout = go.Layout(
    title='Year size Number of Teachers who started posting their projects',
    xaxis=dict(
        showline=True,
        showgrid=True,
        showticklabels=True,
        linecolor='rgb(204, 204, 204)',
        linewidth=2,
        tickmode='auto', # CORRECCIÓN: autotick -> tickmode
        ticks='outside',
        tickcolor='rgb(204, 204, 204)',
        tickdash=8,
        ticklen=5,
        tickfont=dict(
            family='Arial',
            size=12,
            color='rgb(82, 82, 82)',
        ),
    ),
    yaxis=dict( # CORRECCIÓN: Aclarar yaxis
        showline=True,
        showgrid=True,
        showticklabels=True,
        linecolor='rgb(204, 204, 204)',
        linewidth=2,
        tickmode='auto',
        ticks='outside',
    ),
)
```

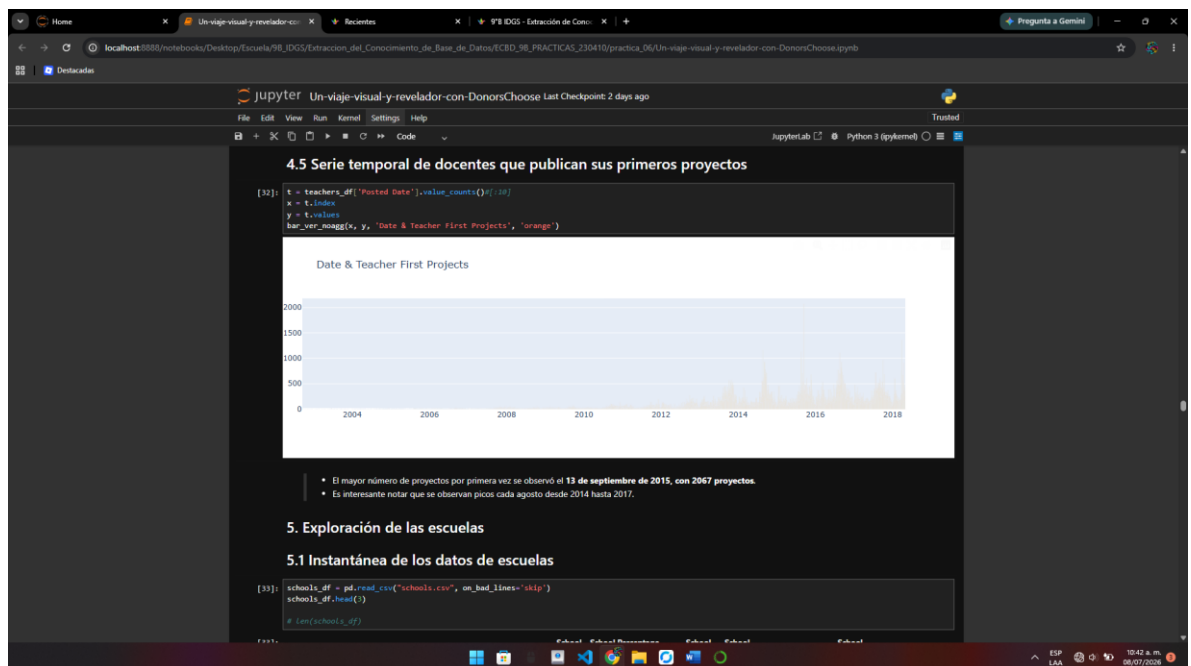


34. ¿Cuál es la distribución de los prefijos de los docentes?

Representación de la distribución de prefijos mediante gráficos de pastel en Plotly.



35. **Serie temporal de docentes publicando sus primeros proyectos**
 Conteo y visualización temporal de los primeros proyectos publicados por docentes.



36. **Instantánea de los datos de escuelas**
 Carga y exploración inicial del conjunto de datos de escuelas utilizando `read_csv()` y `head()`.

Un-viaje-visual-y-revelador-con-DonorsChoose Last checkpoint 2 days ago

5. Exploración de las escuelas

5.1 Instantánea de los datos de escuelas

```
[33]: schools_df = pd.read_csv("schools.csv", on_bad_lines="skip")
schools_df.head()
```

```
[33]:
```

	School ID	School Name	School Metro Type	School Percentage Free Lunch	School State	School Zip	School City	School County	School District
0	00003e0fd601b8eabadeb405769c5e	Capon Bridge Middle School	rural	36.0	West Virginia	26711	Capon Bridge	Hampshire	Hampshire Co School District
1	00004e32a448b4832e1b992500b0731	The Woodlands College Park High School	urban	41.0	Texas	77384	The Woodlands	Montgomery	Conroe Ind School District
2	0002021b67992bdc2d4f1acc1f880c4	Samantha Smith Elementary School	suburban	2.0	Washington	98074	Sammamish	King	Lake Washington Sch Dist 414

Se han publicado proyectos de un total de 72,993 escuelas diferentes en DonorsChoose hasta abril de 2018.

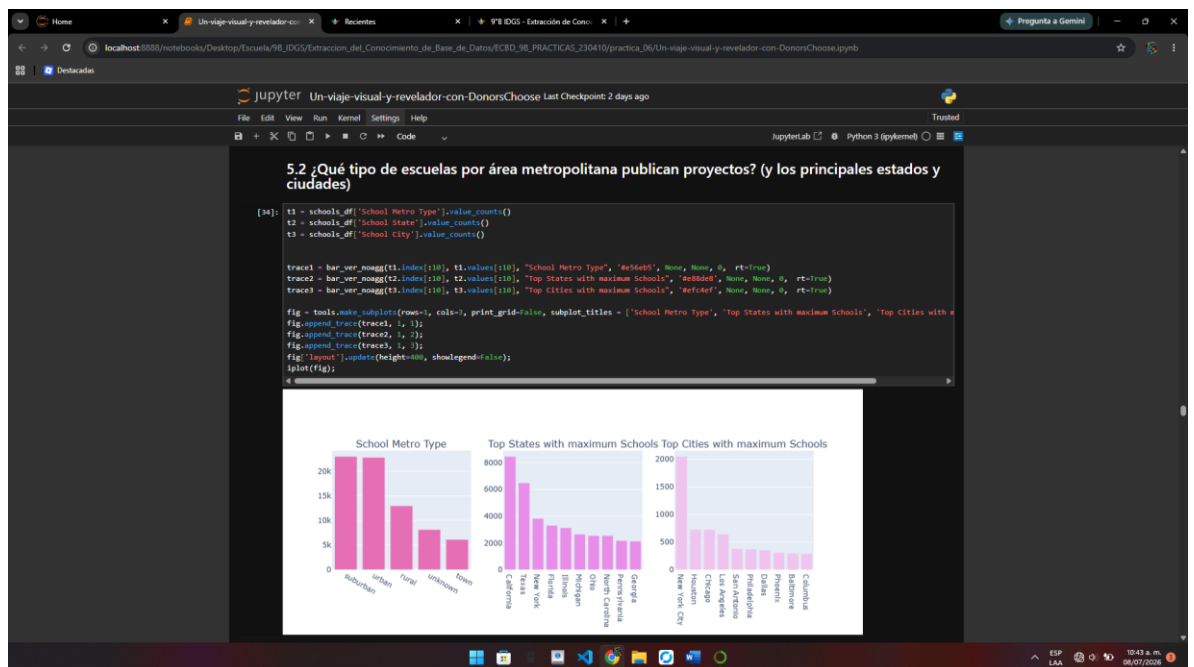
5.2 ¿Qué tipo de escuelas por área metropolitana publican proyectos? (y los principales estados y ciudades)

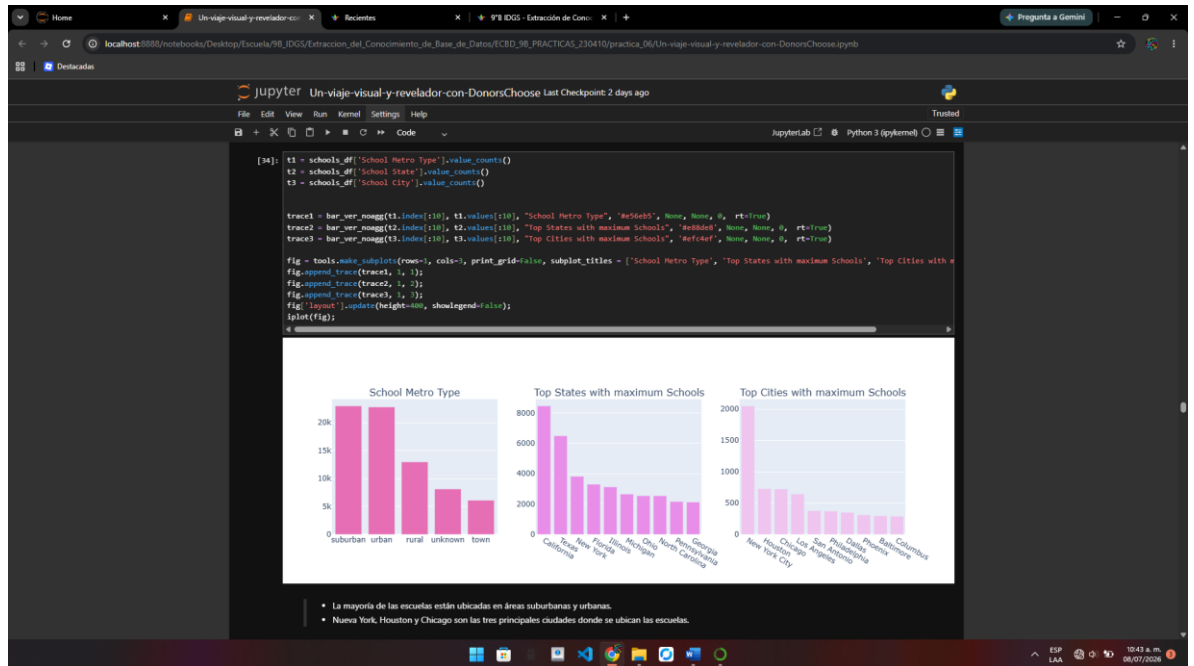
```
[34]: t1 = schools_df['School Metro Type'].value_counts()
t2 = schools_df['School State'].value_counts()
t3 = schools_df['School City'].value_counts()

trace1 = bar_ver_mnagg(t1.index[10], t1.values[10], 'School Metro Type', '#e68080', None, None, 0, rt=True)
trace2 = bar_ver_mnagg(t2.index[10], t2.values[10], 'Top States with maximum Schools', '#e68080', None, None, 0, rt=True)
trace3 = bar_ver_mnagg(t3.index[10], t3.values[10], 'Top Cities with maximum Schools', '#e68080', None, None, 0, rt=True)

fig = tools.make_subplots(rows=1, cols=3, print_grid=False, subplot_titles = ['School Metro Type', 'Top States with maximum Schools', 'Top Cities with maximum Schools'])
fig.append_trace(trace1, 1, 1)
fig.append_trace(trace2, 1, 2)
fig.append_trace(trace3, 1, 3)
```

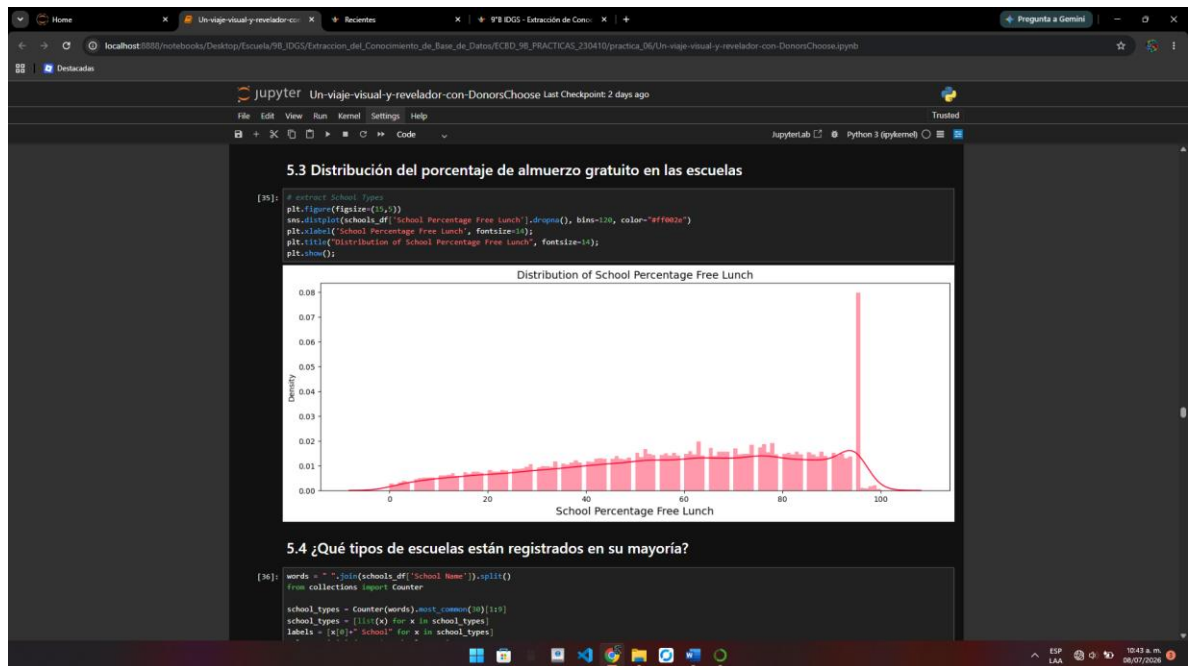
37. ¿Qué tipo de escuelas por área metropolitana publican proyectos (y los principales estados y ciudades)?
 Conteo por área metropolitana y visualización mediante subgráficos en Plotly.





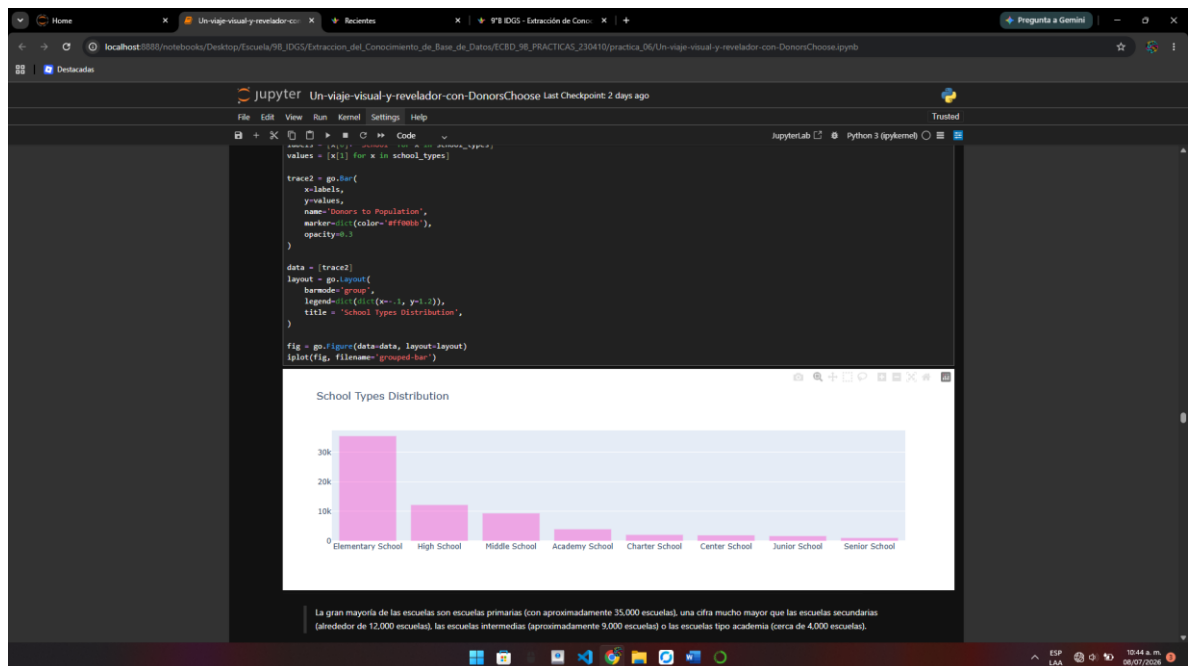
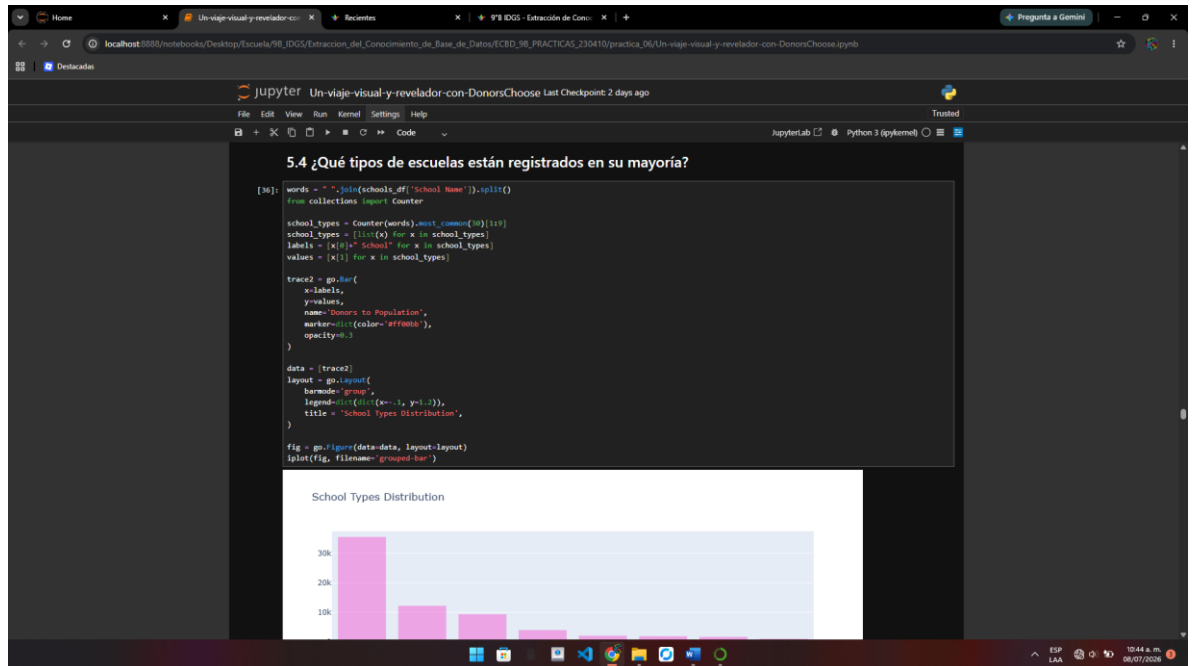
38. Distribución del porcentaje de almuerzo gratuito en las escuelas

Análisis de la distribución mediante histogramas con Seaborn y Matplotlib.



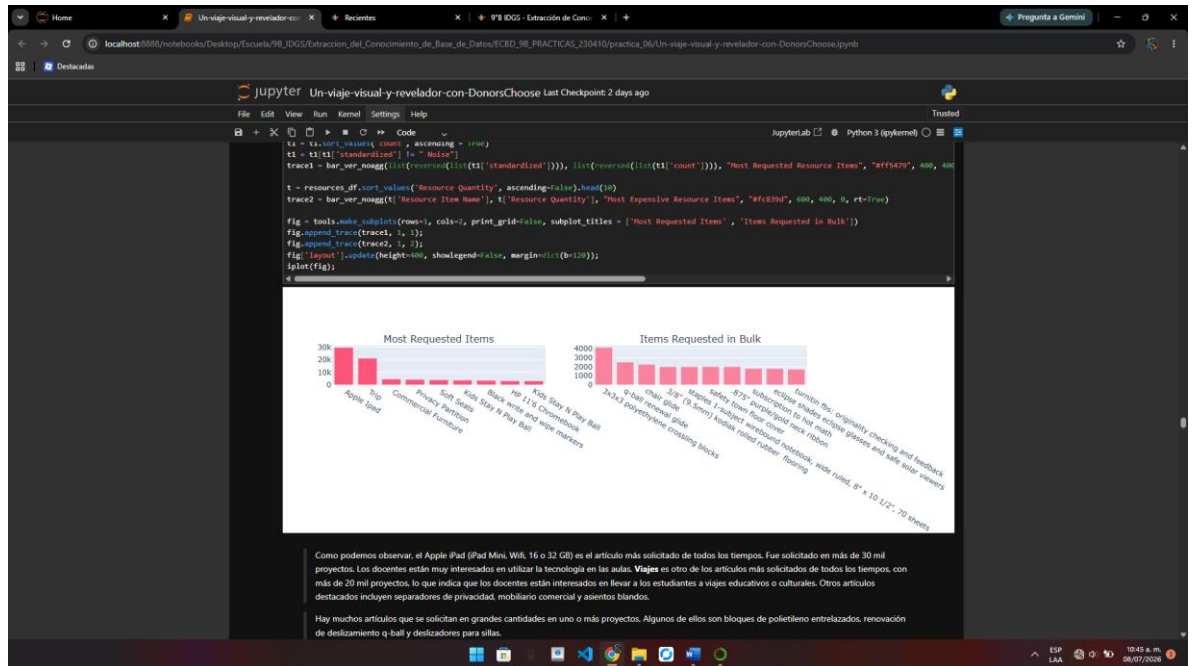
39. ¿Qué tipos de escuelas están registrados en su mayoría?

Conteo de categorías utilizando Counter y representación gráfica en Plotly.



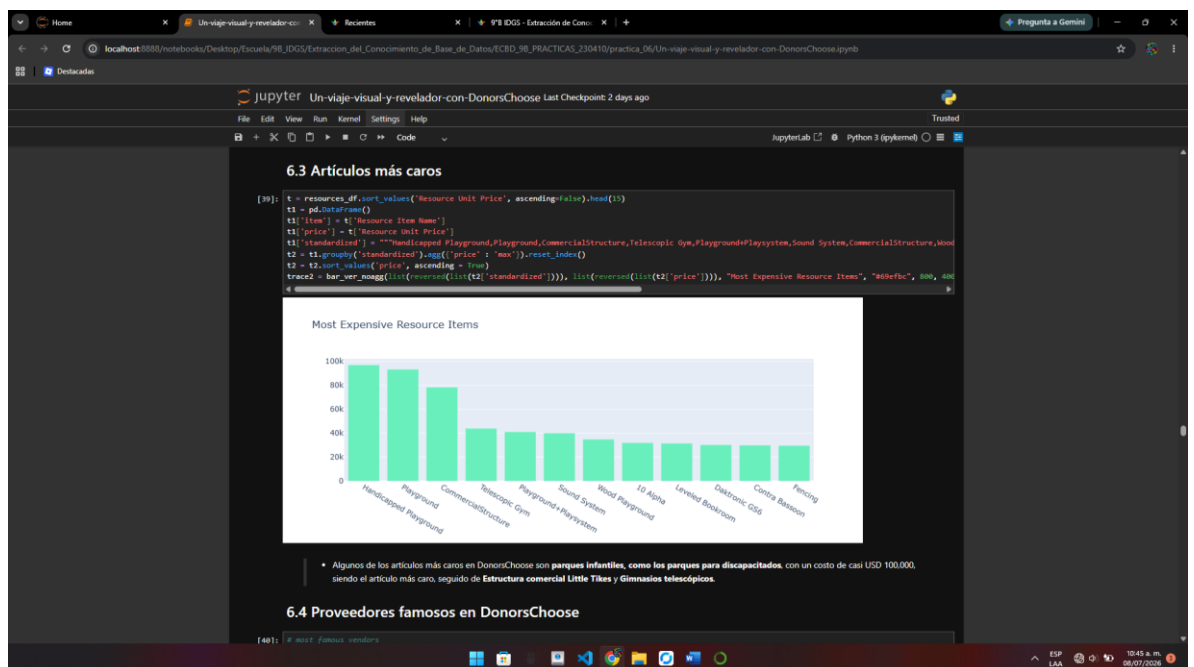
40. Instantánea de los datos de recursos

Carga y revisión inicial del conjunto de datos de recursos utilizando `read_csv()` y `head()`.

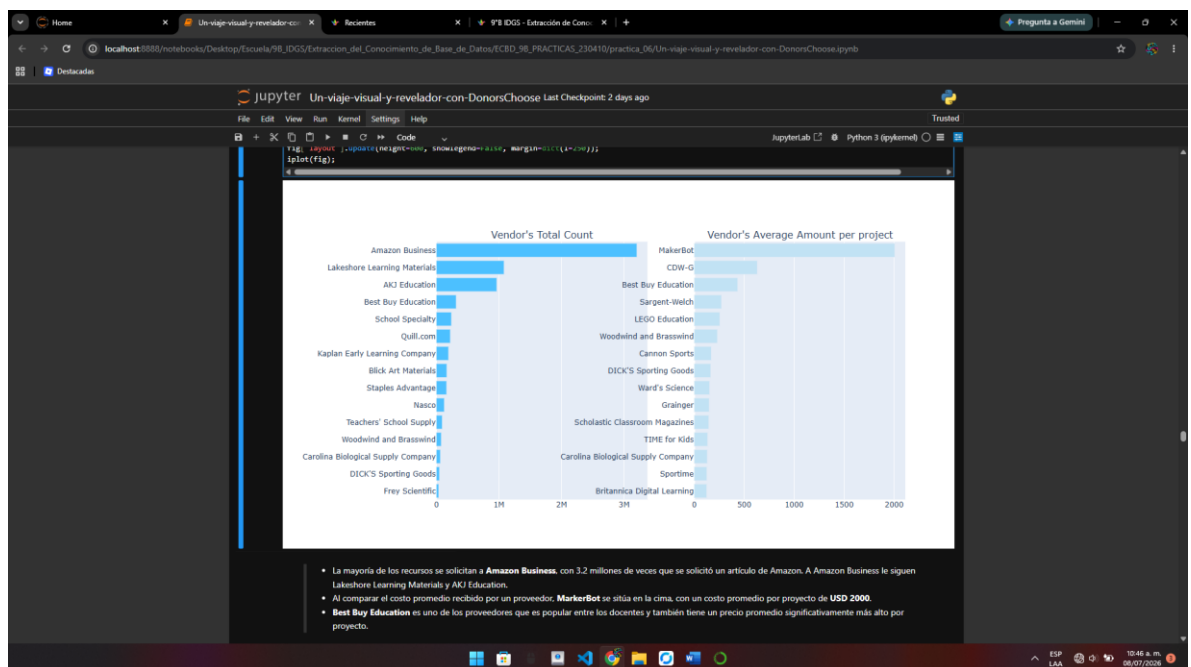
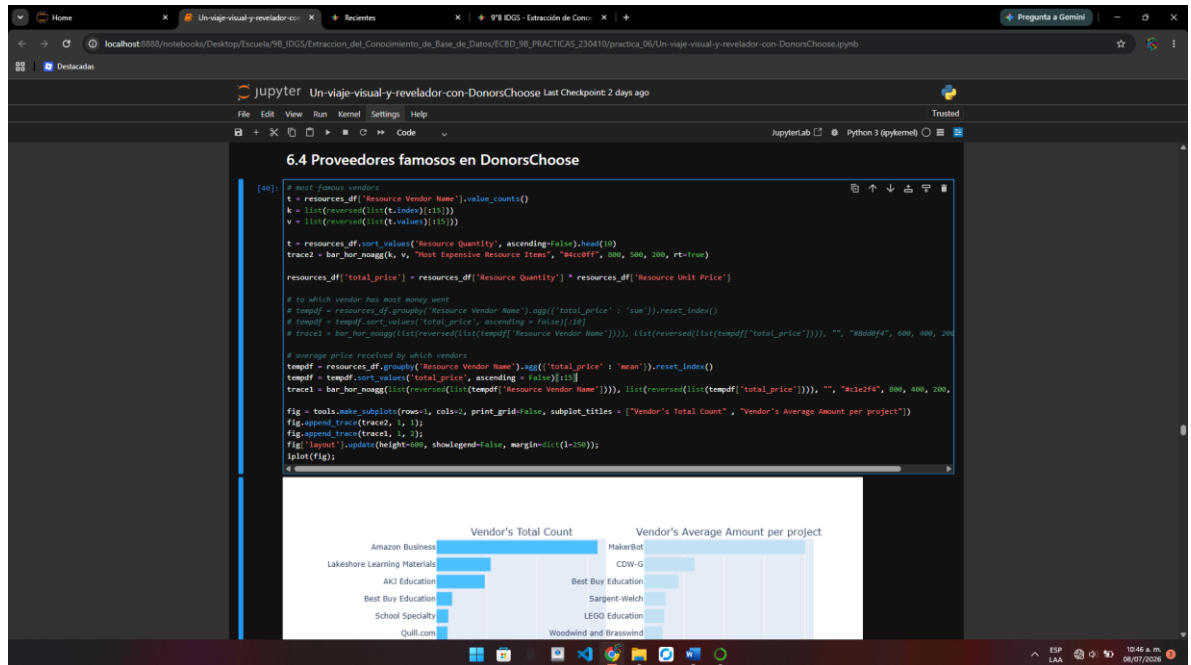


42. ¿Cuáles son los artículos más caros?

Ordenamiento de artículos por precio utilizando `sort_values()` y visualización en Plotly.



43. ¿Quiénes son los proveedores más populares y cuál es el costo promedio asociado con ellos?
Análisis de frecuencia de proveedores y costo promedio mediante pandas y Plotly.



44. Instantánea de los datos de proyectos
Carga del conjunto de datos y conversión de variables de fecha mediante `to_datetime()`.

Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint: 2 days ago

7. Exploración de los proyectos

Los datos de proyectos contienen todos los detalles sobre los proyectos publicados, incluyendo el ID del docente y la escuela, el título, el tipo, la categoría, el costo, etc. Echemos un vistazo a la instantánea.

7.1 Instantánea de los proyectos

```
[42]: projects_df = pd.read_csv(path="Projects.csv")
projects_df.head()
```

	Project ID	School ID	Teacher ID	Teacher Project Posted Sequence	Project Type	Project Title	Project Easy	Project Short Description
0	76858026a19d7b52a470ee4bc383ba	e180c7424cb6c68cb49f141b092a98f	4ee5200e939e299ec3baad8a3c5968	25	Teacher-Led	Stand Up to Bullying: Together We Can!	Did you know that 1-7 students in grades K-12 ...	Did you know that 1-7 students in grades K-12 ...
1	f94a7f090061b4b64642a303e5c331	08b20f1e2125103ed7aa17e8d76c71d4	caa2d4d777b4ad50147b49dc3b156	3	Teacher-Led	Learning in Color	Help us have a fun, interactive listening cent...	Help us have a fun, interactive listening cent...
2	af099a01713ad557b51b1ba0174e832	12875128b1f3b0fb434e5705a7c0ad	6c3bd0d4f20547a00162baef071de0fe	1	Teacher-Led	Second Grade ESL Students Develop Language...	Help Visiting or moving to a new place can be very ...	Help Visiting or moving to a new place can be very ...

7.2 ¿Cuáles son las categorías, subcategorías y categorías de recursos más frecuentes en los proyectos publicados?

```
[43]: # Create some date time features
projects_df['Posted Date'] = pd.to_datetime(projects_df['Project Posted Date'])
```

45. Principales categorías, subcategorías y categorías de recursos de los proyectos

Conteo de categorías utilizando pandas y representación gráfica en Plotly.

7.2 ¿Cuáles son las categorías, subcategorías y categorías de recursos más frecuentes en los proyectos publicados?

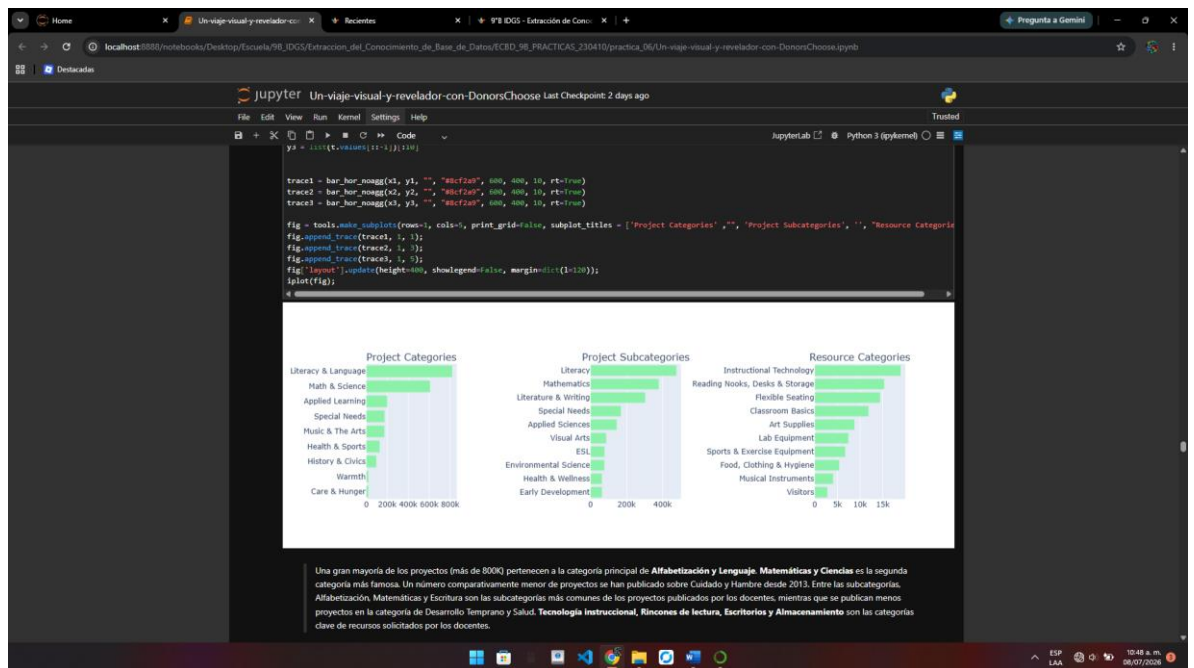
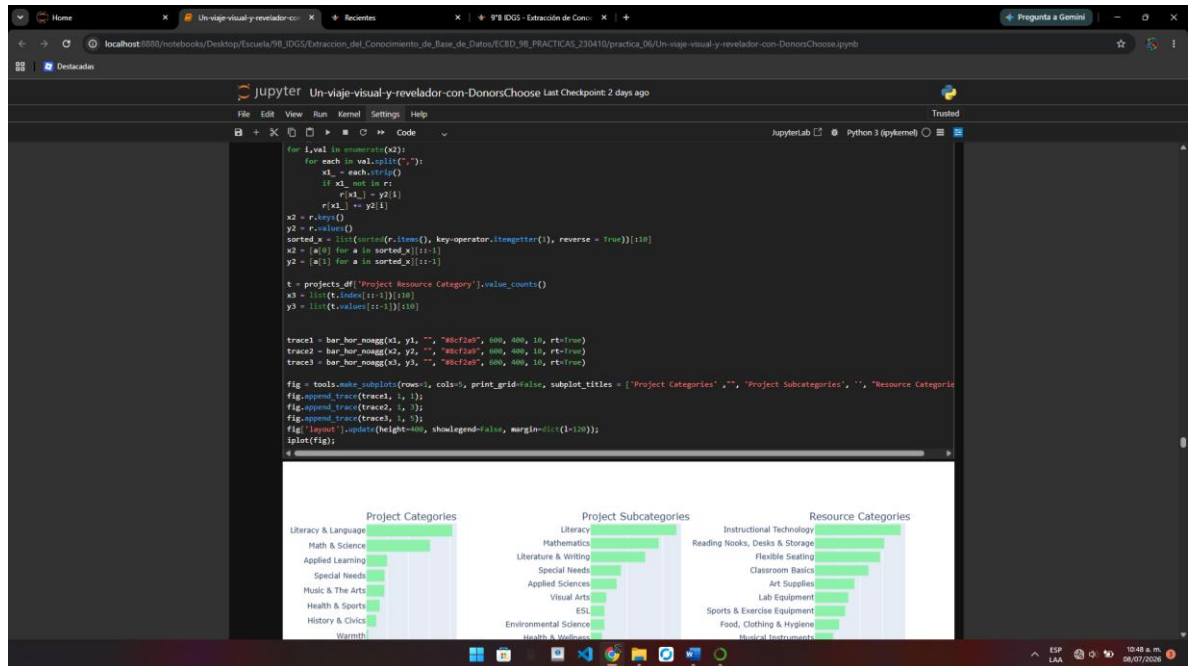
```
[43]: # Create some date time features
projects_df['Posted Date'] = pd.to_datetime(projects_df['Project Posted Date'])

projects_df['Posted Year'] = projects_df['Posted Date'].dt.year
projects_df['Posted Month'] = projects_df['Posted Date'].dt.month
projects_df['Posted Day'] = projects_df['Posted Date'].dt.day
projects_df['Posted Quarter'] = projects_df['Posted Date'].dt.quarter

projects_df['Funded Date'] = pd.to_datetime(projects_df['Project Fully Funded Date'])
projects_df['Funded Year'] = projects_df['Funded Date'].dt.year
projects_df['Funded Month'] = projects_df['Funded Date'].dt.month
projects_df['Funded Day'] = projects_df['Funded Date'].dt.day
projects_df['Funded Quarter'] = projects_df['Funded Date'].dt.quarter
projects_df['Funding Time'] = projects_df['Funded Date'] - projects_df['Posted Date']

t = projects_df['Project Subject Category Tree'].value_counts()
x = list(t.index)
y = list(t.values)
r = {}
for i, val in enumerate(x):
    for each in val.split(","):
        xi = each.strip()
        if xi not in r:
            r[xi] = y[i]
        else:
            r[xi] += y[i]
sorted_x = list(sorted(r.items(), key=operator.itemgetter(1), reverse=True))
xi = [a[0] for a in sorted_x][1:-1]
yi = [a[1] for a in sorted_x][1:-1]

t1 = projects_df['Project Subject Subcategory Tree'].value_counts()
x1 = list(t1.index)
y1 = list(t1.values)
r1 = {}
for i, val in enumerate(x1):
    for each in val.split(","):
```

46. Tipo de proyecto y estado del proyecto

Análisis de la distribución mediante gráficos de dona (go.Pie) en Plotly.

```

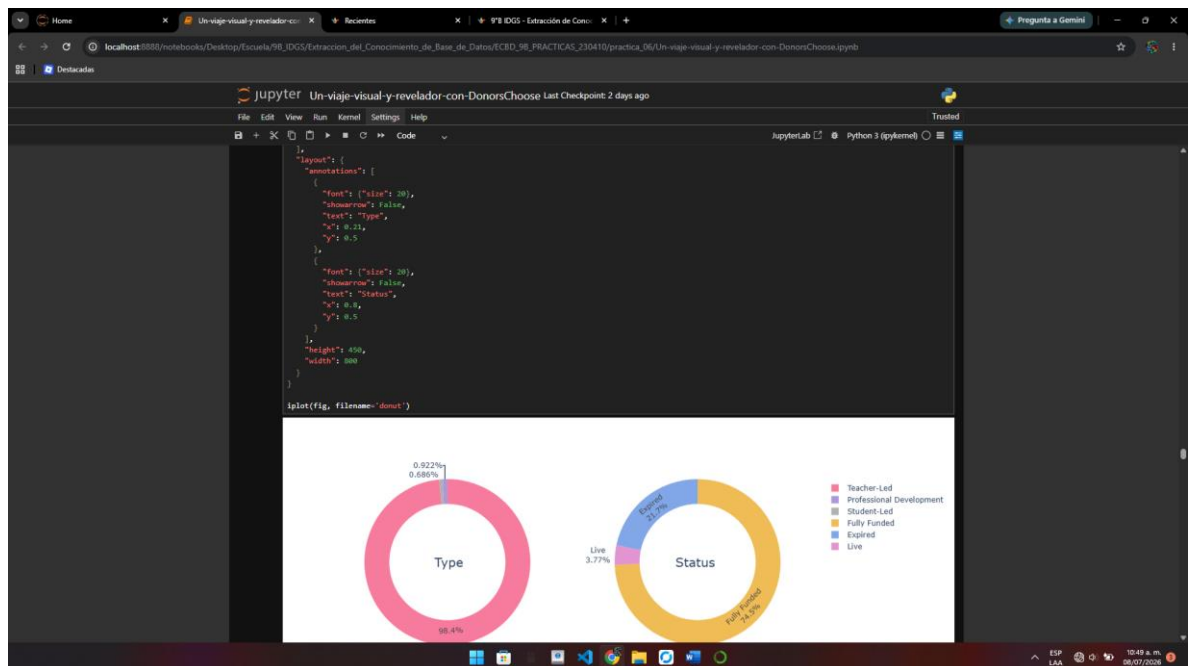
[44]: import pandas as pd
import plotly.graph_objects as go
from plotly.offline import iplot

# Calcular las frecuencias
t = projects_df['Project Type'].value_counts()
values1 = t.values
index1 = t.index

t = projects_df['Project Current Status'].value_counts()
values2 = t.values
index2 = t.index

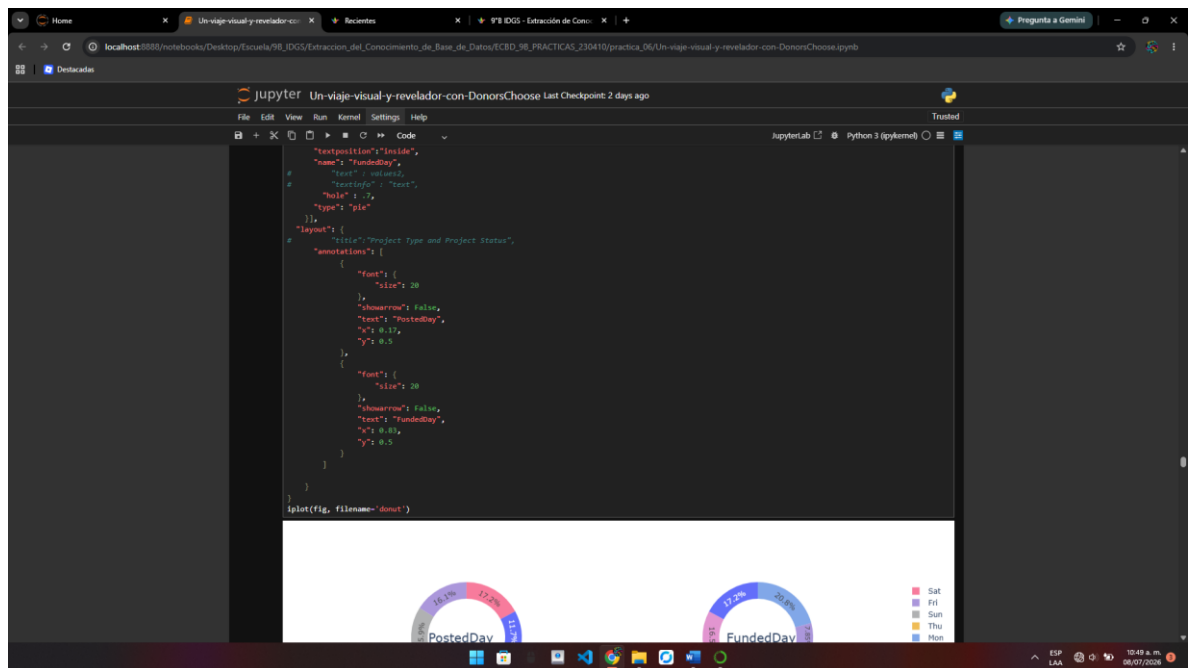
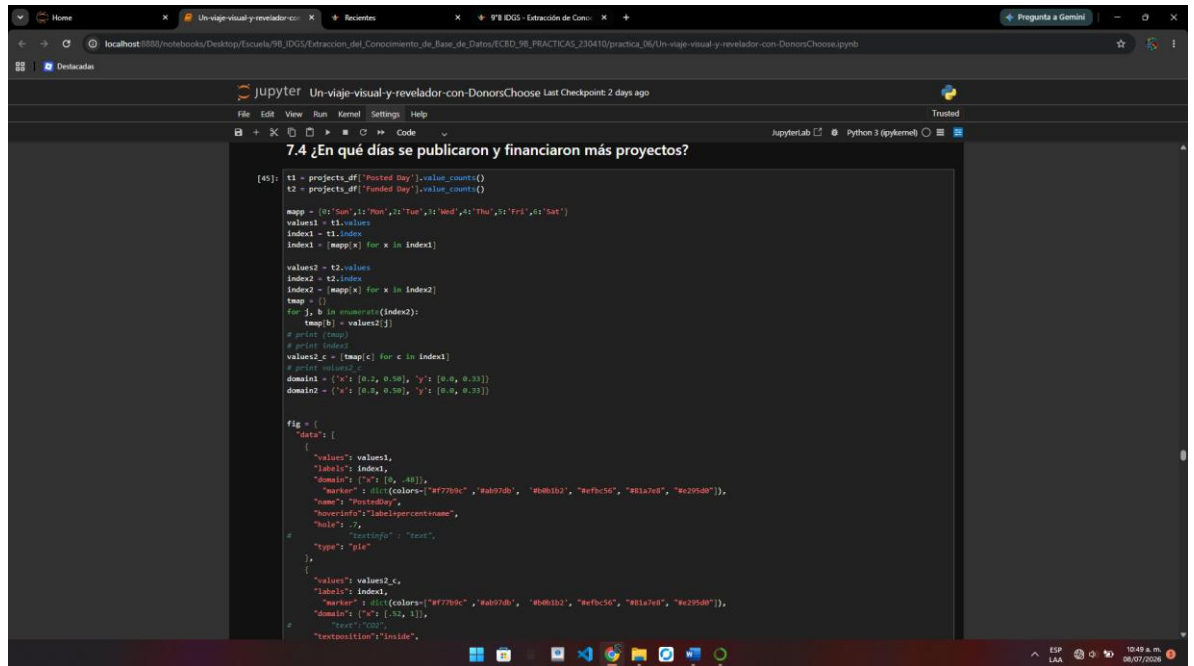
# Definir el figura con dos gráficos de dona
fig = {
    "data": [
        {
            "values": values1,
            "labels": index1,
            "domain": [{"x1": 0, "x2": 40}],
            "marker": dict(colors=["#f7941d", "#8070b3", "#4d1b12"]),
            "name": "Project Type",
            "hoverinfo": "labelpercentname",
            "hole": .3,
            "type": "pie"
        },
        {
            "values": values2,
            "labels": index2,
            "marker": dict(colors=["#fbc02d", "#81c784", "#42a5f5"]),
            "domain": [{"x1": 40, "x2": 100}],
            "name": "Project Status",
            "hoverinfo": "labelpercent", # x!!! COMPLETAR: usar textinfo en lugar de text
            "hole": .3,
            "type": "pie"
        }
    ],
    "layout": {
        "annotations": [

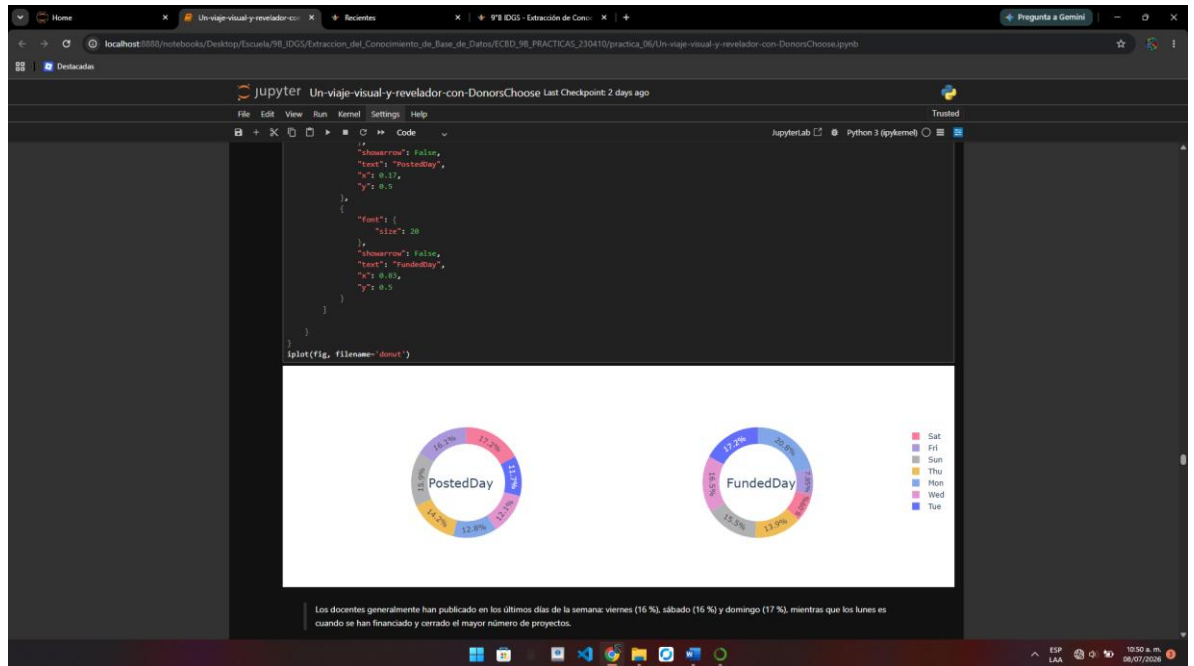
```



47. Día de publicación y día de financiación

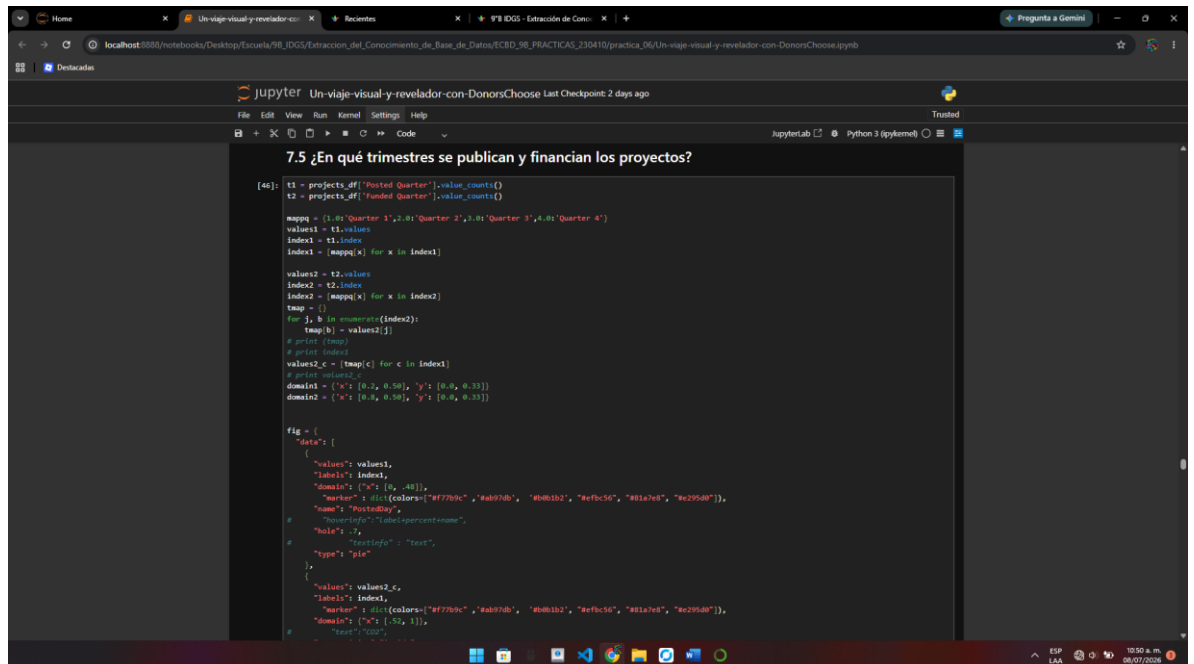
Comparación de los días de publicación y financiamiento utilizando `.dt.weekday` y Plotly.





48. Trimestre de publicación y trimestre de financiación

Comparación de trimestres mediante `.dt.quarter` y gráficos en Plotly.



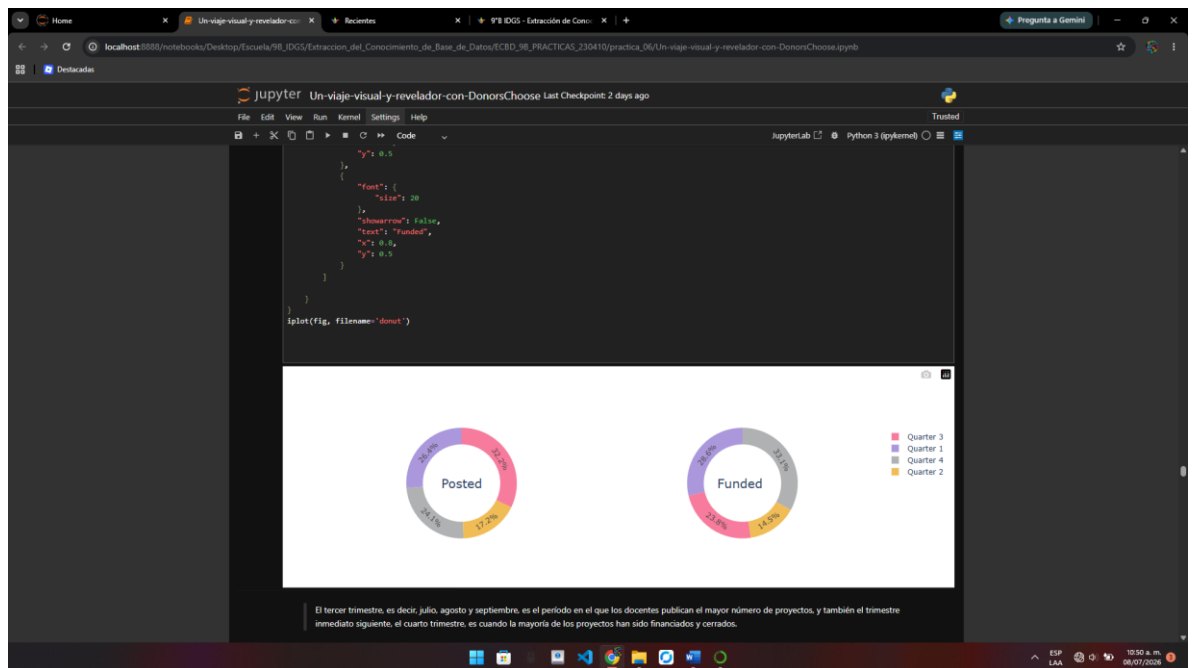
```
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago

File Edit View Run Kernel Settings Help

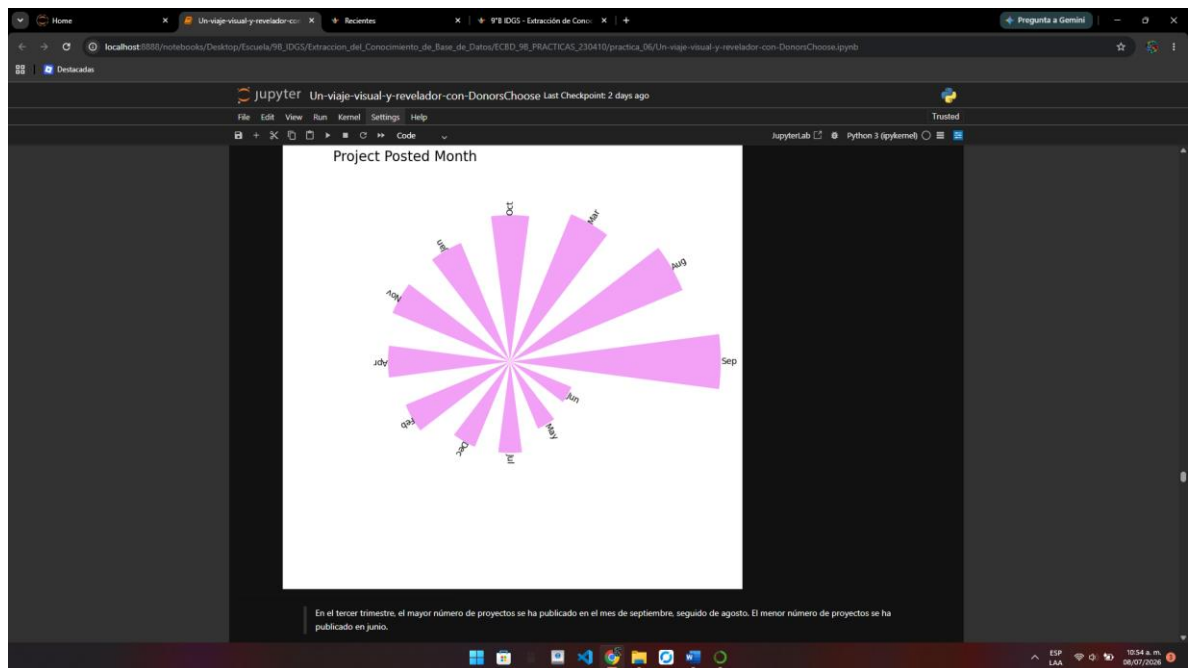
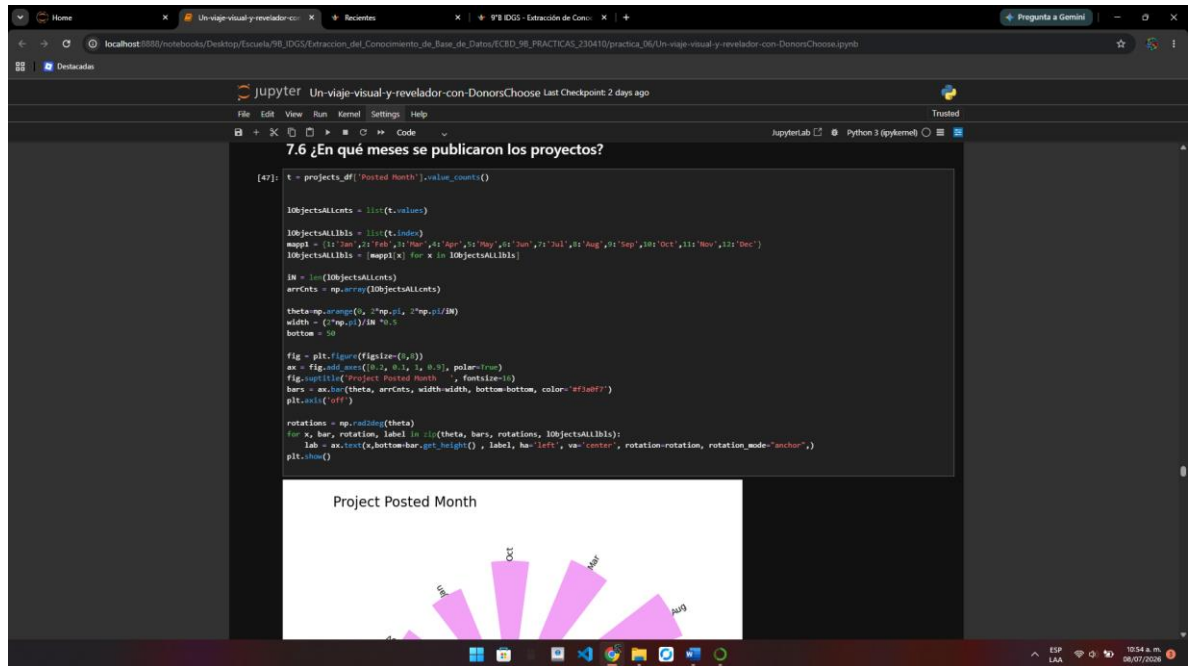
JupyterLab Python 3 (ipykernel)

{
  "values": values2_c,
  "labels": labels,
  "radius": 0.5,
  "domain": [{"x": -0.5, "y": 0.5}],
  "text": "Posted",
  "textposition": "inside",
  "name": "Fundador",
  "text": "Fundador",
  "textcolor": "black",
  "hole": 0.5,
  "type": "pie"
},
{
  "values": values2_c,
  "labels": labels,
  "radius": 0.5,
  "domain": [{"x": -0.5, "y": 0.5}],
  "text": "Funded",
  "textposition": "inside",
  "name": "Fundador",
  "text": "Fundador",
  "textcolor": "black",
  "hole": 0.5,
  "type": "pie"
}
]

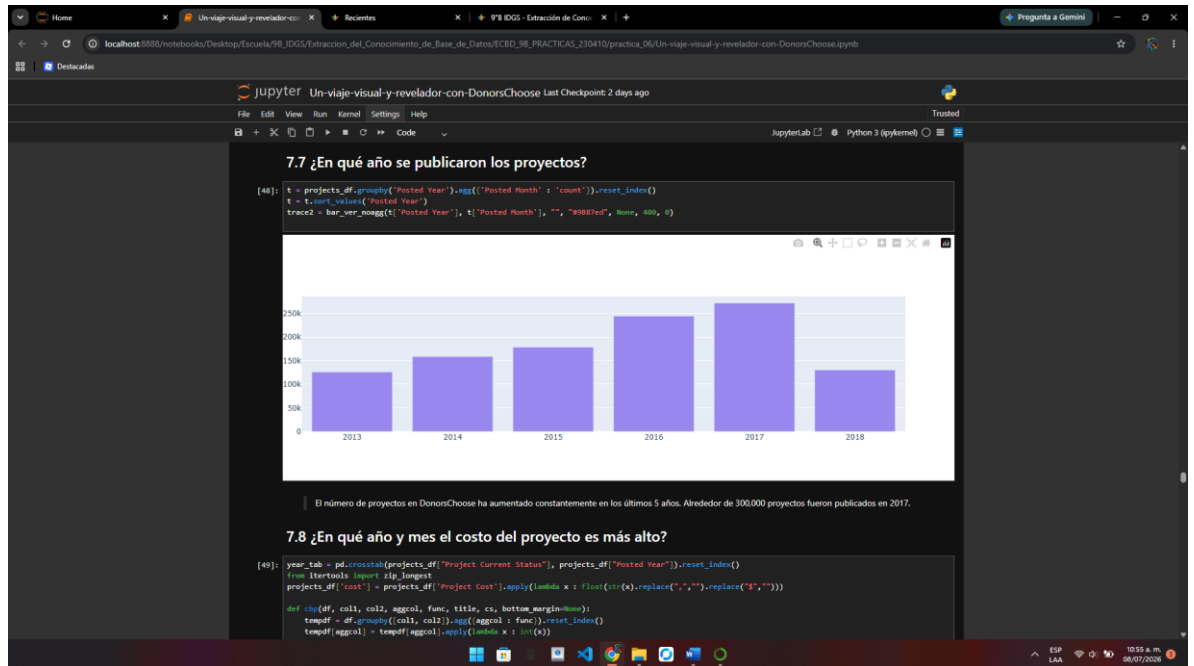
fig = plt.figure(figsize=(10, 10))
fig.add_subplot(111)
fig.pie(values2_c, labels=labels, radius=0.5, domain=[{"x": -0.5, "y": 0.5}], text="Posted", textposition="inside", name="Fundador", textcolor="black", hole=0.5, type="pie")
fig.add_subplot(111)
fig.pie(values2_c, labels=labels, radius=0.5, domain=[{"x": -0.5, "y": 0.5}], text="Funded", textposition="inside", name="Fundador", textcolor="black", hole=0.5, type="pie")
fig.savefig("donut.png")
```



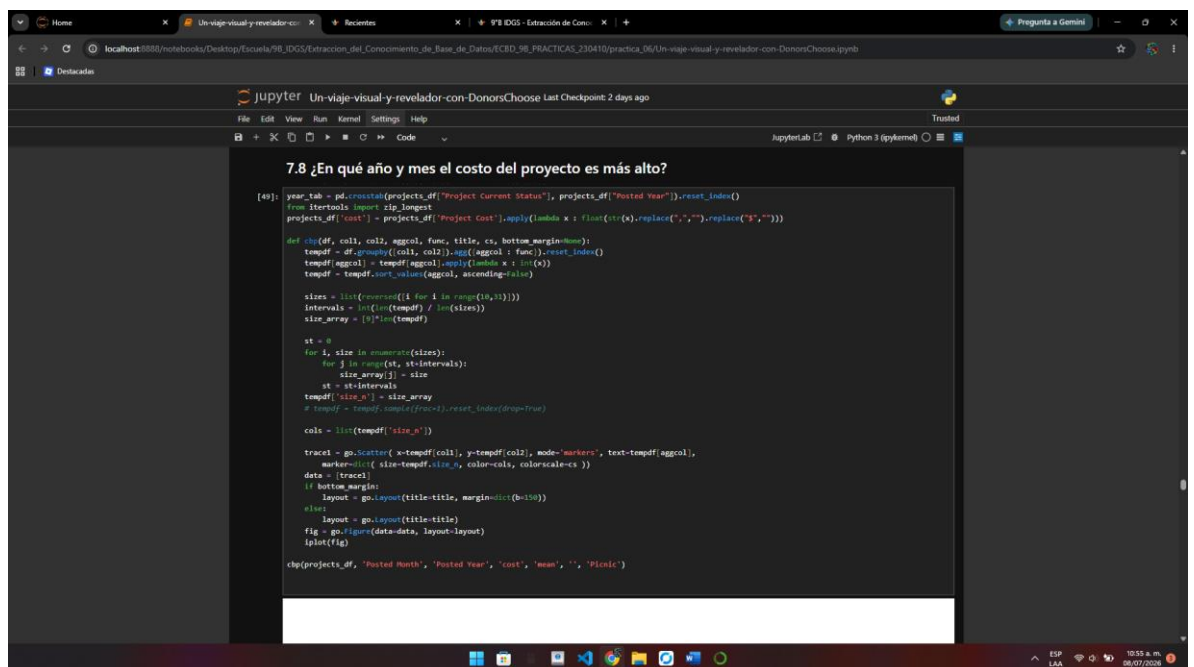
49. ¿En qué mes se publican la mayoría de los proyectos?
Representación de la distribución mensual mediante gráficos polares en Matplotlib.

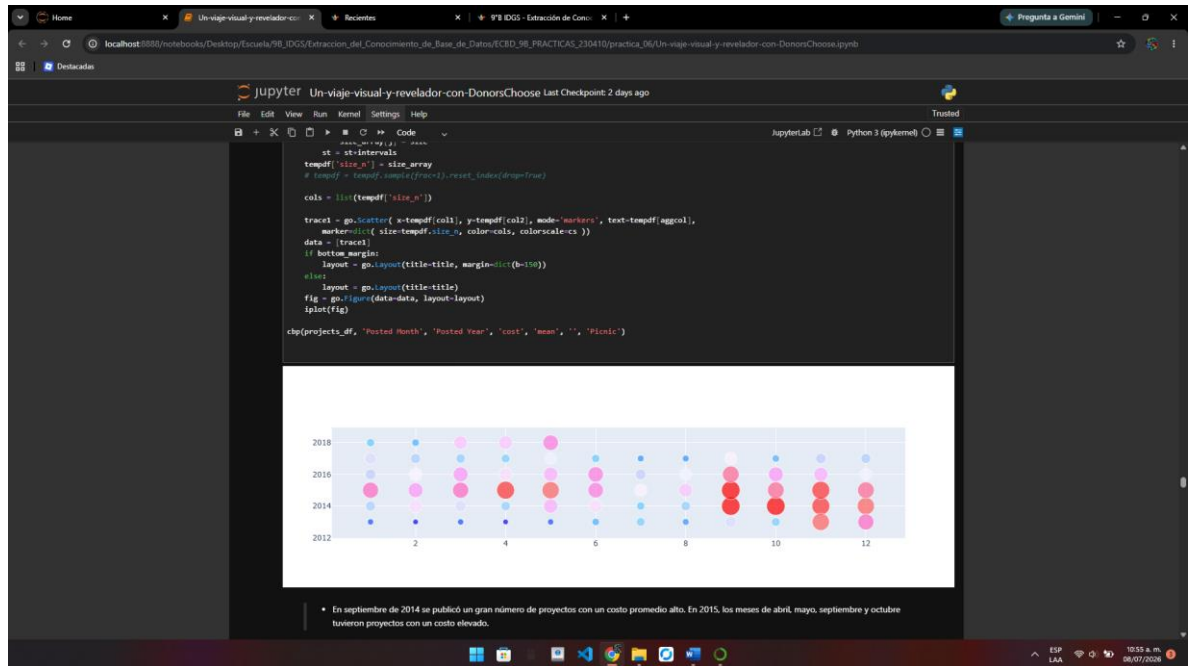


50. **¿Está aumentando el número de proyectos?**
Análisis de la tendencia anual mediante `groupby()` y series temporales en Plotly.

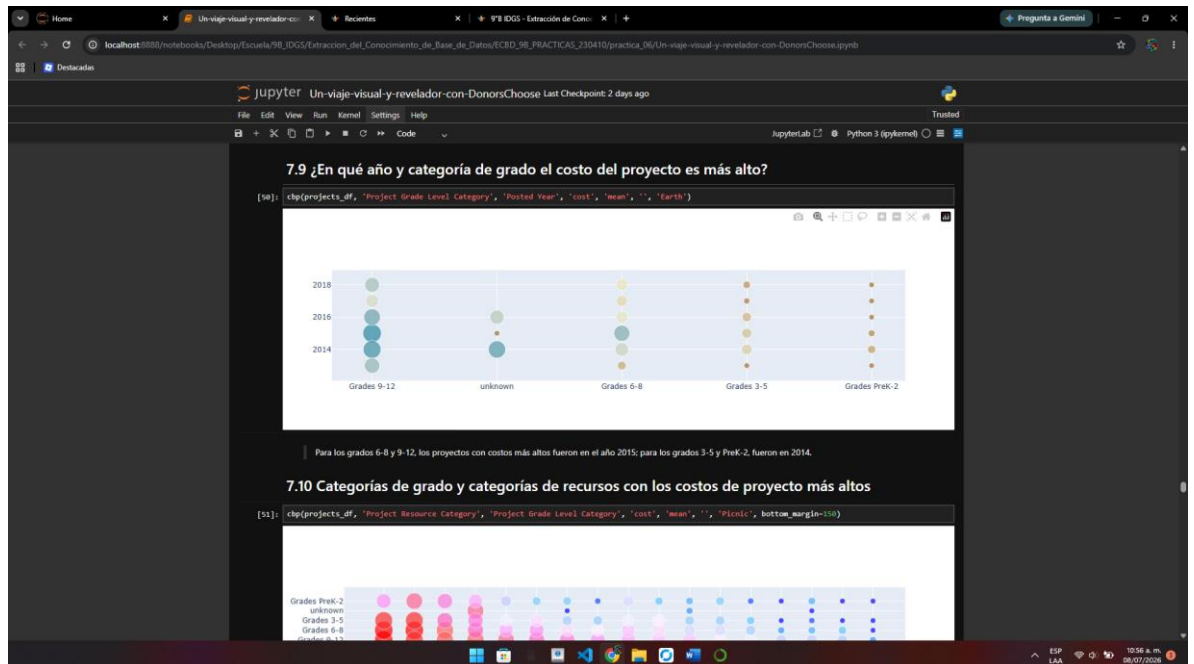


51. ¿En qué año y mes el costo del proyecto es más alto?
Identificación de los mayores costos mediante una función personalizada (`cbp`) y Plotly.

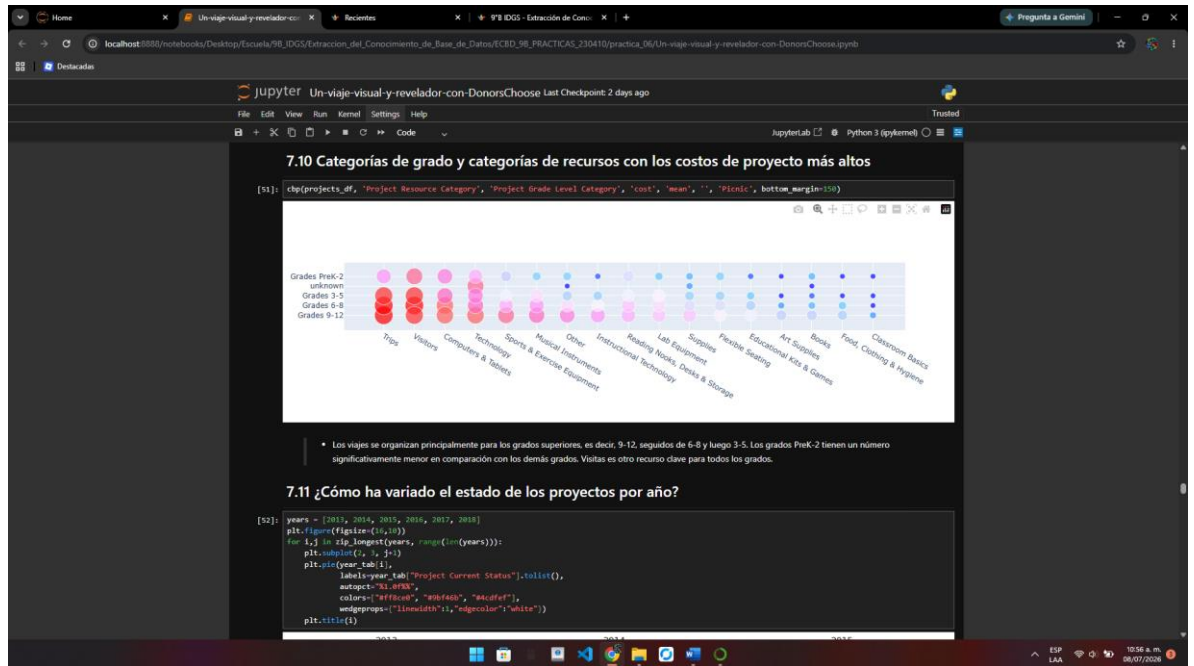




52. ¿En qué año y categoría de grado el costo del proyecto es más alto?
Comparación de costos por año y grado escolar utilizando la función `cbp`.

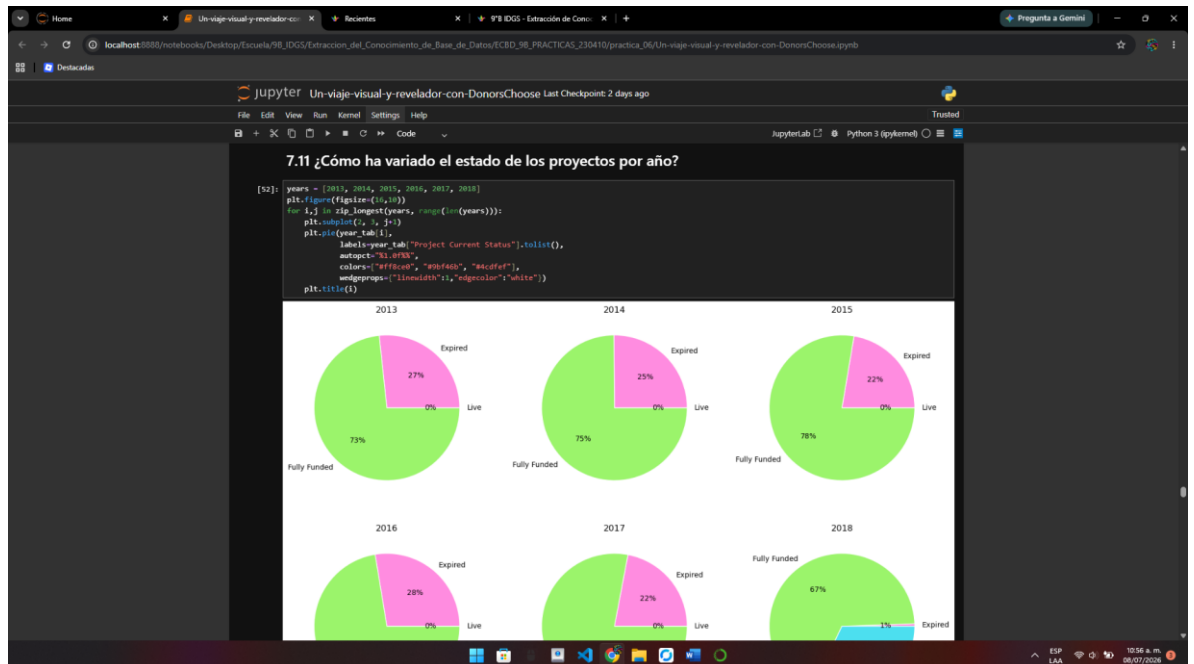


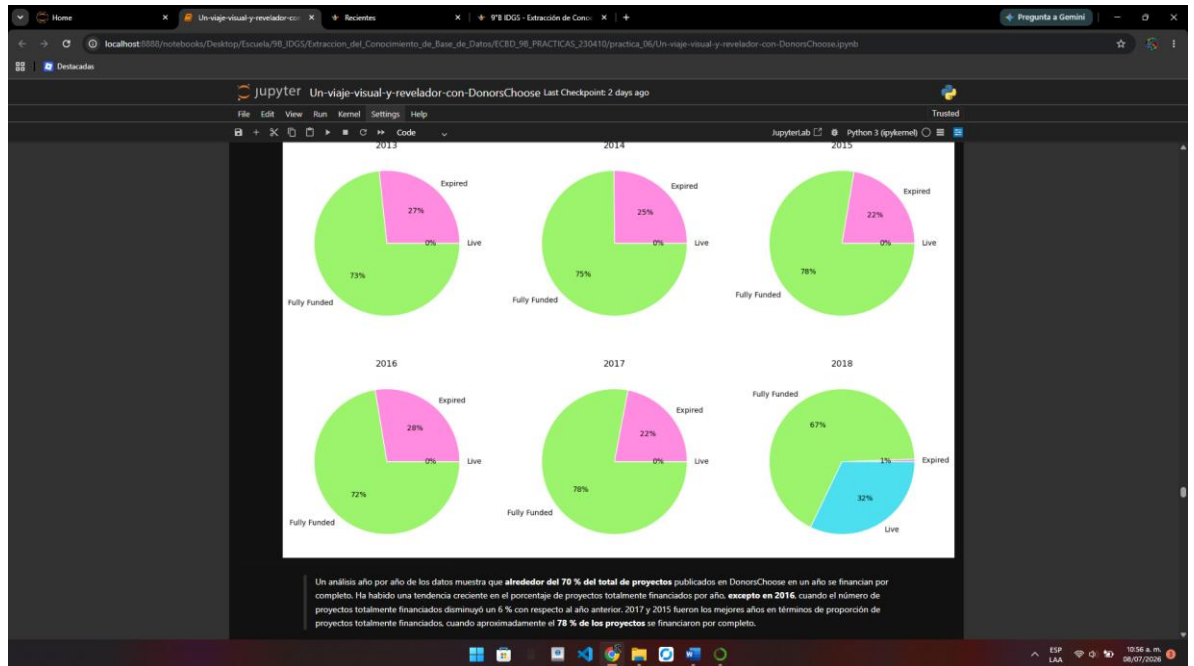
53. ¿Para qué categoría de grado y categoría de recurso el costo del proyecto es más alto?
Análisis de costos por grado y categoría de recurso mediante la función `cbp`.



54. Estado del proyecto por años

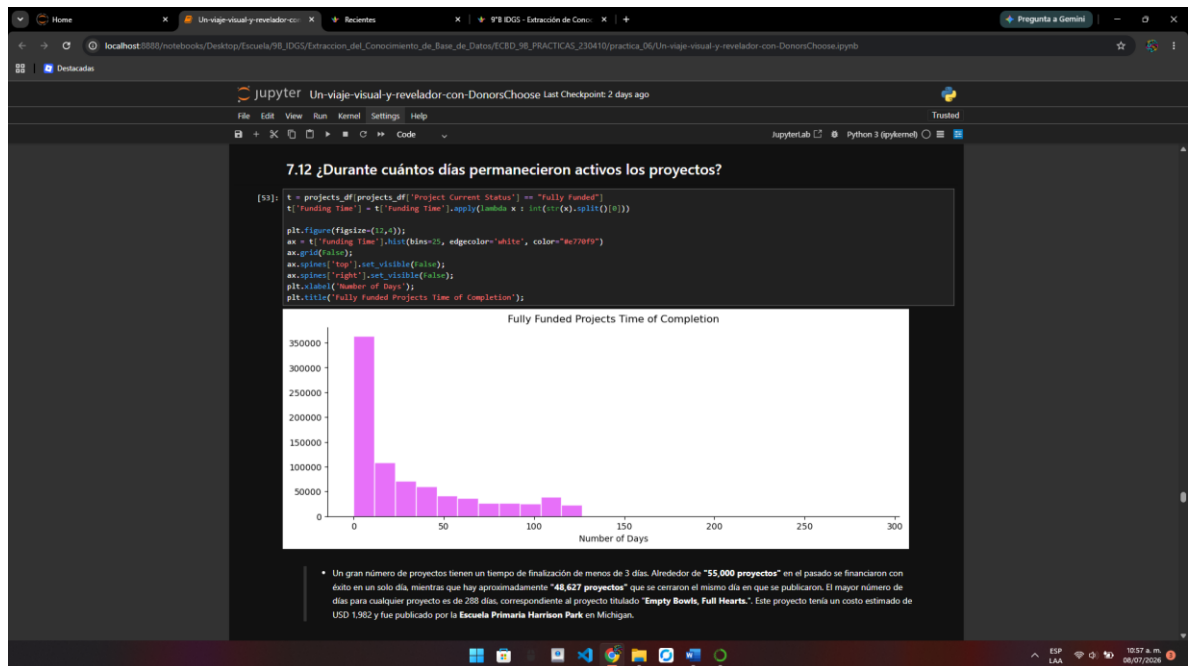
Elaboración de tablas cruzadas (`pd.crosstab`) y gráficos de pastel con Matplotlib.





55. Duración de los proyectos financiados con éxito

Análisis de la duración mediante histogramas en Matplotlib.



56. Estado del proyecto por diferentes estados

Integración de información mediante `merge()` y representación gráfica en Plotly.

```

[54]: projcsch_df = projects_df.merge(schools_df, on="School ID")

states = ['California', 'Texas', 'New York', 'Florida', 'North Carolina',
          'Illinois', 'Georgia', 'South Carolina', 'Michigan',
          'Pennsylvania', 'Massachusetts', 'Indiana', 'Oklahoma',
          'Washington', 'Ohio', 'New Jersey', 'Missouri', 'Arizona',
          'Virginia', 'Louisiana', 'Tennessee', 'Utah', 'Wisconsin',
          'Connecticut', 'Alabama', 'Maryland', 'Nevada', 'Oregon',
          'Colorado', 'Mississippi', 'Minnesota', 'Kentucky', 'Arkansas',
          'Hawaii', 'Idaho', 'Maine', 'Kansas', 'Iowa',
          'New Mexico', 'West Virginia', 'Alaska',
          'New Hampshire', 'Delaware', 'Rhode Island', 'Vermont',
          'South Dakota', 'Montana', 'North Dakota', 'Wyoming']

import itertools
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

length = len(states)
plt.figure(figsize=(15,20))

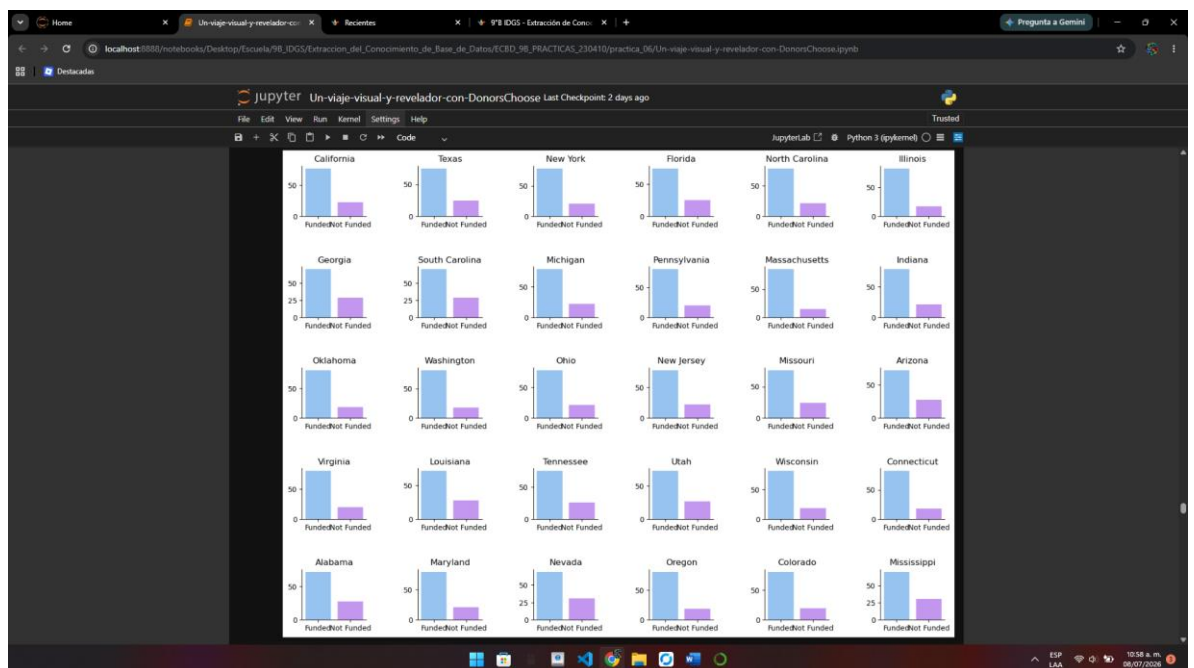
for i, j in itertools.zip_longest(states, range(length)):
    plt.subplot(10, 5, j+1)

    t = dist(projcsch_df[projcsch_df['School State'] == i]['Project Current Status'].value_counts())

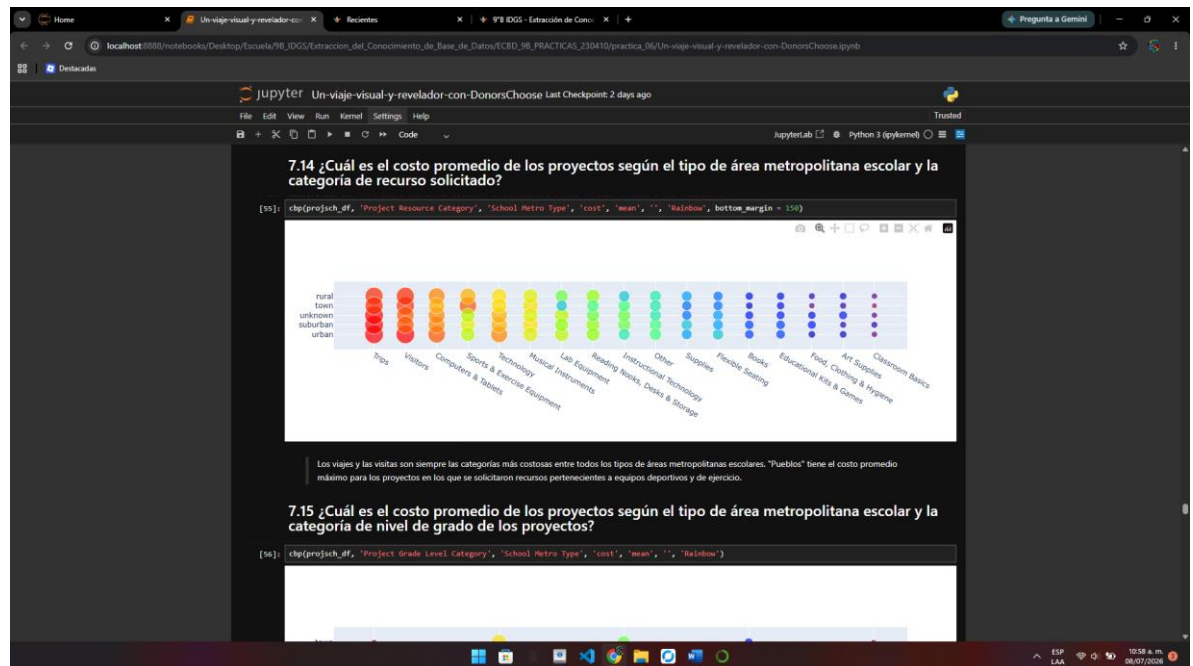
    # Esto ha cambiado en la nueva versión
    t['Not Funded'] = t['Funded']
    t['Total'] = t['Fully Funded'] + t['Not Funded']
    t1 = ('Funded': float(t['Fully Funded']) * 100 / t['Total'],
          'Not Funded': float(t['Not Funded']) * 100 / t['Total'])

    # CORRECCIÓN: usar parámetros numerados x e y
    ax = sns.barplot(x=list(t1.keys()), y=list(t1.values()), alpha=.7, palette='cool')
    ax.grid(True)
    ax.spines['top'].set_visible(False)
    ax.spines['right'].set_visible(False)
    plt.subplotpars.left = 0.05, bottom = 0.05, hspace=1
    plt.title(i, color='black')

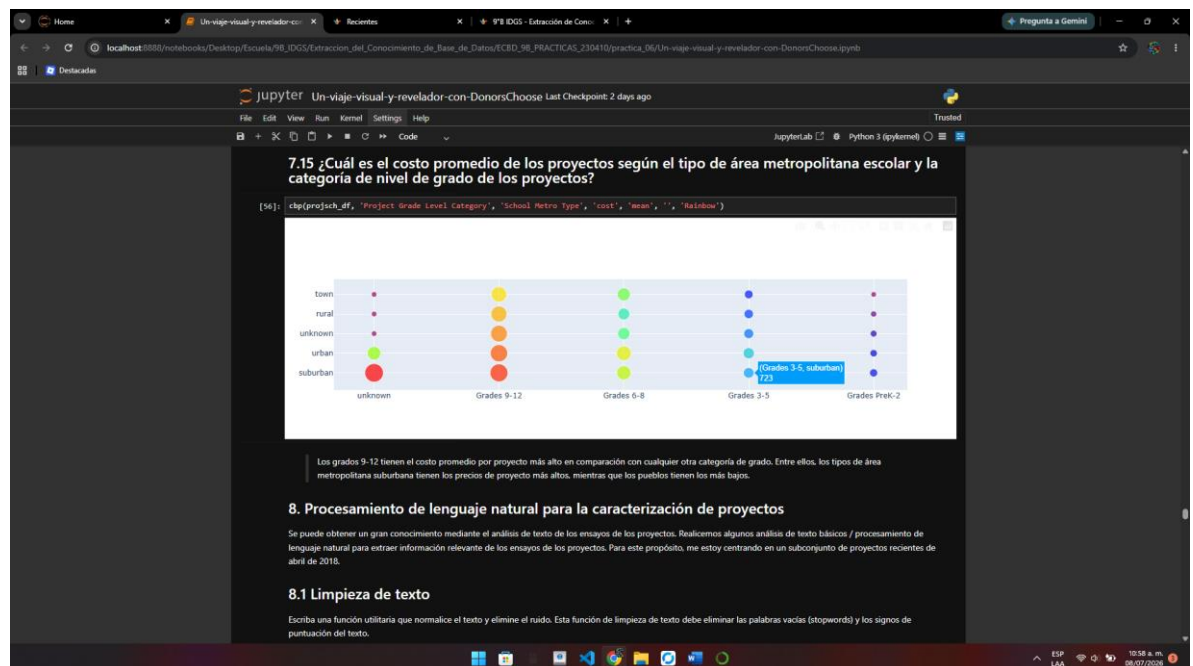
```



57. **Costo promedio de proyectos por tipo de área metropolitana escolar y categoría de recursos**
 Comparación de costos utilizando la función `cbp` y gráficos en Plotly.



58. **Costo promedio de proyectos por tipo de área metropolitana escolar y categoría de nivel de grado**
 Comparación de costos por tipo de escuela y nivel de grado utilizando la función `cbp`.



59. **Limpieza de texto**
 Limpieza y normalización del texto utilizando funciones de cadenas y `nltk.stopwords`.

8. Procesamiento de lenguaje natural para la caracterización de proyectos

Se puede obtener un gran conocimiento mediante el análisis de texto de los ensayos de los proyectos. Realicemos algunos análisis de texto básicos / procesamiento de lenguaje natural para extraer información relevante de los ensayos de los proyectos. Para este propósito, me estoy centrando en un subconjunto de proyectos recientes de abril de 2018.

8.1 Limpieza de texto

Escriba una función utilitaria que normalice el texto y elimine el ruido. Esta función de limpieza de texto debe eliminar las palabras vacías (stopwords) y los signos de puntuación del texto.

```
[57]: import nltk
nltk.download('stopwords')

from nltk.corpus import stopwords
from textblob import TextBlob
import string

stopwords = stopwords.words("english")

def clean(txt):
    # lower case
    txt = txt.lower()

    # punctuation removal
    txt = ''.join(x for x in txt if x not in string.punctuation)

    # stopwords and lemmatization
    clean_txt = ""
    for word in txt.split():
        if word in stopwords:
            continue
        clean_txt += " " + word

    noise = ['students', 'my', 'title', 'would', 'every', 'will', 'many', 'also', 'donutremovesaydivider']
    for ns in noise:
        clean_txt = clean_txt.replace(ns, "")

    return clean_txt
```

```
projects_df = projects_df.merge(schools_df, on='School ID', how='inner')
april18_df = projects_df[(projects_df['Posted Year'] == 2018) & (projects_df['Posted Month'] == 4)]
april18_df['clean_body'] = april18_df['Project body'].apply(clean)
april18_df['clean_title'] = april18_df['Project Head Statement'].apply(clean)

[nltk_data] Downloading package stopwords to
[nltk_data] C:\Users\PC-00\AppData\Localing\ltk_data...
[nltk_data] Unzipping corpora\stopwords.zip.
```

60. Análisis de bolsa de palabras

Generación de n-gramas mediante Counter, gráficos con Seaborn y Matplotlib, y nubes de palabras con WordCloud.


```
Home | Un-viaje-visual-y-revelador-con-DonorsChoose | Recientes | 9º IDG - Extracción de Conoc... | Preguntar a Gemini
localhost:8888/notebooks/Desktop/Escuela/9º_IDG/Extraccion_del_Conocimiento_de_Base_de_Datos/ECID_9º_PRACTICAS_230415/practica_06/Un-viaje-visual-y-revelador-con-DonorsChoose.ipynb
Destacadas

Jupyter Un-viaje-visual-y-revelador-con-DonorsChoose Last checkpoint 2 days ago
File Edit View Run Kernel Settings Help Trusted
Python 3 (ipykernel)

[58]: #for loop of words
from collections import Counter
def generate_ngrams(txt, N):
    grams = [txt[i:i+N] for i in range(len(txt)-N+1)]
    grams = [" ".join(g) for g in grams]
    return grams

april18_df['unigram'] = april18_df['clean_essay'].apply(lambda x: generate_ngrams(x.split(), 1))
april18_df['bigram'] = april18_df['clean_essay'].apply(lambda x: generate_ngrams(x.split(), 2))
april18_df['trigram'] = april18_df['clean_essay'].apply(lambda x: generate_ngrams(x.split(), 3))

all_unigrams = []
for each in april18_df['unigram']:
    all_unigrams.extend(each)
t = Counter(all_unigrams).most_common(20)
x = [a[0] for a in t]
y = [a[1] for a in t]

all_bigrams = []
for each in april18_df['bigram']:
    all_bigrams.extend(each)
t1 = Counter(all_bigrams).most_common(20)
x1 = [a[0] for a in t1]
y1 = [a[1] for a in t1]

all_trigrams = []
for each in april18_df['trigram']:
    all_trigrams.extend(each)
t2 = Counter(all_trigrams).most_common(20)
x2 = [a[0] for a in t2]
y2 = [a[1] for a in t2]

sns.set_style("dark")
fig, axes = plt.subplots(nrows=3, ncols=3, squeeze=True, figsize=(12,10))

bar = sns.barplot(y=x, x=y, ax=axes[0], palette="magma", edgecolor="white");
bar.set(xlabel="", yticks=[]);

#for loop of words
from collections import Counter
def generate_ngrams(txt, N):
    grams = [txt[i:i+N] for i in range(len(txt)-N+1)]
    grams = [" ".join(g) for g in grams]
    return grams

april18_df['unigram'] = april18_df['clean_essay'].apply(lambda x: generate_ngrams(x.split(), 1))
april18_df['bigram'] = april18_df['clean_essay'].apply(lambda x: generate_ngrams(x.split(), 2))
april18_df['trigram'] = april18_df['clean_essay'].apply(lambda x: generate_ngrams(x.split(), 3))

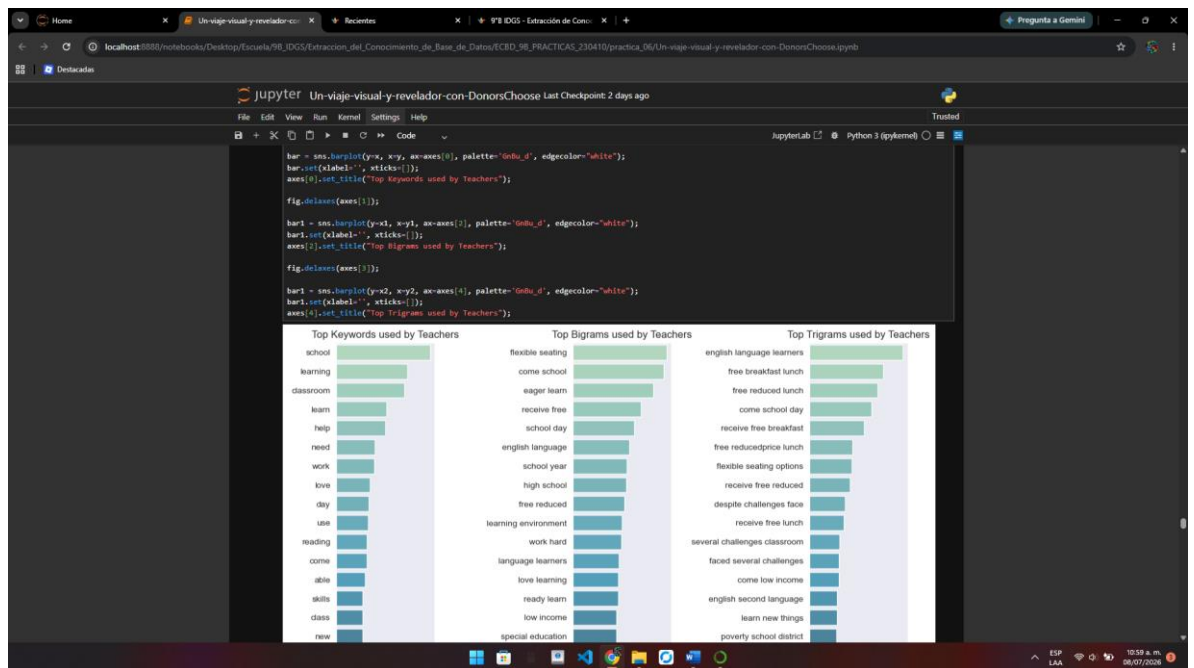
all_unigrams = []
for each in april18_df['unigram']:
    all_unigrams.extend(each)
t = Counter(all_unigrams).most_common(20)
x = [a[0] for a in t]
y = [a[1] for a in t]

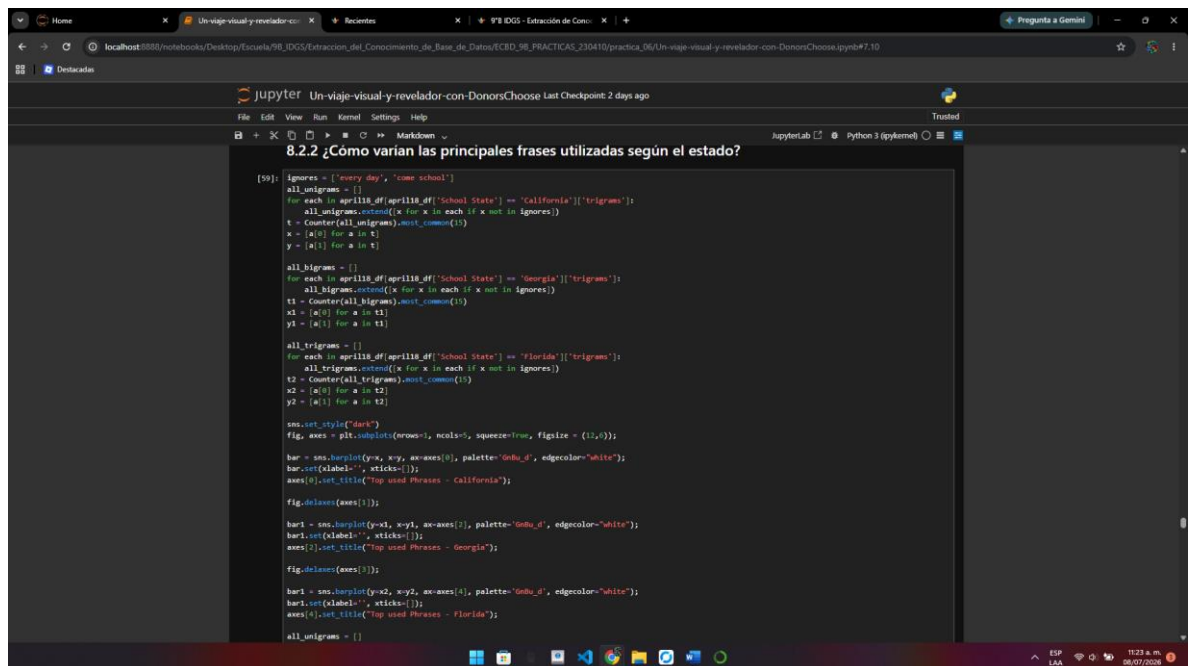
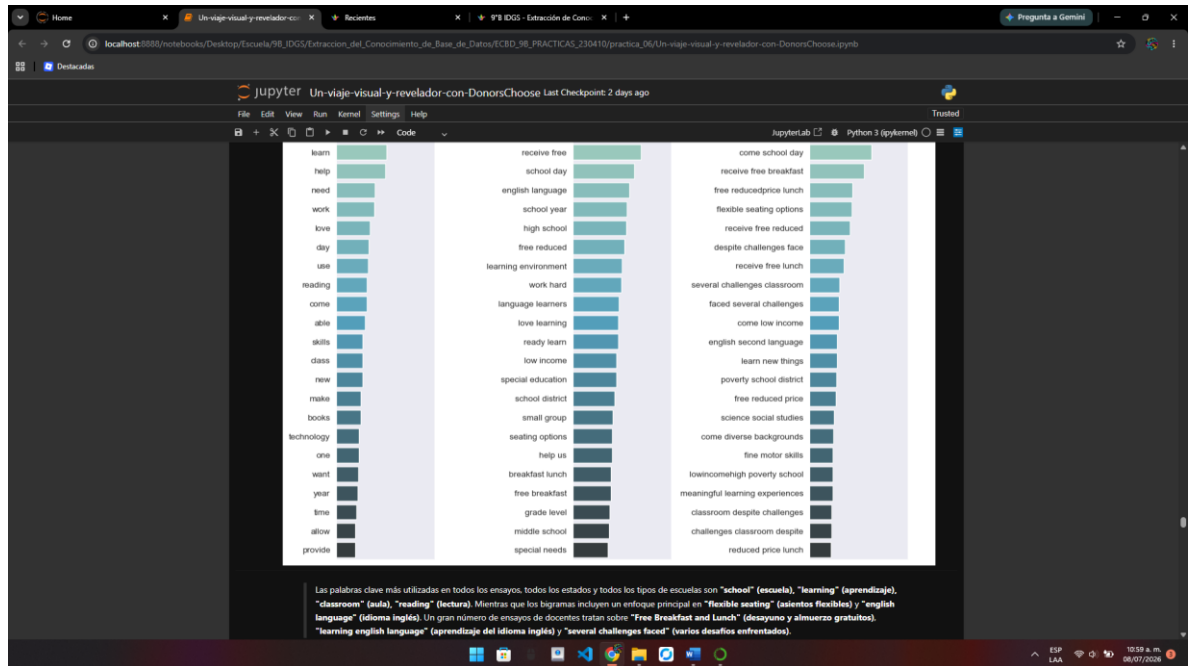
all_bigrams = []
for each in april18_df['bigram']:
    all_bigrams.extend(each)
t1 = Counter(all_bigrams).most_common(20)
x1 = [a[0] for a in t1]
y1 = [a[1] for a in t1]

all_trigrams = []
for each in april18_df['trigram']:
    all_trigrams.extend(each)
t2 = Counter(all_trigrams).most_common(20)
x2 = [a[0] for a in t2]
y2 = [a[1] for a in t2]

sns.set_style("dark")
fig, axes = plt.subplots(nrows=3, ncols=3, squeeze=True, figsize=(12,10))

bar = sns.barplot(y=x, x=y, ax=axes[0], palette="magma", edgecolor="white");
bar.set(xlabel="", yticks=[]);
```





```
hbar1 = sns.barplot(y=x2, x=y2, ax=axes[4], palette='magma', edgecolor='white');
hbar1.set_xlabel('');
axes[4].set_title('Top used Phrases - Florida');

all_unigrams = []
for each in april18_df[april18_df['School State'] == 'Iowa']['trigrams']:
    all_unigrams.extend([x for x in each if x not in ignores])
t = Counter(all_unigrams).most_common(10)
x = [a[0] for a in t]
y = [a[1] for a in t]

all_bigrams = []
for each in april18_df[april18_df['School State'] == 'New York']['trigrams']:
    all_bigrams.extend([x for x in each if x not in ignores])
t1 = Counter(all_bigrams).most_common(10)
x1 = [a[0] for a in t1]
y1 = [a[1] for a in t1]

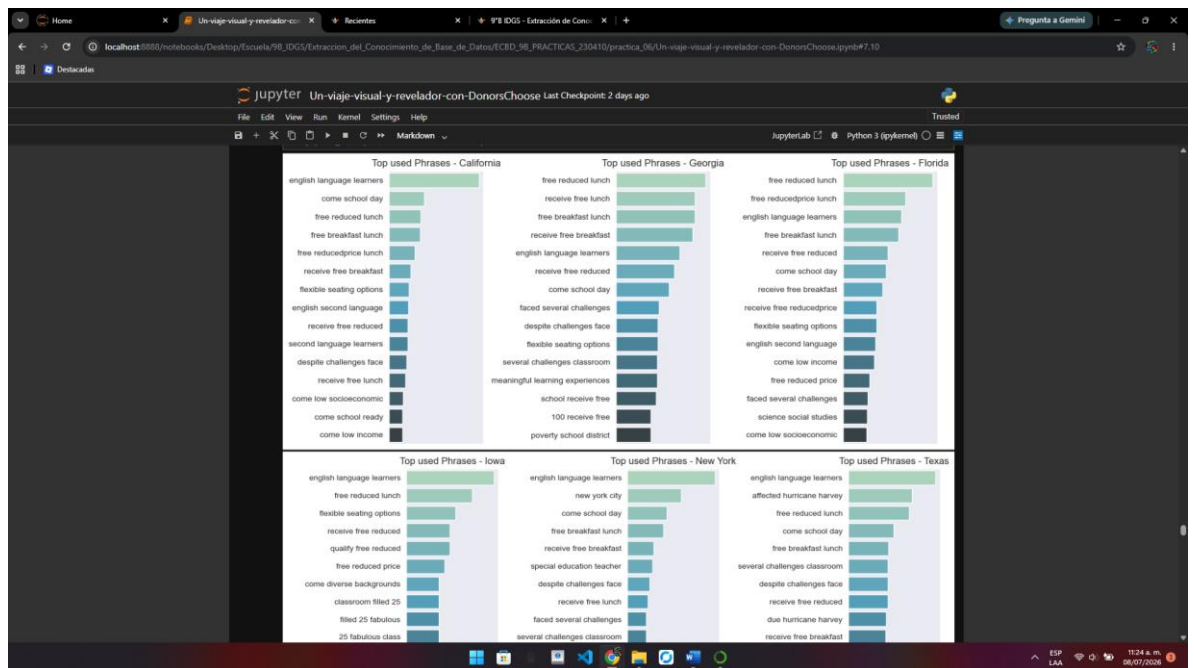
all_trigrams = []
for each in april18_df[april18_df['School State'] == 'Texas']['trigrams']:
    all_trigrams.extend([x for x in each if x not in ignores])
t2 = Counter(all_trigrams).most_common(10)
x2 = [a[0] for a in t2]
y2 = [a[1] for a in t2]

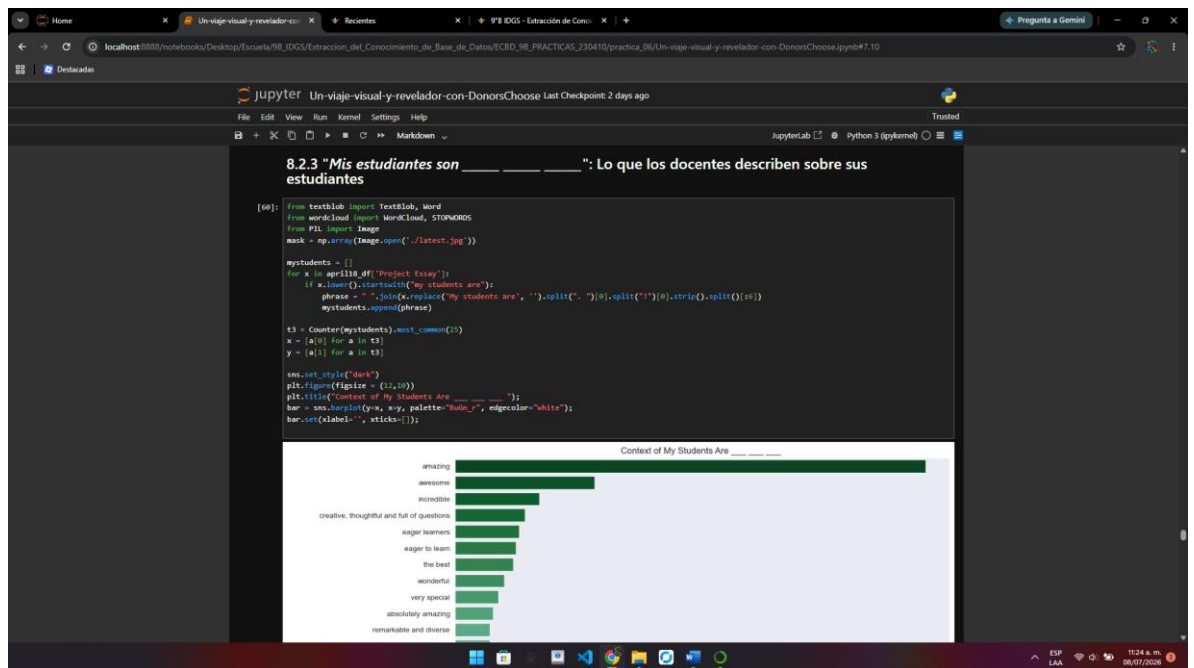
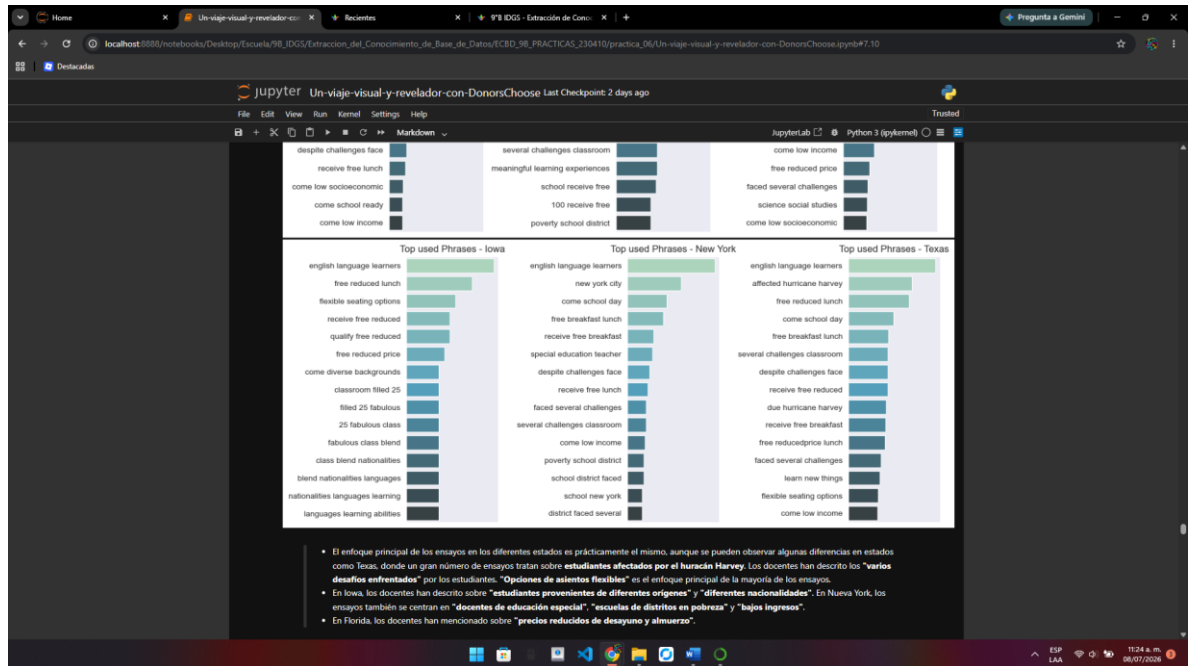
sns.set_style('dark')
fig, axes = plt.subplots(nrows=3, ncols=3, squeeze=True, figsize=(12,9));

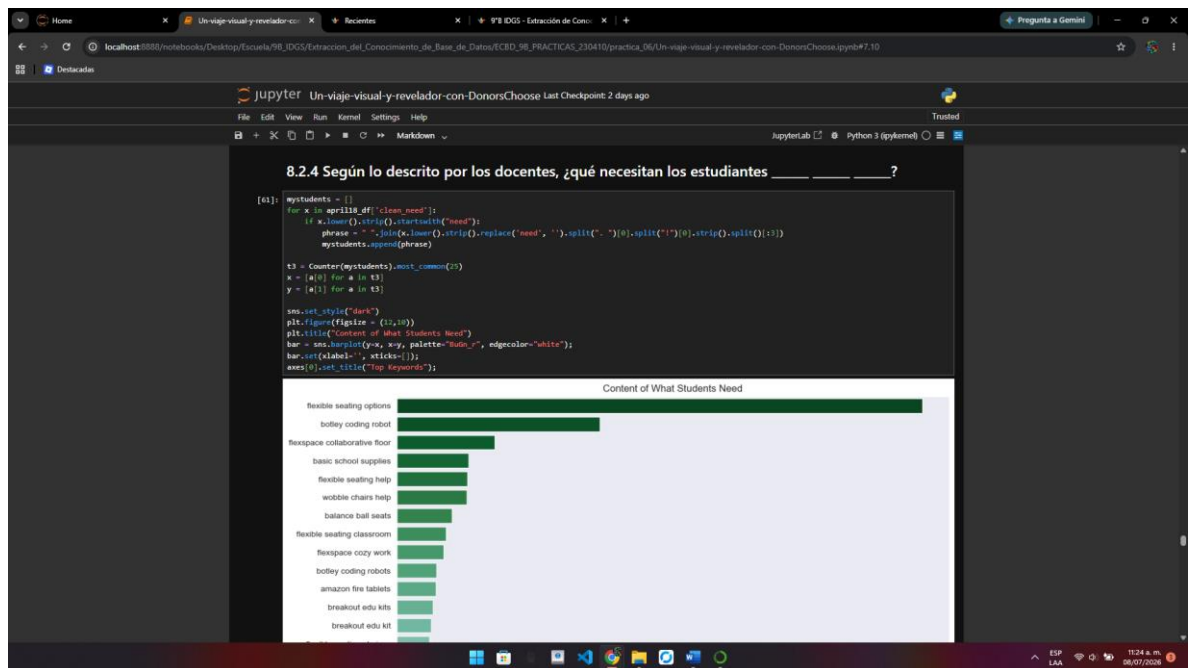
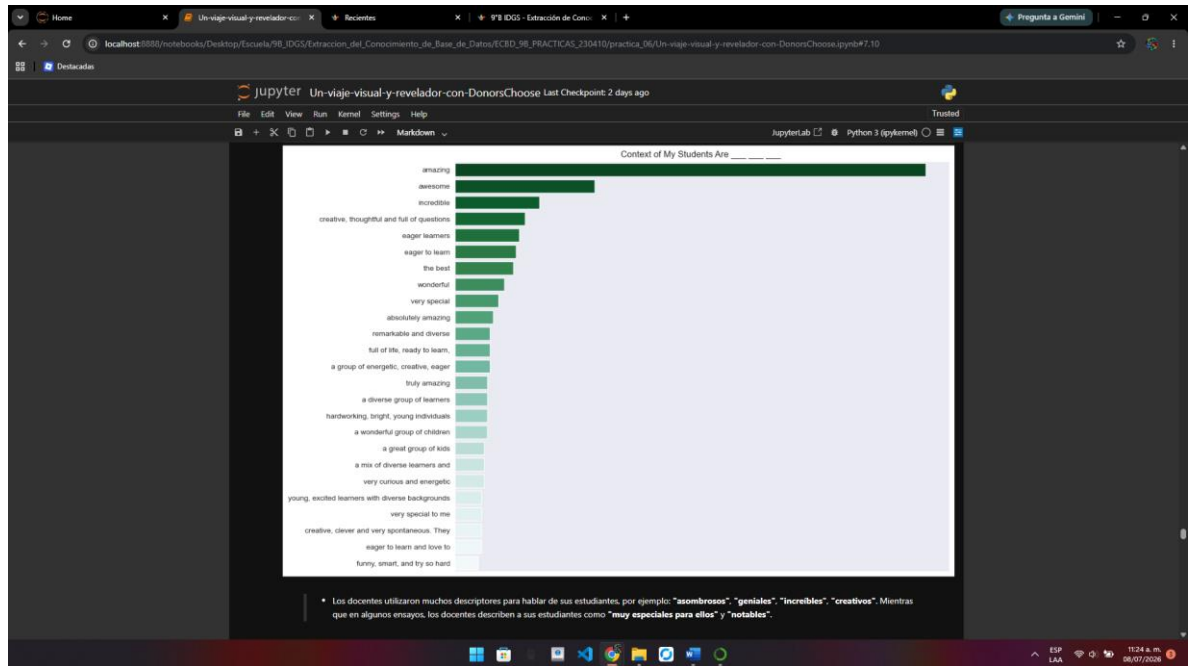
hbar = sns.barplot(y=x, x=y, ax=axes[0], palette='magma', edgecolor='white');
hbar.set_xlabel('');
axes[0].set_title('Top used Phrases - Iowa');
fig.delaxes(axes[1]);

hbar1 = sns.barplot(y=x1, x=y1, ax=axes[2], palette='magma', edgecolor='white');
hbar1.set_xlabel('');
axes[2].set_title('Top used Phrases - New York');
fig.delaxes(axes[3]);

hbar2 = sns.barplot(y=x2, x=y2, ax=axes[4], palette='magma', edgecolor='white');
hbar2.set_xlabel('');
axes[4].set_title('Top used Phrases - Texas');
```








```
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago

opacity=0.9

def trace2 = func(
    x=t2.index,
    y=[float(a)*100/sum(t2.values) for a in t2.values],
    name='Funded',
    marker=dot(color='red204'),
    opacity=0.9,
    showlegend=True
)

return trace1, trace2

# Crear los traces
tr1, tr2 = get_trace('Project Grade Level Category', go.Scatter)
tr3, tr4 = get_trace('School Metro Type', go.Scatter)
tr5, tr6 = get_trace('School State', go.Bar)
tr7, tr8 = get_trace('Project Resource Category', go.Bar)
tr9, tr10 = get_trace('Posted Month', go.Bar, leg=True)

##### Word Count Analysis #####
expired['wc'] = expired['Project Essay'].apply(lambda x: len(x.split()))
funded['wc'] = funded['Project Essay'].apply(lambda x: len(x.split()))

bins = [0, 50, 100, 150, 200, 250, 300, 350, 400, 450, 500, 10000]

# Expired word count
expired['wc'] = pd.cut(expired['wc'], bins)
t1 = dict(expired['wc'].value_counts())
labels = [str(b).replace("'", "").replace(" ", "") for b in t1.keys()]
vals = [float(a)*100/sum(t1.values()) for a in list(t1.values())]
d = {}
for k, v in enumerate(labels):
    d[v] = vals[k]
order = ['0-50', '50-100', '100-150', '150-200', '200-250', '250-300', '300-350', '350-400', '400-450', '450-500', '500-10000']
values = [d[v] for v in order if v in d]

tr13 = go.Bar(
    x=order,
    y=values,
    name='Expired',
    marker=dot(color='red204'),
    opacity=0.9
)

# Funded word count
funded['wc'] = pd.cut(funded['wc'], bins)
t2 = dict(funded['wc'].value_counts())
labels = [str(b).replace("'", "").replace(" ", "") for b in t2.keys()]
vals = [float(a)*100/sum(t2.values()) for a in list(t2.values())]
d = {}
for k, v in enumerate(labels):
    d[v] = vals[k]
values = [d[v] for v in order if v in d]

tr14 = go.Bar(
    x=order,
    y=values,
    name='Funded',
    marker=dot(color='red204'),
    opacity=0.9
)

##### Cost Analysis #####
bins = [0, 100, 500, 1000, 1500, 2000, 2500, 3000, 70000]

# Expired cost
expired['binexpcost'] = pd.cut(expired['cost'], bins)
t1 = dict(expired['binexpcost'].value_counts())
labels = [str(b).replace("'", "").replace(" ", "") for b in t1.keys()]
vals = [float(a)*100/sum(t1.values()) for a in list(t1.values())]
d = {}
for k, v in enumerate(labels):
    d[v] = vals[k]
order = ['100-500', '500-1000', '1000-1500', '1500-2000', '2000-2500', '2500-3000', '3000-70000']
values = [d[v] for v in order if v in d]

tr15 = go.Scatter(
    x=order,
    y=values,
    name='Expired',
    fill='transparent',
    marker=dot(color='red204'),
    opacity=0.9
)

# Funded cost
funded['binexpcost'] = pd.cut(funded['cost'], bins)
t2 = dict(funded['binexpcost'].value_counts())
```

```
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago

# Funded word count
funded['wc'] = pd.cut(funded['wc'], bins)
t2 = dict(funded['wc'].value_counts())
labels = [str(b).replace("'", "").replace(" ", "") for b in t2.keys()]
vals = [float(a)*100/sum(t2.values()) for a in list(t2.values())]
d = {}
for k, v in enumerate(labels):
    d[v] = vals[k]
values = [d[v] for v in order if v in d]

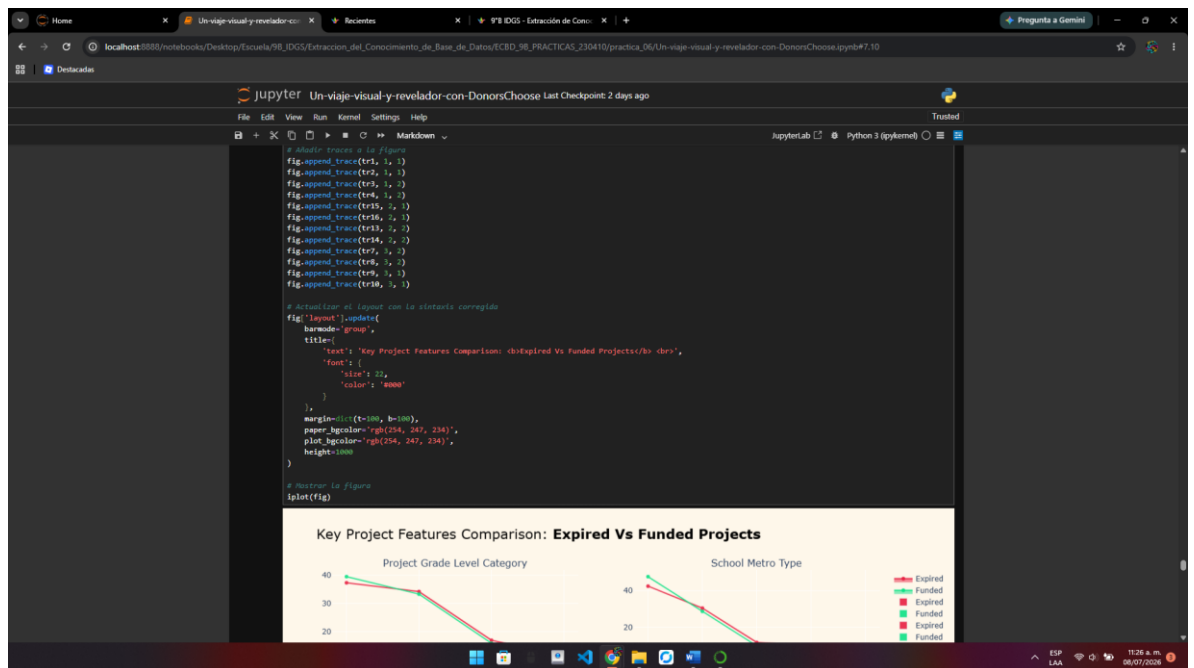
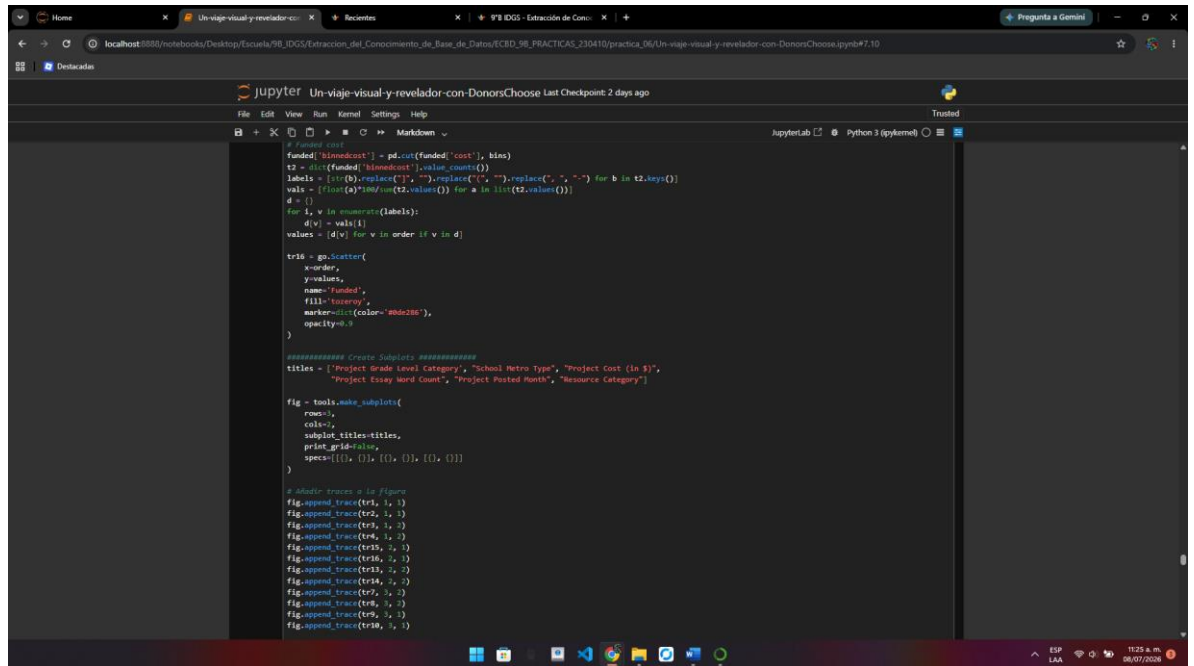
tr14 = go.Bar(
    x=order,
    y=values,
    name='Funded',
    marker=dot(color='red204'),
    opacity=0.9
)

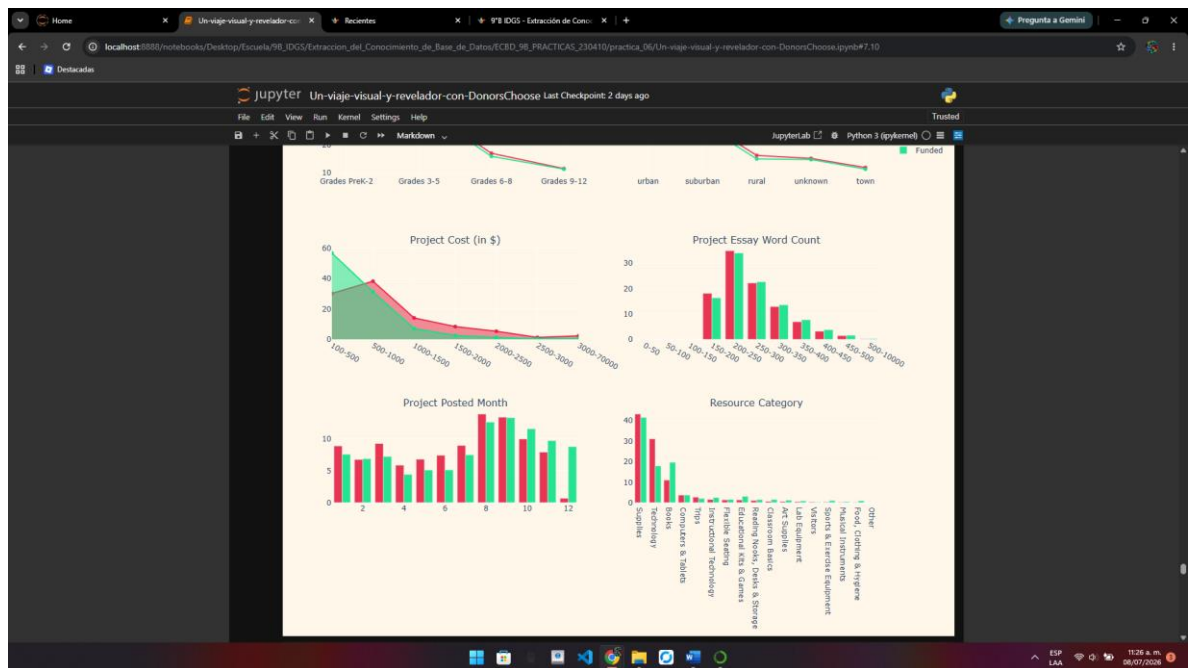
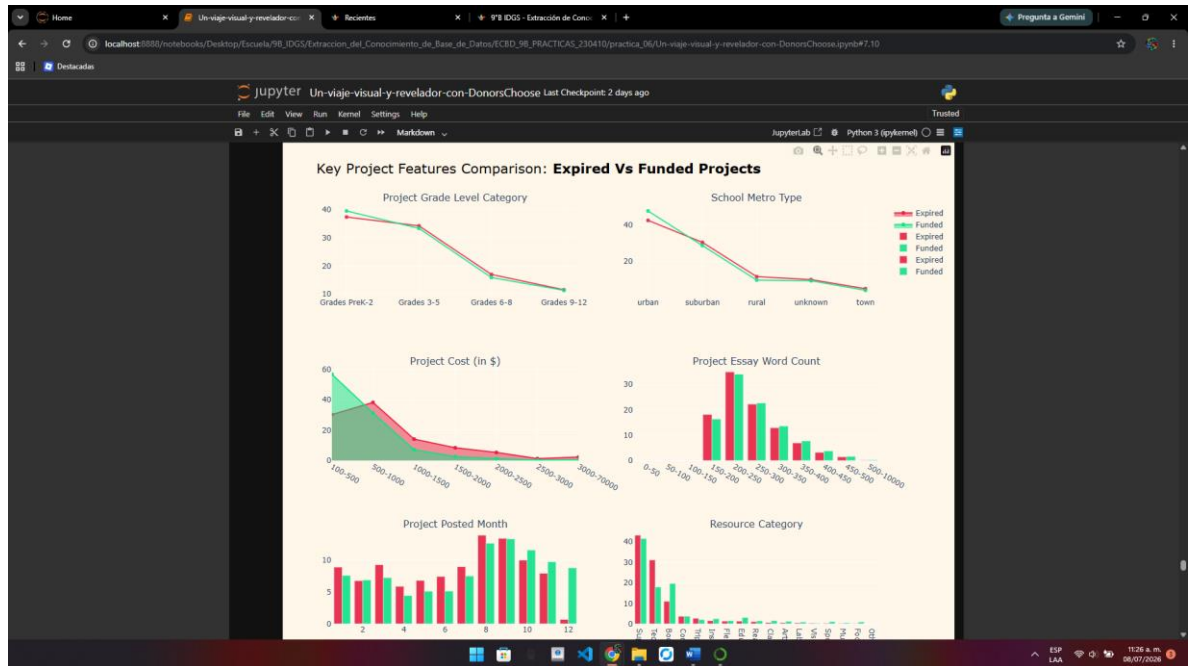
##### Cost Analysis #####
bins = [0, 100, 500, 1000, 1500, 2000, 2500, 3000, 70000]

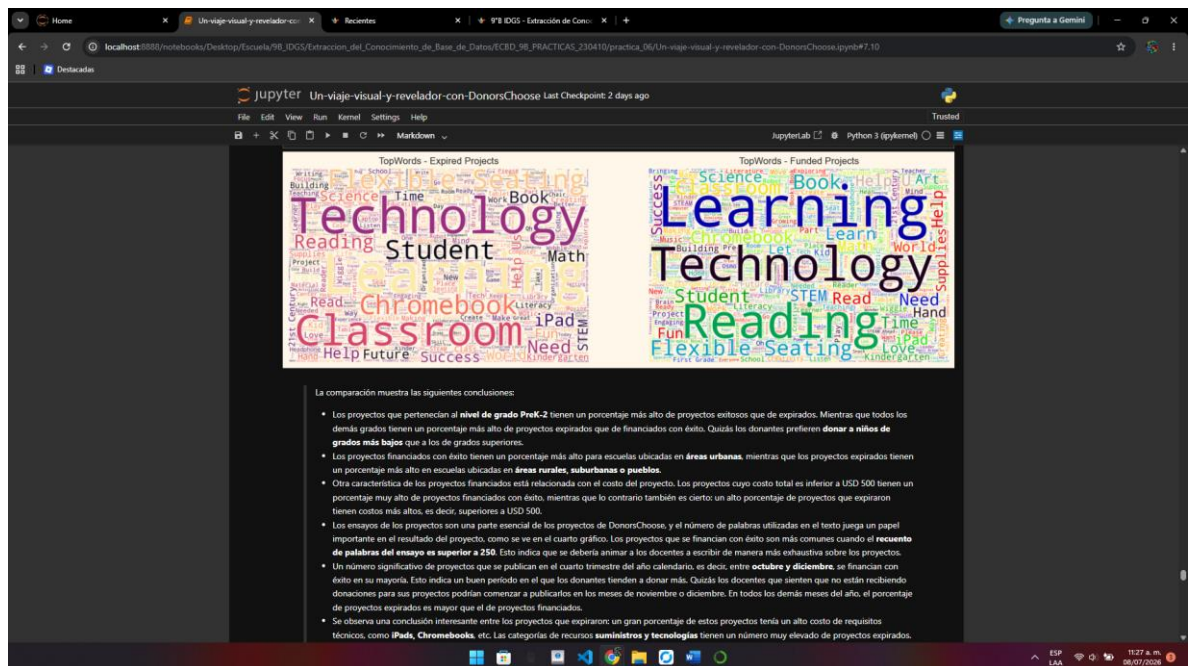
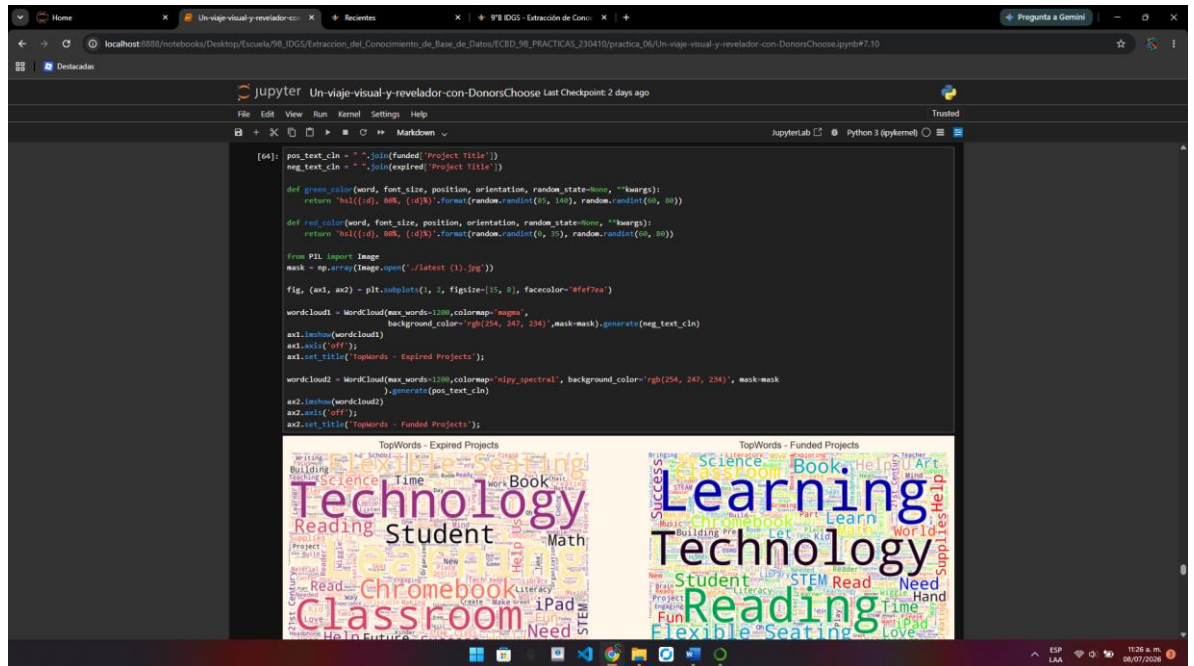
# Expired cost
expired['binexpcost'] = pd.cut(expired['cost'], bins)
t1 = dict(expired['binexpcost'].value_counts())
labels = [str(b).replace("'", "").replace(" ", "") for b in t1.keys()]
vals = [float(a)*100/sum(t1.values()) for a in list(t1.values())]
d = {}
for k, v in enumerate(labels):
    d[v] = vals[k]
order = ['100-500', '500-1000', '1000-1500', '1500-2000', '2000-2500', '2500-3000', '3000-70000']
values = [d[v] for v in order if v in d]

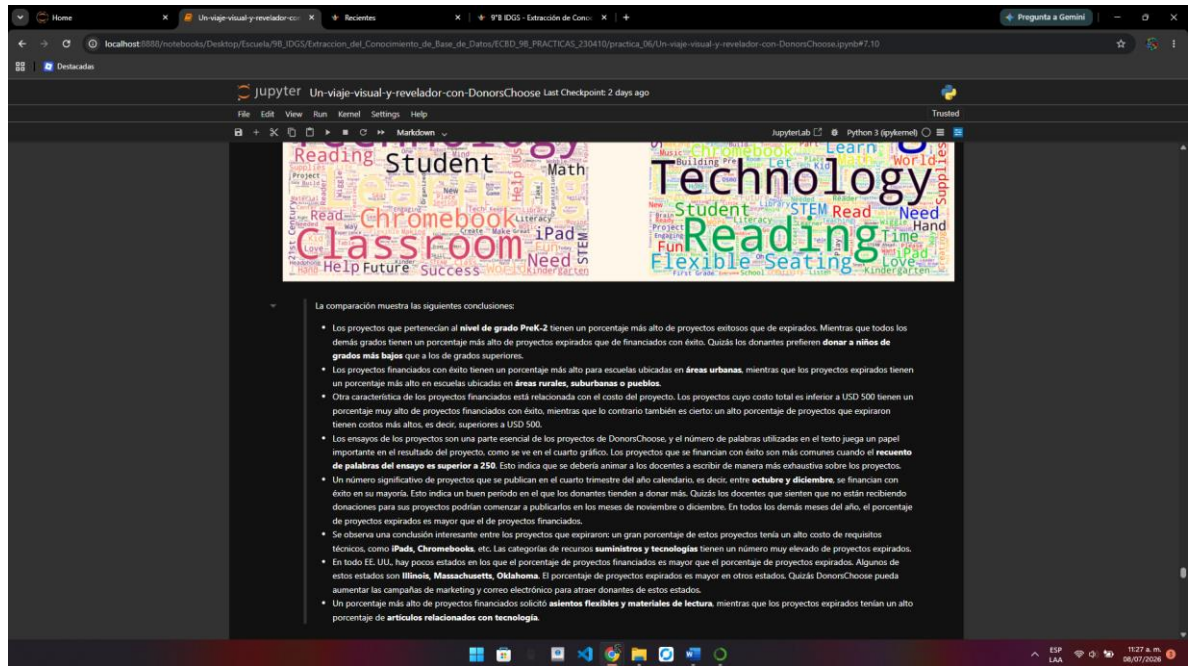
tr15 = go.Scatter(
    x=order,
    y=values,
    name='Expired',
    fill='transparent',
    marker=dot(color='red204'),
    opacity=0.9
)

# Funded cost
funded['binexpcost'] = pd.cut(funded['cost'], bins)
t2 = dict(funded['binexpcost'].value_counts())
```







62. Proyectos con duración de financiación más corta versus más larga

Comparación de proyectos según su duración de financiamiento mediante pandas, Plotly y WordCloud.

```
[65]: projects_df['Funding Time'] = (pd.to_datetime(projects_df['Project Fully Funded Date']) - pd.to_datetime(projects_df['Project Posted Date'])).dt.days
tempdf = projects_df[projects_df['Funding Time'].notna()].copy()

tempdf['time'] = tempdf['Funding Time'].astype(int)
tempdf = tempdf[tempdf['Posted Year'] == 2017]

df1 = tempdf[tempdf['time'] <= 3]
df2 = tempdf[tempdf['time'] >= 60]

df1 = df1.merge(teachers_df, on='Teacher ID')
df2 = df2.merge(teachers_df, on='Teacher ID')

# CORREGIDO: Añadir verificación de columnas
def get_trace(col, func, flag=None, leg=False):
    # Verificar si la columna existe en df1 o df2
    if col not in df1.columns:
        # Buscar columnas alternativas
        for c in df1.columns:
            if col.replace(' ', '') in c.replace(' ', ''):
                col = c
                print(f"Usando columna: {col}")
                break
    else:
        print(f"⚠️ Columna '{col}' no encontrada. Creando traza vacía.")
    trace1 = func(df1[col].y[1:], name='Small Funding Duration', showlegend=leg)
    trace2 = func(df2[col].y[1:], name='Large Funding Duration', showlegend=leg)
    return trace1, trace2

t1 = df1[col].value_counts()
t2 = df2[col].value_counts()

if flag == 'num':
    t1 = df1[df1['NumProjects'] <= 20][col].value_counts()
    t2 = df2[df2['NumProjects'] <= 20][col].value_counts()

trace1 = func(
    x=t1.index,
```

```
trac1 = func(
    x=t1.index,
    y=[float(a)*100/sum(t1.values) for a in t1.values],
    name='Small Funding Duration',
    marker=dot(color='blue'),
    showlegend=True,
    opacity=0.9
)

trac2 = func(
    x=t2.index,
    y=[float(a)*100/sum(t2.values) for a in t2.values],
    name='Large Funding Duration',
    marker=dot(color='red'),
    opacity=0.9,
    showlegend=True
)

return trac1, trac2

# CORREGIDO: Usar los nombres correctos de columnas después del merge
tr1, tr2 = get_trace('Project Grade Level Category', go.Scatter)
tr3, tr4 = get_trace('School Metro Type', go.Scatter) # cambio aqui
tr5, tr6 = get_trace('School State', go.Bar) # cambio aqui
tr7, tr8 = get_trace('Project Resource Category', go.Bar)
tr9, tr10 = get_trace('Posted Month', go.Bar, leg=True)

#####

df1['wc'] = df1['Project Essay'].apply(lambda x: len(x.split()))
df2['wc'] = df2['Project Essay'].apply(lambda x: len(x.split()))

bins = [0, 50, 100, 150, 200, 250, 300, 350, 400, 450, 500, 10000]

df1['wc'] = pd.cut(df1['wc'], bins)
t1 = df1[df1['wc'].value_counts()]
labels = [str(b).replace(",","").replace(" ","").replace(".", "") for b in t1.keys()]
vals = [float(a)*100/sum(t1.values()) for a in list(t1.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
order = ['0-50', '50-100', '100-150', '150-200', '200-250', '250-300', '300-350', '350-400', '400-450', '450-500', '500-10000']
values = [d[v] for v in order]

tr13 = go.Bar(
    x=order,
    y=values,
```

```
tr13 = go.Bar(
    x=order,
    y=values,
    name='Small Funding Duration',
    marker=dot(color='blue'), showlegend=False,
    opacity=0.9
)

df2['wc'] = pd.cut(df2['wc'], bins)
t2 = df2[df2['wc'].value_counts()]
labels = [str(b).replace(",","").replace(" ","").replace(".", "") for b in t2.keys()]
vals = [float(a)*100/sum(t2.values()) for a in list(t2.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
order = ['0-50', '50-100', '100-150', '150-200', '200-250', '250-300', '300-350', '350-400', '400-450', '450-500', '500-10000']
values = [d[v] for v in order]

tr14 = go.Bar(
    x=order,
    y=values,
    name='Large Funding Duration', showlegend=False,
    opacity=0.9
)

##### corr #####

bins = [0, 100, 500, 1000, 1500, 2000, 2500, 3000, 70000]

df1['bincount'] = pd.cut(df1['cont'], bins)
t1 = df1[df1['bincount'].value_counts()]
labels = [str(b).replace(",","").replace(" ","").replace(".", "") for b in t1.keys()]
vals = [float(a)*100/sum(t1.values()) for a in list(t1.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
order = ['100-500', '500-1000', '1000-1500', '1500-2000', '2000-2500', '2500-3000', '3000-70000']
values = [d[v] for v in order]

tr15 = go.Scatter(
    x=order,
```



```
tr15 = g1.Scatter(
    x=order,
    y=values,
    name='Small Funding Time',fill='tozero',
    marker=100(color='red204'),showlegend=False,
    opacity=0.9
)

df2['binnedcost'] = pd.cut(df2['cost'], bins)
t2 = df2['binnedcost'].value_counts()
labels = [v1(b).replace(" ", "-").replace(" ", "-") for b in t1.keys()]
vals = [t2(a)*100/sum(t2.values()) for a in list(t2.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
values = [d[v] for v in order]

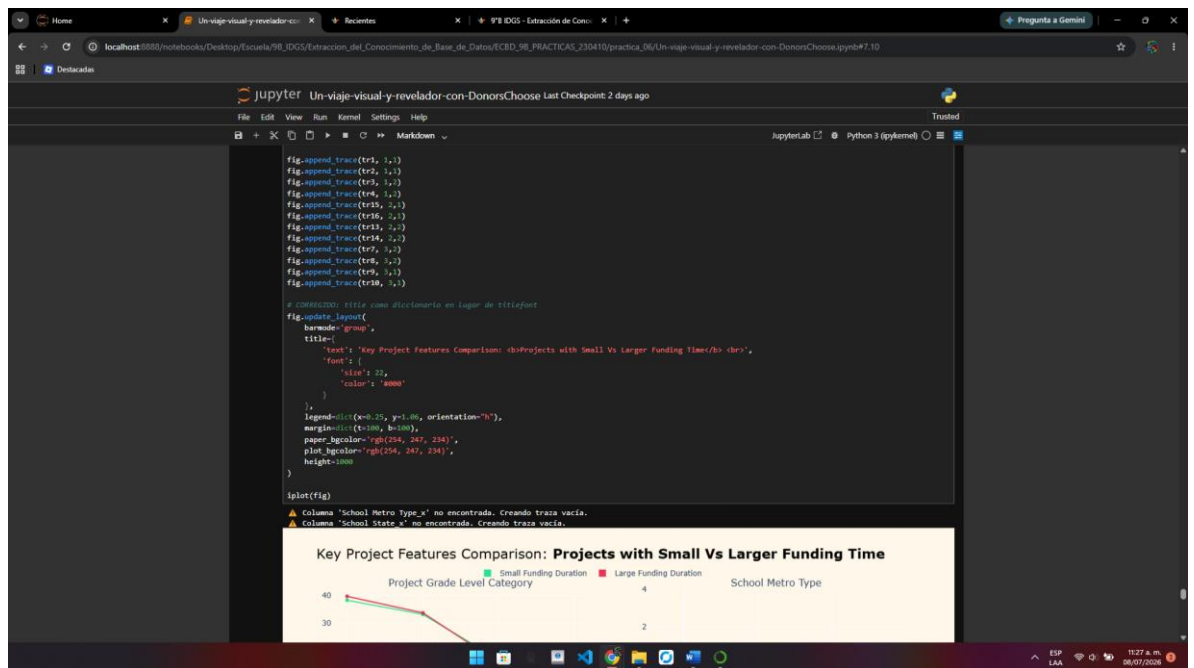
tr16 = g1.Scatter(
    x=order,
    y=values,
    name='Large Funding Time',fill='tozero',
    marker=100(color='red204'),showlegend=False,
    opacity=0.9
)

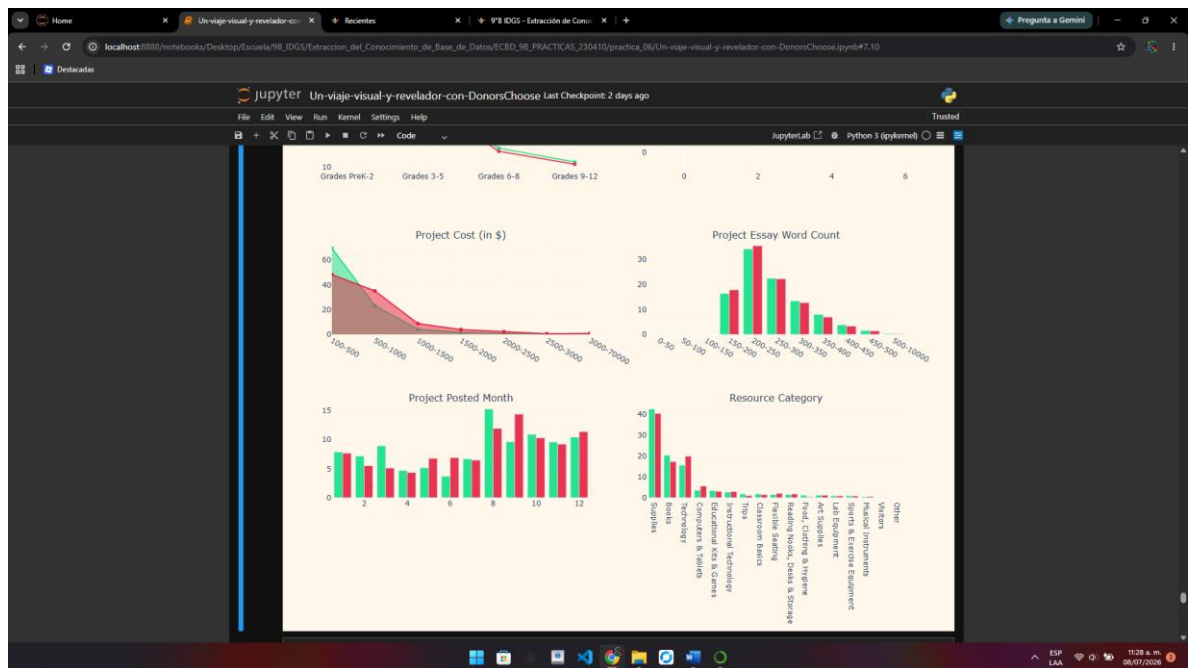
#####

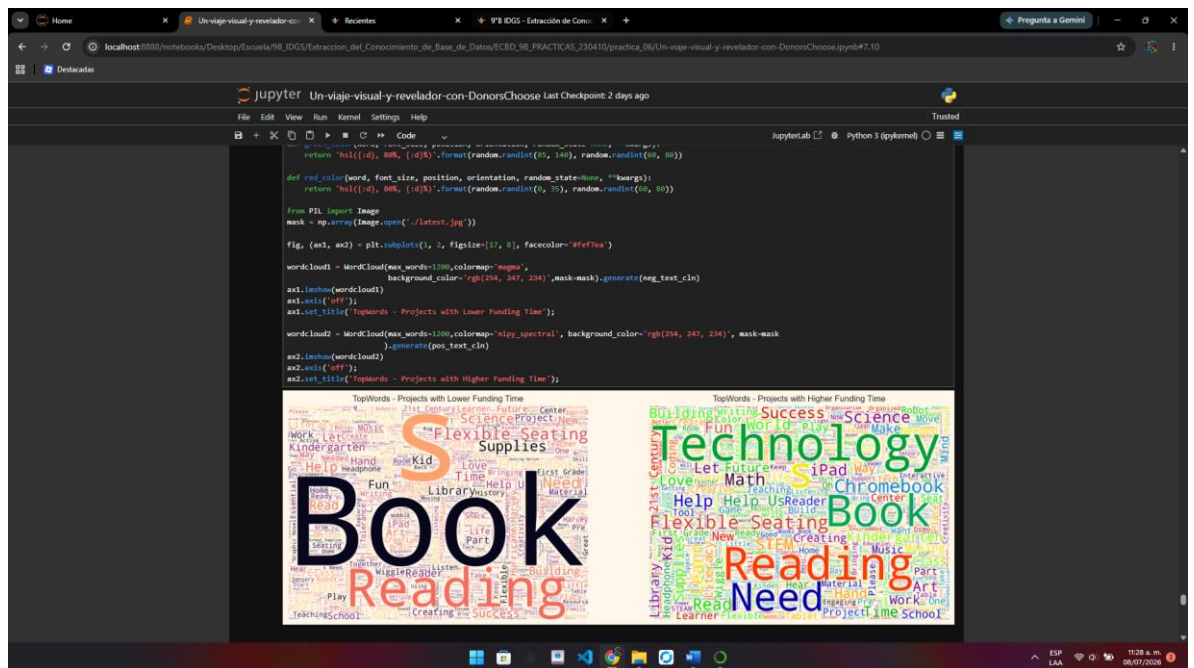
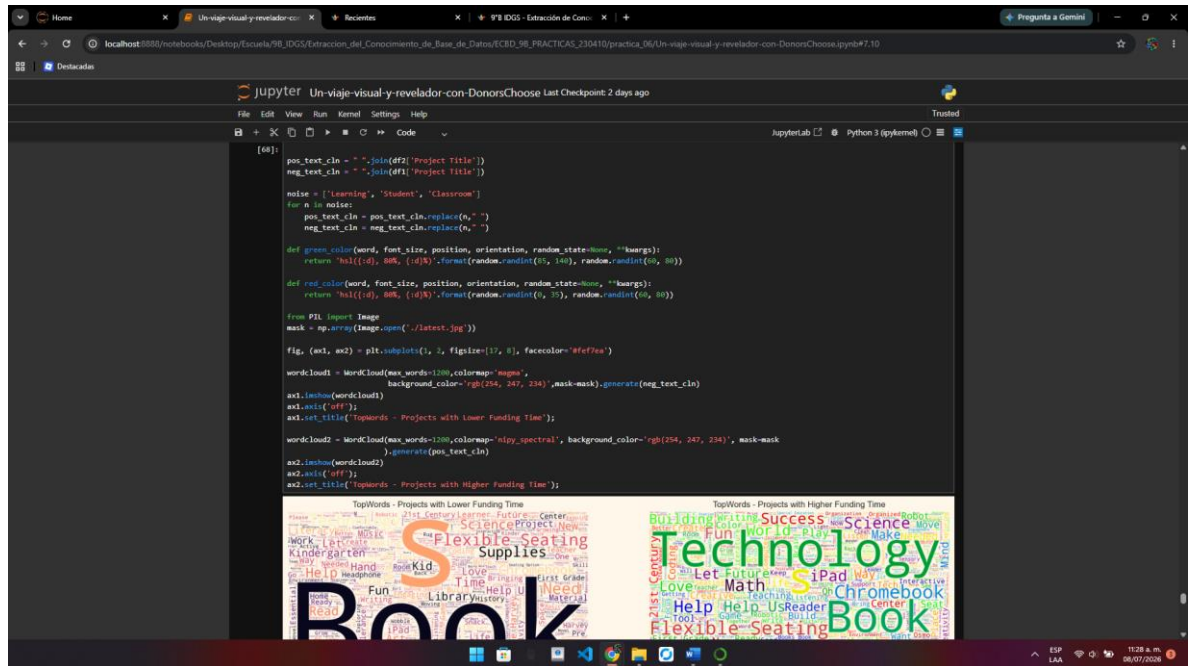
titles = ['Project Grade Level Category', 'School Metro Type', 'Project Cost (in $)',
          'Project Essay Word Count', 'Project Posted Month', 'Resource Category']

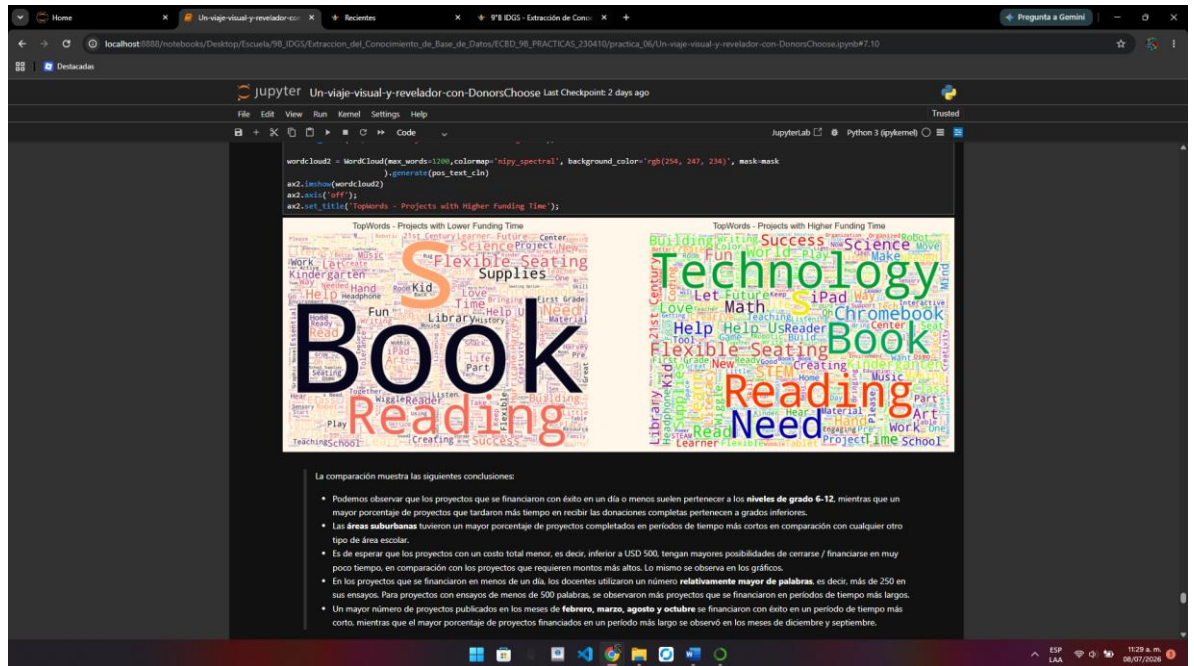
# CORRECCION: usar make_subplots en lugar de tools.make_subplots
from plotly.subplots import make_subplots
fig = make_subplots(rows=1, cols=1, subplot_titles=titles,
                    specs=[[{"x": 1}, {"x": 1}, {"x": 1}, {"x": 1}]]

fig.append_trace(tr15, 1,1)
fig.append_trace(tr16, 1,1)
fig.append_trace(tr2, 1,1)
fig.append_trace(tr3, 1,1)
fig.append_trace(tr4, 1,1)
fig.append_trace(tr5, 1,1)
fig.append_trace(tr6, 1,1)
fig.append_trace(tr7, 1,1)
fig.append_trace(tr8, 1,1)
fig.append_trace(tr9, 1,1)
fig.append_trace(tr10, 1,1)
```



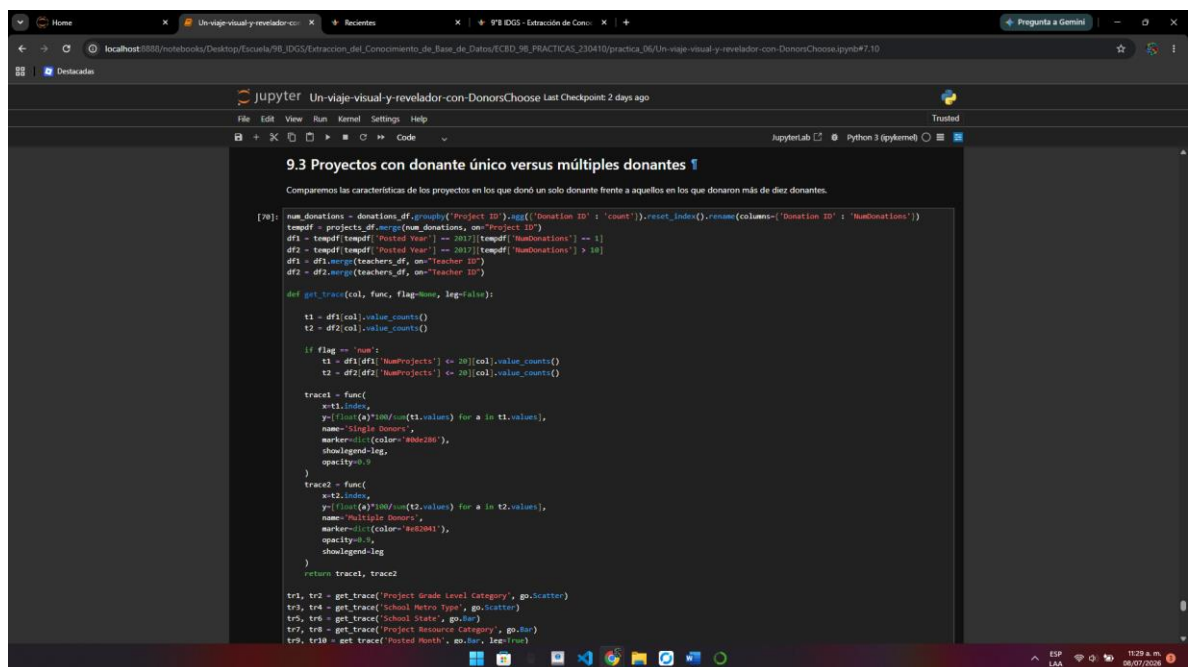






63. Proyectos con donante único versus múltiples donantes

Comparación entre proyectos con un solo donante y múltiples donantes utilizando `groupby()`, `Plotly` y `WordCloud`.



```
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago

File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

#####
df1['ec'] = df1['Project Essay'].apply(lambda x: len(x.split()))
df2['ec'] = df2['Project Essay'].apply(lambda x: len(x.split()))

bins = [0, 50, 100, 150, 200, 250, 300, 350, 400, 450, 500, 10000]

df1['ec'] = pd.cut(df1['ec'], bins)
t1 = df1[df1['ec'].value_counts()]
labels = [str(b).replace("'", "").replace(" ", "").replace(" ", ".") for b in t1.keys()]
vals = [float(a)*100/sum(t1.values()) for a in list(t1.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
order = ['0-50', '50-100', '100-150', '150-200', '200-250', '250-300', '300-350', '350-400', '400-450', '450-500', '500-10000',]
values = [d[v] for v in order]

tr13 = go.Bar(
    x=order,
    y=values,
    name='Single Donors',
    marker=dict(color='red208'), showlegend=False,
    opacity=0.5
)

df2['ec'] = pd.cut(df2['ec'], bins)
t2 = df2[df2['ec'].value_counts()]
labels = [str(b).replace("'", "").replace(" ", "").replace(" ", ".") for b in t2.keys()]
vals = [float(a)*100/sum(t2.values()) for a in list(t2.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
order = ['0-50', '50-100', '100-150', '150-200', '200-250', '250-300', '300-350', '350-400', '400-450', '450-500', '500-10000',]
values = [d[v] for v in order]

tr15 = go.Bar(
    x=order,
    y=values,
    name='Multiple Donors',
    marker=dict(color='red208'), showlegend=False,
    opacity=0.5
)
```

```
Un-viaje-visual-y-revelador-con-DonorsChoose Last Checkpoint 2 days ago

File Edit View Run Kernel Settings Help Trusted
JupyterLab Python 3 (ipykernel)

##### cut #####
bins = [0, 100, 500, 1000, 1500, 2000, 2500, 3000, 70000]

df1['blimedcost'] = pd.cut(df1['cost'], bins)
t1 = df1[df1['blimedcost'].value_counts()]
labels = [str(b).replace("'", "").replace(" ", "").replace(" ", ".") for b in t1.keys()]
vals = [float(a)*100/sum(t1.values()) for a in list(t1.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
order = ['100-500', '500-1000', '1000-1500', '1500-2000', '2000-2500', '2500-3000', '3000-70000',]
values = [d[v] for v in order]

tr15 = go.Scatter(
    x=order,
    y=values,
    name='Single Donors', fill='tozero',
    marker=dict(color='red208'), showlegend=False,
    opacity=0.5
)

df2['blimedcost'] = pd.cut(df2['cost'], bins)
t2 = df2[df2['blimedcost'].value_counts()]
labels = [str(b).replace("'", "").replace(" ", "").replace(" ", ".") for b in t2.keys()]
vals = [float(a)*100/sum(t2.values()) for a in list(t2.values())]
d = {}
for i, v in enumerate(labels):
    d[v] = vals[i]
values = [d[v] for v in order]

tr16 = go.Scatter(
    x=order,
    y=values,
    name='Multiple Donors', fill='tozero',
    marker=dict(color='red208'), showlegend=False,
    opacity=0.5
)

#####
```